

Create a file named:

SWAP-INDIA-IMPLEMENTATION-SPEC.md

Give this file to your AI coding agent along with the existing repository.

MARKDOWN

Swap India

AI Coding Agent Implementation Specification

Version: 1.0

Product: Mobile-first Indian train seat coordination PWA

Initial market: India

Initial transport mode: Reserved Indian trains

Primary users: Families, groups, elderly passengers, students, tour groups, and travelers seeking better berth arrangements

1. Product Definition

Swap India helps passengers visually understand their train coach, find compatible passengers, and coordinate possible berth exchanges.

The product is not an official railway reservation system.

The application must never imply that:

- A passenger can sell or permanently transfer a railway seat.
- An in-app agreement automatically changes a railway reservation.
- An unregistered berth is available.
- A passenger is officially verified only because they uploaded a ticket image.
- A swap is guaranteed.
- The app has official railway approval unless a real authorized integration exists.

Preferred product language:

> See your berth. Find a better fit. Coordinate safely.

Core user flow:

```text

Create journey

→ Select train and date

→ Select boarding and destination stations

→ Enter coach and berth

→ View visual coach map

→ Choose preferences

→ Publish availability

→ Find compatible passengers

→ Compare proposed seats visually

→ Send request

→ Both passengers accept

→ Revalidate before departure

→ Coordinate with railway staff when required

## 2. Product Goals

### Primary goals

1. Make train berths understandable through visual maps.
2. Help families and groups coordinate seats.
3. Enable safe passenger-to-passenger swap requests.
4. Build journey-specific matching density.
5. Provide accurate, transparent seat-map confidence.
6. Work well on low-cost Android devices and slow Indian mobile networks.
7. Support English and Hindi at launch.
8. Create a viral group-invitation loop.
9. Monetize coordination convenience without charging for basic participation.
10. Build a provider-independent architecture for future railway and flight integrations.

### Non-goals for the first release

Do not build initially:

- Official railway ticket transfers.
- Seat resale or auctioning.
- Passenger-to-passenger payments.
- Three-way swaps.
- Automated booking changes.
- Universal live seat availability.
- Public passenger directories.
- Flight support.
- Complex ticket OCR.
- Full railway account login.

- Guaranteed seat upgrades.
- Anonymous public train chat rooms.

### 3. Target Users

#### Persona A: Family traveler

Needs:

- Seats together.
- Lower berth for elderly members.
- Children near parents.
- One visual family view.

#### Persona B: Elderly or accessibility-focused traveler

Needs:

- Lower berth.
- Easier access.
- Fewer stairs.
- Near family or companion.

Do not require public disclosure of medical information.

#### Persona C: Group traveler

Examples:

- Wedding group.
- College trip.
- Religious trip.
- Tour group.
- Company travel group.

Needs:

- Invite link.
- Group roster.
- Coach visualization.
- Seating coordination.
- Group admin controls.

## Persona D: Individual comfort seeker

Needs:

- Lower instead of upper berth.
- Side berth preference.
- Same coach.
- Away from toilet or entrance.
- Better proximity to companion.

## Persona E: Tour or travel organizer

Needs:

- Manage many passengers.
- Import or collect berth details.
- See group distribution.
- Send reminders.
- Coordinate changes.

---

## 4. Launch Strategy

Start with a concentrated network rather than nationwide availability.

### Initial transport scope

Support:

- Sleeper Class.

- AC 3 Tier.
- AC 2 Tier.
- Selected coach types.
- Curated layouts.
- Manual journey entry.
- A limited route set.

## Recommended pilot strategy

Select 3–5 high-density route clusters based on:

- Long travel duration.
- High family travel.
- High group travel.
- Frequent split-berth problems.
- Strong digital adoption.
- Availability of reliable train data.

Do not build the system around hardcoded routes. Routes must be configurable in the admin panel.

---

## 5. Technical Architecture

### Recommended stack

Use the existing repository stack if viable. If greenfield:

#### Frontend

- Next.js or React.
- TypeScript.
- CSS modules, Tailwind, or the existing design system.
- SVG seat-map renderer.
- React Query or equivalent server-state library.

- IndexedDB for offline journey data.
- Service worker.
- Web app manifest.
- Accessible component primitives.

## Backend

- Node.js with TypeScript.
- REST or typed RPC API.
- PostgreSQL.
- Redis for rate limiting, queues, locks, and short-lived matching jobs.
- Background worker.
- Object storage for temporary evidence uploads.
- Admin dashboard.
- Structured audit logging.

## Infrastructure

- Managed PostgreSQL.
- Managed Redis.
- Managed object storage.
- CDN.
- Error monitoring.
- Application logs.
- Database backups.
- CI/CD pipeline.
- Preview environments.

## Architecture principles

1. Keep seat-map geometry separate from live passenger data.
2. Keep railway providers behind adapter interfaces.
3. Keep matching rules server-side.

4. Use database constraints for conflicting commitments.
5. Treat all availability as time-sensitive.
6. Never trust client-side authorization.
7. Make uncertain data visibly uncertain.
8. Design for localization from the beginning.
9. Keep sensitive journey evidence private.
10. Make every important action auditable.

## 6. Repository Structure

Use this structure unless the existing repository has a better equivalent:

```
TEXT
src/
 app/
 page.tsx
 journeys/
 page.tsx
 new/
 page.tsx
 [journeyId]/
 page.tsx
 map/
 page.tsx
 preferences/
 page.tsx
 matches/
 page.tsx
 requests/
 page.tsx
 [requestId]/
 page.tsx
 inbox/
 page.tsx
 profile/
 page.tsx
 help/
 page.tsx
 legal/
 page.tsx
 privacy/
 page.tsx
 terms/
 page.tsx

 components/
 layout/
 navigation/
 journey/
```

|                         |
|-------------------------|
| train-search/           |
| seat-map/               |
| SeatMap.tsx             |
| CoachStrip.tsx          |
| CoachMap.tsx            |
| Berth.tsx               |
| SeatDetailSheet.tsx     |
| BerthElevation.tsx      |
| SeatLegend.tsx          |
| SeatMapToolbar.tsx      |
| SeatListAlternative.tsx |
| matching/               |
| requests/               |
| messaging/              |
| notifications/          |
| forms/                  |
| feedback/               |
| accessibility/          |
| server/                 |
| auth/                   |
| journeys/               |
| trains/                 |
| layouts/                |
| matching/               |
| swap-requests/          |
| messaging/              |
| notifications/          |
| moderation/             |
| payments/               |
| admin/                  |
| lib/                    |
| auth/                   |
| db/                     |
| validation/             |
| permissions/            |
| localization/           |
| dates/                  |
| timezones/              |
| rate-limit/             |
| feature-flags/          |
| analytics/              |
| errors/                 |
| providers/              |
| trains/                 |
| TrainProvider.ts        |
| ManualTrainProvider.ts  |
| ProviderRegistry.ts     |
| notifications/          |
| payments/               |
| data/                   |
| layouts/                |
| sleeper/                |
| ac-3-tier/              |
| ac-2-tier/              |
| workers/                |
| match-worker.ts         |

expire-requests-worker.ts  
notification-worker.ts  
journey-revalidation-worker.ts

types/  
journey.ts  
seat-map.ts  
matching.ts  
requests.ts  
users.ts

tests/  
unit/  
integration/  
e2e/  
accessibility/

---

## 7. Domain Model

### Main entities

**TEXT**  
users  
user\_profiles  
operators  
stations  
trains  
train\_services  
service\_stops  
coach\_types  
layout\_templates  
layout\_versions  
coach\_instances  
berths  
traveler\_journeys  
journey\_members  
seat\_claims  
swap\_preferences  
match\_candidates  
swap\_requests  
swap\_events  
messages  
notifications  
reports  
blocks  
payments  
admin\_actions  
analytics\_events

### Important concepts

## Physical berth

The actual physical location in a coach.

## Journey seat claim

A user's reported or verified use of a berth for a specific journey segment.

## Preference

What the user wants.

## Swap request

A proposed exchange between two travelers.

## Official reservation

Railway-owned reservation data. This must not be represented as changed unless an authorized integration confirms it.

## 8. Database Schema

Use PostgreSQL.

### users

**SQL**

```
create table users (
 id uuid primary key,
 phone_hash text unique not null,
 phone_verified_at timestampz,
 email text,
 created_at timestampz not null default now(),
 updated_at timestampz not null default now(),
 deleted_at timestampz
);
```

Do not store plain phone numbers unless operationally required. If stored, encrypt them and restrict access.

### user\_profiles

**SQL**

```
create table user_profiles (
```

```
user id uuid primary key references users(id),
display name text not null,
preferred language text not null default 'en',
avatar url text,
allow group invites boolean not null default true,
notification preferences jsonb not null default '{}',
created at timestamptz not null default now(),
updated at timestamptz not null default now()
);
```

## stations

```
SQL
create table stations (
 id uuid primary key,
 code text not null,
 name text not null,
 normalized name text not null,
 city text,
 state text,
 country code text not null default 'IN',
 latitude numeric,
 longitude numeric,
 aliases jsonb not null default '[]',
 created at timestamptz not null default now()
);
```

## trains

```
SQL
create table trains (
 id uuid primary key,
 operator name text not null,
 train number text not null,
 train name text,
 active boolean not null default true,
 metadata jsonb not null default '{}',
 unique(operator name, train number)
);
```

## train\_services

```
SQL
create table train_services (
 id uuid primary key,
 train id uuid not null references trains(id),
 departure date date not null,
 service identifier text not null,
 source text not null,
 confidence text not null,
 status text not null default 'scheduled',
 created at timestamptz not null default now(),
 unique(train id, departure date, service identifier)
```

```
);
```

## service\_stops

SQL

```
create table service_stops (
 id uuid primary key,
 service_id uuid not null references train_services(id),
 station_id uuid not null references stations(id),
 stop_index integer not null,
 arrival_time timestampz,
 departure_time timestampz,
 unique(service_id, stop_index)
);
```

## layout\_templates

SQL

```
create table layout_templates (
 id uuid primary key,
 transport text not null default 'train',
 class_code text not null,
 coach_type text not null,
 operator_name text,
 name text not null,
 width numeric not null,
 height numeric not null,
 confidence text not null,
 source text not null,
 active boolean not null default true,
 created_at timestampz not null default now()
);
```

## layout\_versions

SQL

```
create table layout_versions (
 id uuid primary key,
 layout_template_id uuid not null references layout_templates(id),
 version integer not null,
 geometry jsonb not null,
 fixtures jsonb not null default '[]',
 notes text,
 reviewed_by uuid references users(id),
 reviewed_at timestampz,
 created_at timestampz not null default now(),
 unique(layout_template_id, version)
);
```

## traveler\_journeys

**SQL**

```
create table traveler_journeys (
 id uuid primary key,
 user id uuid not null references users(id),
 service id uuid not null references train_services(id),
 boarding_stop_index integer not null,
 destination_stop_index integer not null,
 class_code text not null,
 coach_code text,
 berth_code text,
 verification_level text not null default 'manual',
 visibility text not null default 'matches_only',
 status text not null default 'active',
 created_at timestamptz not null default now(),
 updated_at timestamptz not null default now()
);
```

**Constraints:****TEXT**

```
boarding_stop_index < destination_stop_index
journey_user must own or belong to the journey
journeys cannot be matched across different services
```

**swap\_preferences****SQL**

```
create table swap_preferences (
 id uuid primary key,
 traveler_journey_id uuid not null references traveler_journeys(id),
 desired_features jsonb not null default '[]',
 acceptable_features jsonb not null default '[]',
 preferred_coach text,
 preferred_bay text,
 notes text,
 active boolean not null default true,
 created_at timestamptz not null default now()
);
```

**swap\_requests****SQL**

```
create table swap_requests (
 id uuid primary key,
 from_journey_id uuid not null references traveler_journeys(id),
 to_journey_id uuid not null references traveler_journeys(id),
 from_berth_code text not null,
 to_berth_code text not null,
 status text not null default 'pending',
 expires_at timestamptz not null,
 requester_message text,
 accepted_at timestamptz,
 cancelled_at timestamptz,
 completed_at timestamptz,
```

```
 created at timestamptz not null default now()
);
```

Use transactional locking when accepting requests.

## 9. Seat Map Data Format

TS

```
type LayoutConfidence =
 | "verified"
 | "expected"
 | "user_confirmed"
 | "illustrative";
```

```
type BerthFeature =
 | "lower"
 | "middle"
 | "upper"
 | "side_lower"
 | "side_upper"
 | "window side"
 | "near door"
 | "near toilet"
 | "easy_access";
```

```
interface SeatMap {
 id: string;
 version: number;
 transport: "train";
 classCode: string;
 coachType: string;
 width: number;
 height: number;
 confidence: LayoutConfidence;
 source: string;
 lastReviewedAt?: string;
 berths: Berth[];
 fixtures: Fixture[];
}
```

```
interface Berth {
 id: string;
 label: string;
 bayId?: string;
 x: number;
 y: number;
 width: number;
 height: number;
 level?: number;
 features: BerthFeature[];
 restrictions: string[];
}
```

```
interface Fixture {
 type: "door" | "toilet" | "aisle" | "stairs" | "wall";
```

```
x: number;
y: number;
width: number;
height: number;
}
```

The renderer must not infer availability from geometry.

## 10. Train Data Provider Architecture

Use adapters:

**TS**

```
interface TrainProvider {
 searchTrains(input: TrainSearchInput): Promise<TrainSearchResult[]>;
 resolveService(input: ResolveServiceInput): Promise<ResolvedService>;
 resolveStops(input: ResolveStopsInput): Promise<ServiceStop[]>;
 resolveCoachLayout(input: ResolveCoachLayoutInput):
 Promise<LayoutResolution>;
}
```

Initial provider:

**TEXT**

```
ManualTrainProvider
```

It should support:

- Train number.
- Train name.
- Date.
- Stations.
- Class.
- Coach type.
- Layout template.
- Confidence status.

Future providers must be added without changing UI components.

### Layout resolution response

**TS**

```
interface LayoutResolution {
 layoutTemplateId: string;
 confidence: "verified" | "expected" | "user_confirmed" | "illustrative";
 source: string;
 lastCheckedAt?: string;
 requiresUserConfirmation: boolean;
}
```

If `requiresUserConfirmation` is true, show a confirmation step before allowing matching.

## 11. Matching Engine

Start with deterministic rules.

### Eligibility

Two journey records are eligible when:

**TEXT**

- same train service
- same travel date
- same class or explicitly compatible class
- overlapping journey segments
- both journeys active
- both users opted into matching
- both users have valid seat claims
- no active conflicting request
- no blocked relationship
- no moderation restriction

Segment overlap:

**TS**

```
const overlaps =
 Math.max(a.boardingStopIndex, b.boardingStopIndex) <
 Math.min(a.destinationStopIndex, b.destinationStopIndex);
```

For MVP, prefer exact same boarding and destination stations.

### Ranking

**TEXT**

- same coach: +40
- same bay: +30
- desired berth feature satisfied: +25
- group proximity improved: +25
- accessibility preference satisfied: +25
- same boarding and destination: +20
- verified journey: +15

short physical distance: +10  
time remaining above threshold: +5

Never rank a listing as available merely because a passenger has not joined.

## Match statuses

**TEXT**  
candidate  
hidden  
shown  
request\_sent  
rejected  
expired  
blocked

## Matching worker

Run matching:

- When a journey becomes active.
- When preferences change.
- When a new journey is added.
- When a request is cancelled.
- When a journey is revalidated.
- On scheduled background intervals for upcoming journeys.

---

## 12. Swap Request State Machine

**TEXT**  
draft  
→ pending  
→ accepted\_by\_recipient  
→ accepted\_by\_both  
→ awaiting\_external\_confirmation  
→ completed

Alternative endings:

**TEXT**  
cancelled  
expired  
declined

disputed  
invalidated

#### Rules:

- Requests must expire.
- A user cannot have unlimited active requests.
- A berth cannot be committed to multiple accepted requests.
- Acceptance must happen inside a database transaction.
- Equipment changes invalidate affected requests.
- Users can withdraw before completion.
- The system must revalidate before final confirmation.
- Offline acceptance must never be treated as final.

## 13. API Design

### Authentication

#### TEXT

```
POST /api/auth/request-otp
POST /api/auth/verify-otp
POST /api/auth/logout
GET /api/auth/session
DELETE /api/account
```

### Train search

#### TEXT

```
GET /api/trains/search?q=
GET /api/trains/:trainNumber
GET /api/services/:serviceId
GET /api/services/:serviceId/stops
```

### Journeys

#### TEXT

```
POST /api/journeys
GET /api/journeys
GET /api/journeys/:journeyId
PATCH /api/journeys/:journeyId
DELETE /api/journeys/:journeyId
```

```
POST /api/journeys/:journeyId/confirm-layout
POST /api/journeys/:journeyId/publish
POST /api/journeys/:journeyId/pause
```

## Seat maps

```
TEXT
GET /api/journeys/:journeyId/seat-map
GET /api/journeys/:journeyId/seat-map/availability
GET /api/journeys/:journeyId/berths/:berthId
```

## Preferences

```
TEXT
PUT /api/journeys/:journeyId/preferences
GET /api/journeys/:journeyId/preferences
```

## Matches

```
TEXT
GET /api/journeys/:journeyId/matches
POST /api/matches/:matchId/hide
POST /api/matches/:matchId/report
```

## Requests

```
TEXT
POST /api/swap-requests
GET /api/swap-requests
GET /api/swap-requests/:requestId
POST /api/swap-requests/:requestId/accept
POST /api/swap-requests/:requestId/decline
POST /api/swap-requests/:requestId/cancel
POST /api/swap-requests/:requestId/complete
```

## Messaging

```
TEXT
GET /api/conversations
GET /api/conversations/:conversationId/messages
POST /api/conversations/:conversationId/messages
POST /api/conversations/:conversationId/report
```

## Group invitations

```
TEXT
```

```
POST /api/journeys/:journeyId/invites
GET /api/invites/:token
POST /api/invites/:token/accept
POST /api/invites/:token/revoke
```

Invite links must not contain:

- PNR.
- Ticket barcode.
- Phone number.
- Full legal name.
- Private berth details unless the recipient is authenticated and authorized.

## 14. PWA Requirements

### Manifest

Include:

- App name.
- Short name.
- Icons.
- Theme color.
- Background color.
- Standalone display.
- Portrait-primary orientation.
- Share target if appropriate.

### Offline capabilities

Allow offline users to:

- Open previously loaded journeys.
- View cached seat maps.
- View their own berth.

- View saved preferences.
- View last-known request status.
- Read safety instructions.

Do not allow offline users to:

- Accept a swap.
- Confirm current availability.
- Publish a listing.
- Make a payment.
- Complete a request.
- Send a message that appears immediately delivered.

Display:

**TEXT**

Offline mode

Last updated: [time]

Reconnect to confirm availability and requests.

## Performance targets

Target:

- First meaningful screen in under 3 seconds on a mid-range mobile connection.
- Seat map interaction without visible lag.
- Small initial JavaScript bundle.
- Lazy-load admin tools and secondary screens.
- Compress icons and illustrations.
- Avoid heavy 3D rendering.
- Use SVG or CSS geometry for maps.

---

## 15. Mobile UX Requirements

### Bottom navigation

TEXT

Home

My Journey

Matches

Inbox

Profile

## Home screen

Include:

- Add journey.
- Recent journey cards.
- Group invite shortcut.
- Map preview.
- Safety message.
- Sample coach map.

## Seat map screen

Include:

- Train and date header.
- Coach selector.
- Map confidence badge.
- Last updated time.
- Zoom controls.
- Locate my berth.
- Show matches.
- List view.
- Legend.
- Accessible description.

## Berth detail sheet

Include:

- Berth number.
- Visual position.
- Coach and bay.
- Features.
- Current participation state.
- Map confidence.
- Swap action.
- Report incorrect location.

## Accessibility

Required:

- Keyboard navigation.
- Screen-reader labels.
- Focus management.
- Minimum touch target size.
- Color plus icon/text status.
- Reduced-motion support.
- List alternative for every visual map.
- Hindi and English labels.
- High contrast.
- No information conveyed by color alone.

## 16. Trust, Privacy, and Moderation

### Verification levels

TEXT  
 manual  
 evidence\_checked  
 operator\_verified

Never expose uploaded evidence to other travelers.

## Do not expose

- PNR.
- QR code.
- Ticket barcode.
- Aadhaar.
- Passport.
- Full ticket image.
- Phone number.
- Email.
- Public passenger manifest.
- Full legal name by default.

## Safety controls

- Report.
- Block.
- Cancel.
- Expire.
- Rate limit.
- Account suspension.
- Suspicious behavior detection.
- Payment scam warning.
- User-visible safety guidance.
- Admin audit log.

## Structured safety messages

Provide predefined message options:

### TEXT

I can meet near the coach entrance.  
I will confirm after boarding.  
Please verify with railway staff.  
I cannot continue with this request.

I am traveling with a family member.

## Moderation rules

Block or review:

- Requests for direct payment.
- Ticket resale offers.
- Repeated spam.
- Harassment.
- Phone-number requests.
- Off-platform payment pressure.
- Threats.
- Impersonation.
- Fake verification claims.

## 17. Viral Growth Flywheel

The product should grow through travel groups, not only individual app installs.

### Core flywheel

#### TEXT

One traveler creates a journey  
→ Invites family or travel group  
→ More passengers join the same service  
→ More seats and preferences become visible  
→ More useful matches appear  
→ More successful coordination occurs  
→ Users share their journey  
→ New travelers join future journeys  
→ Matching density improves

## Viral loop A: Group invitation

### Trigger

User adds a train journey.

## Prompt

Traveling with others? Invite your group to see everyone on one coach map.

## Shared link

The link opens a safe landing page showing:

- Train.
- Date.
- Route.
- Group name.
- Number of members, if permitted.

The recipient must sign in before seeing private group details.

## Reward

The group creator receives:

- Group dashboard.
- Group map.
- Member status.
- Reminder controls.
- Free group coordination for a limited group size.

The invited passenger receives:

- One-click journey setup.
- Pre-filled train and date.
- Immediate group context.

## Viral loop B: Better-seat request

A user sees a possible match and sends a request.

The recipient receives:

- A deep link.
- Visual comparison.

- Clear benefit.
- Minimal setup.
- No need to search manually.

The request page should be understandable before signup, but private information must remain protected.

## **Viral loop C: Family coordination**

Create a family setup flow:

### **TEXT**

Create family trip

→ Add members

→ Assign or collect berths

→ View all seats

→ Identify distance between members

→ Find possible arrangements

→ Share coordination summary

Potential shareable artifact:

Our family journey map is ready.

Do not share PNRs or private ticket images.

## **Viral loop D: Group organizer**

Let one organizer manage:

- Wedding groups.
- College trips.
- Tour groups.
- Religious groups.
- Corporate travel.

Organizers invite passengers through WhatsApp. Every passenger joins a real journey, improving marketplace liquidity.

## **Viral loop E: Train-specific map pages**

Create public pages for supported train and coach layouts:

### **TEXT**

Train 12951 coach layout  
Sleeper berth map  
AC 3-tier berth map  
How to find berth 42

The page should offer:

Add your journey and see your exact berth.

Do not display current passenger information publicly.

## **Viral loop F: Post-journey sharing**

After a completed journey, show:

- Journey summary.
- Map used.
- Group coordination result.
- Optional shareable card.

Avoid showing private berth or passenger information in public images.

## **Viral loop G: Referral rewards**

Use non-cash rewards first:

- Free Group Trip Pass.
- Extra active journey.
- Advanced reminder access.
- Saved preferences.
- Organizer dashboard trial.

Avoid rewarding spam invitations.

## **Anti-spam rules**

- Limit invitations per journey.
- Detect repeated invalid invites.
- Require authentication after reasonable preview.
- Rate-limit messages.
- Do not send unsolicited SMS or WhatsApp messages.

- Require explicit consent for notifications.
- Allow invite revocation.

---

## 18. Monetization Architecture

### Free tier

Include:

- Journey creation.
- Visual map.
- Basic preferences.
- Basic matching.
- Basic requests.
- Group invitations.
- Report and block.

### Paid Group Trip Pass

Possible features:

- Larger group size.
- Group dashboard.
- Family profiles.
- Advanced alerts.
- Seat-distance visualization.
- Organizer controls.
- Exportable group summary.
- Priority support.

The pass must not promise a successful swap.

### Suggested experiments

Test:

#### TEXT

₹19 individual journey add-on

₹29 small family coordination pass

₹49 larger group pass

₹99 monthly frequent traveler plan

₹299 quarterly plan

₹599 annual plan

These are experiments, not guaranteed prices.

## B2B plans

Target:

- Tour operators.
- Travel agents.
- Student trip organizers.
- Religious travel organizers.
- Wedding planners.
- Corporate travel coordinators.

Charge the organizer, not every passenger.

## Payment requirements

- Use a licensed Indian payment gateway.
- Support UPI.
- Support cards and net banking.
- Store payment provider references, not sensitive payment data.
- Implement idempotency.
- Implement refunds.
- Keep an immutable payment audit trail.
- Do not create a wallet in the first release.

## 19. Analytics Events

Track product usefulness without collecting unnecessary personal data.

## Acquisition

### TEXT

landing\_viewed  
install\_prompt\_shown  
pwa\_installed  
invite\_created  
invite\_opened  
invite\_accepted  
referral\_completed

## Journey activation

### TEXT

journey\_started  
train\_selected  
date\_selected  
stations\_selected  
class\_selected  
coach\_entered  
berth\_entered  
layout\_viewed  
layout\_confirmed  
preferences\_saved  
journey\_published

## Matching

### TEXT

matches\_loaded  
match\_viewed  
match\_hidden  
match\_reported  
swap\_request\_created  
swap\_request\_accepted  
swap\_request\_declined  
swap\_request\_expired  
swap\_request\_cancelled  
swap\_completed

## Trust

### TEXT

verification\_started  
verification\_completed  
user\_reported  
user\_blocked  
layout\_error\_reported  
safety\_message\_viewed

## Monetization

### TEXT

paywall\_viewed  
checkout\_started  
payment\_succeeded  
payment\_failed  
refund\_requested  
refund\_completed

## Important dashboards

Measure separately:

- No other traveler joined.
- No compatible match.
- User did not complete setup.
- User did not understand the map.
- Request was declined.
- Request expired.
- User reported inaccurate layout.

---

## 20. Admin Dashboard

Required before public beta.

### Admin sections

#### TEXT

Overview  
Users  
Journeys  
Train services  
Layout templates  
Layout reports  
Swap requests  
Moderation  
Payments  
Feature flags  
Notifications  
Analytics  
Audit logs

## Admin actions

- Suspend user.
- Restore user.
- Review report.
- Correct layout.
- Disable layout.
- Mark journey invalid.
- Cancel abusive request.
- Refund payment.
- Update supported train.
- Change confidence status.
- View source and last-reviewed date.
- Export moderation audit.

## 21. Security Requirements

### Authentication

- OTP rate limiting.
- Login attempt throttling.
- Session rotation.
- Secure cookies.
- Device/session management.
- Account deletion.

### Authorization

Every private query must check:

**TEXT**

current user owns resource

OR current user belongs to authorized group

OR resource is intentionally public

Never rely only on hidden UI controls.

## API security

- Request validation.
- Schema validation.
- CSRF protection where relevant.
- Rate limiting.
- Idempotency keys for mutations.
- Audit logs for sensitive actions.
- No secrets in client bundles.
- No ticket evidence in public URLs.

## Data retention

- Delete unnecessary evidence automatically.
- Expire old journey visibility.
- Remove inactive journey data according to retention policy.
- Allow user deletion.
- Keep only the minimum audit data required.

## 22. Testing Requirements

### Unit tests

Test:

- Segment overlap.
- Matching score.
- Preference compatibility.
- Request expiry.
- Request state transitions.

- Layout parsing.
- Seat feature detection.
- Time-zone conversion.
- Currency formatting.
- Localization fallback.

## Integration tests

Test:

- Journey creation.
- Layout resolution.
- Match creation.
- Request acceptance.
- Concurrent acceptance.
- Request cancellation.
- Expiry worker.
- Notification worker.
- Permission checks.
- Group invite access.
- Payment idempotency.

## End-to-end tests

Test:

1. New user creates a journey.
2. User selects a coach and berth.
3. User confirms an illustrative layout.
4. User sets preferences.
5. User sees a match.
6. User sends a request.
7. Second user accepts.

8. Both see the accepted state.
9. A competing request is rejected.
10. Request expires.
11. User reports another user.
12. User views journey offline.
13. Offline user cannot falsely confirm a swap.
14. User deletes account.

## Accessibility tests

- Keyboard-only flow.
- Screen-reader seat map.
- List alternative.
- Focus trap in bottom sheets.
- Reduced motion.
- Contrast.
- Hindi interface.
- Touch target validation.

---

## 23. Delivery Milestones

### Milestone 1: Visual prototype

Build:

- PWA shell.
- Home screen.
- Journey setup.
- Sample train data.
- Visual coach maps.
- Berth selection.

- Seat detail sheet.
- Preference selection.
- Swap preview.
- Mobile and desktop responsive layout.

Acceptance criterion:

A user understands the location and difference between two berths without reading railway abbreviations.

## **Milestone 2: Working private beta**

Build:

- OTP auth.
- Database.
- Journey records.
- Seat claims.
- Preferences.
- Matching.
- Swap requests.
- Expiry.
- Notifications.
- Reporting.
- Blocking.
- Admin layout management.

Acceptance criterion:

Two authenticated users can coordinate a possible swap without exposing private booking information or double-committing a berth.

## **Milestone 3: Group viral loop**

Build:

- Journey invitation.

- Family group.
- Group map.
- WhatsApp-friendly deep links.
- Invite tracking.
- Group dashboard.
- Referral rewards.
- Abuse and rate limits.

Acceptance criterion:

A user can create a journey and successfully bring multiple travelers into the same journey context.

## **Milestone 4: Monetization**

Build:

- Group Trip Pass.
- UPI payment.
- Refunds.
- Payment audit.
- Paid feature gates.
- Organizer dashboard.

Acceptance criterion:

Users understand exactly what they are buying, and no paid feature falsely promises a successful swap.

## **Milestone 5: Provider integrations**

Build:

- Provider adapter interface.
- Licensed train data integration.
- Layout confidence pipeline.
- Revalidation.

- Service changes.
- Equipment changes.
- Admin override.

Acceptance criterion:

Uncertain or unsupported train layouts produce an honest fallback instead of fabricated accuracy.

---

## 24. AI Coding Agent Instructions

The coding agent must follow these rules:

1. Inspect the existing repository before editing.
2. Preserve the current framework if practical.
3. Do not replace working infrastructure without justification.
4. Do not add secrets.
5. Do not add fake API integrations.
6. Use mock providers behind real interfaces.
7. Keep layout data separate from availability.
8. Add tests for all state transitions.
9. Build mobile-first.
10. Make the visual seat map the primary experience.
11. Add accessibility from the first implementation.
12. Use feature flags for incomplete integrations.
13. Use realistic empty states.
14. Never label unknown seats as available.
15. Never claim official railway integration without proof.
16. Never expose private booking data.
17. Add loading, error, offline, and stale-data states.
18. Add analytics events without leaking personal information.
19. Make all mutations idempotent.

20. Explain architectural decisions in the pull request.

## Agent workflow

### TEXT

1. Inspect repository.
2. Identify framework and existing conventions.
3. Run existing tests.
4. Create implementation plan.
5. Build design system foundation.
6. Build sample train data.
7. Build seat-map renderer.
8. Build journey flow.
9. Add backend schema.
10. Add matching engine.
11. Add request state machine.
12. Add group invitation loop.
13. Add moderation.
14. Add PWA support.
15. Add tests.
16. Run lint, typecheck, test, and build.
17. Document setup and known limitations.

## 25. First Coding Task

The first coding task should be:

Build a mobile-first train journey demo using local sample data, with a visual Sleeper and AC 3 Tier coach map, berth selection, preference selection, match preview, PWA manifest, offline fallback, and accessible list view.

Do not begin with payments, flights, live railway APIs, or ticket OCR.

### First-task acceptance criteria

- The app loads on a mobile viewport.
- A user can select a sample train.
- A user can select a date.
- A user can select class and coach.
- A coach map renders visually.
- Berths are selectable.
- The user's berth is visually highlighted.

- Potential swap seats use a separate state.
- Unknown seats are not shown as available.
- Berth detail opens in a mobile bottom sheet.
- A list alternative exists.
- Layout confidence is visible.
- Offline mode displays cached demo data.
- No private or real booking data is included.
- Tests pass.
- Type checking passes.
- Production build passes.

---

## 26. Definition of Done

A feature is complete only when:

- It works on mobile.
- It works on desktop.
- It has loading and error states.
- It has empty states.
- It handles offline mode where relevant.
- It has accessible labels.
- It has authorization checks.
- It has validation.
- It has tests.
- It has analytics events where useful.
- It does not expose private data.
- It does not claim unsupported railway functionality.
- It has documentation.
- It passes lint, typecheck, tests, and production build.

---

## 27. Product North Star

The north-star experience is:

A passenger opens the app, sees their actual berth on a clear coach map, understands what nearby options mean, invites their travel group, finds a compatible passenger, and coordinates a safer and more comfortable arrangement without confusion or false promises.

The business flywheel is:

### TEXT

Useful visual map

→ More journey creation

→ More group invitations

→ More passengers on the same train

→ Better match coverage

→ More successful coordination

→ More sharing and referrals

→ Higher journey density

→ More paid group coordination

→ More organizer adoption

Build density, trust, and map accuracy before expanding into more transport modes.

## Files to give the coding agent

Along with `SWAP-INDIA-IMPLEMENTATION-SPEC.md`, provide:

```text

package.json

README.md

src/ or app/

public/

database schema or migrations

framework configuration

.env.example

existing tests

deployment configuration

Exclude:

TEXT

.env

.env.local

API keys

private certificates

node_modules

.next

dist

build
database dumps containing real user data

Recommended instruction to place above the file

TEXT

Read SWAP-INDIA-IMPLEMENTATION-SPEC.md before changing the repository.

First inspect the existing codebase and report:

1. Current framework and architecture.
2. Existing routes and components.
3. Existing database and authentication.
4. Existing PWA support.
5. Existing tests.
6. What can be reused.
7. What should be added.
8. Any conflicts with the specification.

Do not start implementation until you provide a short repository assessment and phased implementation plan.

Implement the first milestone only unless explicitly instructed otherwise: the mobile-first visual Indian train seat-map experience using local sample data.

you

can you help me with the mobile pwa designs

Gpt 6 Astra

Yes. I can help design the complete mobile-first PWA for Swap India, including the visual system, screen flows, component specifications, and implementation-ready UI guidance.

Recommended Design Direction

The app should feel:

- Trustworthy, calm, and practical
- Optimized for Indian train travel
- Visual rather than text-heavy
- Fast on low-end Android phones
- Friendly for families and older travelers
- Clear about what is verified versus user-provided
- Useful even with limited connectivity

Suggested visual personality:

- Warm ivory background
- Deep railway green as the primary color
- Saffron or amber for attention states
- Indigo or navy for verified information
- Coral only for warnings or destructive actions
- Rounded cards, large touch targets, clear bottom sheets
- Visual coach maps as the main product identity

Avoid a generic banking dashboard or marketplace interface.

Core Mobile Navigation

Use five bottom-navigation items:

TEXT
Home
My Journey
Matches
Inbox
Profile

The primary action should always be easy to reach:

TEXT
+ Add Journey

For a new user, **Home** and **My Journey** should guide them toward adding their first train journey.

Main Screen Set

1. Welcome / Landing Screen

Purpose: Explain the product immediately.

Content:

TEXT

See your berth.
Find a better fit.
Coordinate safely.

View your train coach visually and connect with passengers
who may be open to a compatible seat exchange.

[Add my journey]
[Explore sample coach map]

Passenger coordination only.
This app does not automatically change railway reservations.

Visual:

- Large illustrated coach map
- A highlighted lower berth, upper berth, and group of connected travelers
- Small trust indicators:
 - Visual maps
 - Privacy-first
 - Group-friendly

2. Home Screen

Header

TEXT
Good morning, Rahul
Where are you travelling?

Primary card

TEXT
Add your train journey

See your coach, berth, and possible seating options.

[Add journey]

Recent journey card

TEXT
Mumbai → Delhi
Train 12951 · 24 Oct

Coach B3 · Berth 42

[Open seat map]

Group invitation card

TEXT

Travelling with family or friends?

Create a shared journey map and invite everyone.

[Create group journey]

Trust notice

TEXT

Your booking details stay private.

Only relevant journey and seat information is shared.

3. Add Journey Flow

Use a short multi-step flow rather than one long form.

Step 1: Choose transport

For the initial version:

TEXT

What are you travelling by?

[Train]

Flights coming soon

Step 2: Search train

Fields:

TEXT

Train number or name

Travel date

Search result:

TEXT

12951

Mumbai Central → New Delhi
Mumbai Central 16:35
New Delhi 08:35

Step 3: Choose route segment

TEXT

Where are you boarding?

Mumbai Central

Where are you getting off?

New Delhi

Show a visual route line with station stops.

Step 4: Choose class

Large visual cards:

TEXT

Sleeper

Open berths · Non-AC

AC 3 Tier

Open berths · Air-conditioned

AC 2 Tier

More space · Air-conditioned

Step 5: Enter coach and berth

TEXT

Which coach are you in?

[B3]

What is your berth number?

[42]

Not sure about your berth?

[I'll select it on the map]

Step 6: Confirm map

TEXT

We found an expected B3 layout.

Please confirm that berth 42 looks like this:

[Visual coach map]

Layout status:

Expected layout · Please confirm

[Confirm my berth]

[Report a different layout]

This screen is important because the app should never pretend uncertain layout data is exact.

4. Visual Coach Map Screen

This is the core product experience.

Layout

TEXT

← Mumbai → Delhi

Train 12951 · 24 Oct

B3

AC 3 Tier

Expected layout · Updated 10 min ago

[B1] [B2] [B3] [B4] [B5]

Below the coach selector:

TEXT

[-] [Locate me] [+]

[Show matches]

Then the map.

Visual states

Use both color and icons:

- Your berth: solid green with person icon
- Possible match: amber outline with swap icon
- Group member: blue with group icon
- Request pending: striped amber
- Accepted: solid blue with check icon

- Unknown: gray with question mark
- Unavailable: muted gray with lock icon
- Reported layout issue: red warning indicator

Never show unknown seats as empty or available.

Berth card

When a user taps a berth:

TEXT

Berth 41

Lower berth

Bay 5 · Same coach

Near your current berth

Status:

Passenger may be open to a swap

[Compare with my berth]

[Close]

For your own berth:

TEXT

Your berth

Berth 42 · Upper

Bay 5 · Coach B3

[Set preferences]

[Share with group]

5. Train Map Visualization

For AC 3 Tier, show a bay visually:

TEXT

| | |
|-------------|-------------|
| Bay 5 | |
| [41 Lower] | [44 Lower] |
| [42 Middle] | [45 Middle] |
| [43 Upper] | [46 Upper] |
| —— aisle —— | |

[47 Side Lower]

[48 Side Upper]

For Sleeper, use the same visual structure but adapt styling and labels.

Add a small elevation switch:

TEXT

[Top view] [Berth levels]

The elevation view can display:

TEXT

Upper

Middle

Lower

This is much easier for users than forcing them to understand abbreviations like **SU**, **MB**, or **LB**.

6. Preferences Screen

Use visual preference cards rather than a dense form.

TEXT

What matters most to you?

Select any:

[Lower berth]

[Same coach]

[Same bay]

[Near family]

[Easier access]

[Away from toilet]

[Side berth]

[More privacy]

Then:

TEXT

How flexible are you?

[Very flexible]

[Somewhat flexible]

[Only if it is a better fit]

For accessibility-sensitive preferences:

TEXT

Would easier access help you?

[Yes]

[No]

[Prefer not to say]

Do not ask for medical details.

7. Matches Screen

Use visual match cards.

TEXT

Possible match

Your berth:

B3 · 42 Upper

Their berth:

B3 · 41 Lower

Same bay · Same coach

Your preference: Lower berth

[View on map]

[Compare]

Another example:

TEXT

Group match

Your family is spread across B2 and B3.

This passenger is open to moving closer to your group.

[View group map]

Avoid showing a match as guaranteed.

Use text such as:

Possible match

rather than:

Available seat

8. Swap Preview Screen

This screen should visually explain the exchange.

Header

TEXT

Review possible swap

Comparison

TEXT

| YOUR CURRENT BERTH | PROPOSED BERTH |
|--------------------|----------------|
|--------------------|----------------|

| | |
|---------------|---------------|
| B3 · 42 Upper | B3 · 41 Lower |
| Bay 5 | Bay 5 |

| | |
|-----------------|-----------------|
| [Map highlight] | [Map highlight] |
|-----------------|-----------------|

Outcome

TEXT

You would move from an upper berth
to a lower berth in the same bay.

Conditions

TEXT

Both passengers must accept.
The app does not automatically change your railway booking.
Confirm with railway staff if required.

Actions:

TEXT

[Send swap request]
[Not interested]

9. Request Status Screen

Use a clear timeline.

TEXT

Swap request

B3 · 42 Upper ↔ B3 · 41 Lower

- | | |
|---------------------------|---|
| 1. Request sent | ✓ |
| 2. Waiting for response | ● |
| 3. Both passengers accept | ○ |
| 4. Confirm before travel | ○ |

Actions:

TEXT

[Cancel request]

[Send structured message]

[Report issue]

For accepted requests:

TEXT

Both passengers accepted.

Please verify any required change with railway staff.

This agreement does not automatically modify your reservation.

10. Group Journey Screen

This should be a major viral feature.

TEXT

Family journey

Mumbai → Delhi

Train 12951 · 24 Oct

4 members joined

[Share invite]

[View coach map]

Member cards:

TEXT

Anita

B3 · 42 Upper

Needs: Lower berth

Vikram
B3 · 45 Lower
Open to swap

Meera
B2 · 18 Lower

Show the whole group on the coach map with a blue group highlight.

Primary CTA:

TEXT
[Invite another traveller]

The invite message should be WhatsApp-friendly:

TEXT
Join our journey map for Train 12951 from Mumbai to Delhi on 24 Oct.

Add your berth and see where everyone is seated:
[Safe invite link]

Do not include PNR, ticket image, phone number, or private booking details.

11. Inbox Screen

Use structured conversation cards.

TEXT
Requests
Messages
Updates

Request card:

TEXT
Swap request from Priya

They have B3 · 41 Lower
You have B3 · 42 Upper

[Review]

Use quick actions:

TEXT
[Accept]

[Decline]
[Ask a question]

Avoid making the app feel like an unmoderated social network.

12. Profile Screen

Include:

TEXT
Your profile
Preferred language
Saved preferences
My journeys
Notifications
Privacy
Safety centre
Help
Delete account
Log out

Important privacy controls:

TEXT
Who can find me?
[Only people on my journey]

Show my display name as:
[Rahul]

Allow group invitations:
[On]

Design System

Colors

Suggested CSS variables:

CSS
:root {
 --color-ink: #17302b;
 --color-primary: #176b59;
 --color-primary-dark: #0e4d41;
 --color-primary-soft: #dff1e8;
}

```
--color-saffron: #e58a32;
--color-saffron-soft: #fff0d8;

--color-blue: #3f6fa8;
--color-blue-soft: #e8f0fb;

--color-danger: #b94b45;
--color-danger-soft: #fde9e7;

--color-background: #f7f4ed;
--color-surface: #fffd8;
--color-border: #dedbd2;
--color-muted: #6e766f;
}
```

Typography

Use an expressive but highly readable typeface.

Good combinations:

- `DM Sans` for interface text
- `Newsreader` or `Fraunces` for selected headings
- `Noto Sans Devanagari` for Hindi support

Avoid using a decorative font for data-heavy screens.

Shape language

- Cards: 18–24px radius
- Buttons: 14–16px radius
- Bottom sheets: 28px top radius
- Chips: pill-shaped
- Seat cells: rounded 10–14px corners
- Avoid excessive shadows
- Use borders and background contrast for structure

Touch targets

Minimum:

`TEXT`
44px × 44px

Prefer:

TEXT

48px × 48px

This is particularly important for map controls and older users.

Loading, Empty, and Error States

Loading

Show a simplified coach skeleton rather than a generic spinner.

TEXT

Loading your coach map...

[animated map placeholder]

No matches

TEXT

No compatible passenger yet

Your journey is published. Invite your travel group
to improve the chance of finding a match.

[Invite travellers]

[Update preferences]

Unsupported layout

TEXT

We do not have a confirmed layout for this coach yet.

You can continue with a user-confirmed map,
but please check your berth visually before publishing.

[Choose a similar layout]

[Enter manually]

Offline

TEXT

You are offline.

Showing the last saved journey map.

Availability and requests need a connection to update.

Last updated today at 8:42 PM.

Mobile Interaction Principles

1. Keep one primary action per screen.
2. Use bottom sheets for berth details.
3. Keep navigation reachable with one hand.
4. Avoid dense tables.
5. Use visual maps before labels.
6. Preserve state when users go back.
7. Save progress during journey setup.
8. Use large tap targets.
9. Explain railway abbreviations only as secondary information.
10. Never hide important warnings in tooltips.
11. Make stale information visibly stale.
12. Prefer structured messages before open chat.
13. Use Hindi-friendly text lengths.
14. Optimize for slow network conditions.
15. Never block the user with unnecessary signup before they understand the product.

PWA-Specific Design

Install prompt

Do not show immediately on first load.

Show after the user:

- Views a seat map.
- Creates a journey.

- Returns to the app.
- Uses the product successfully once.

Prompt:

TEXT

Keep your journey map ready for travel day.

Install Swap for faster access, offline maps,
and journey reminders.

[Install app]

[Maybe later]

Offline journey card

TEXT

Saved for offline use

Last updated:

24 Oct · 8:42 PM

[Open map]

Push permission

Ask at the moment of value:

TEXT

Get notified when someone responds to your swap request?

[Allow notifications]

[Not now]

Do not ask for notification permission on the landing screen.

Recommended First Design Deliverables

Create these before coding:

1. Mobile design tokens.
2. User-flow diagram.
3. Home screen.

4. Add Journey flow.
5. Coach map.
6. Berth detail bottom sheet.
7. Preferences screen.
8. Matches screen.
9. Swap preview.
10. Group journey screen.
11. Inbox.
12. Profile and privacy.
13. Offline state.
14. Empty and unsupported-layout states.
15. Hindi text variants.
16. Accessibility annotations.
17. Responsive tablet and desktop adaptations.

First Build Recommendation

Start with a clickable prototype containing:

TEXT

Home

→ Add train journey

→ Select class

→ Select coach and berth

→ View visual map

→ Set preferences

→ View sample match

→ Compare seats

→ Send request

→ View request status

Do not start with the full backend or payments. Prove that users understand and enjoy the visual seat-map experience first.

The product's strongest design advantage should be:

A passenger can understand their exact place in the coach within seconds, without knowing railway berth abbreviations.

you

wheres the mobile pwa designs i need designs

Gpt 6 Astra



Same trains.
Happier journeys.

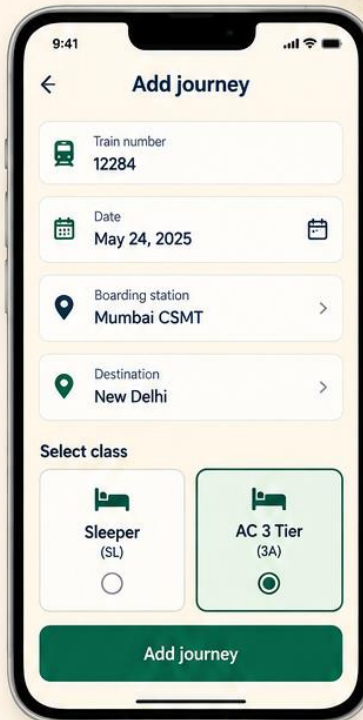
People swap. Journeys connect.

India
Travels Better
Together



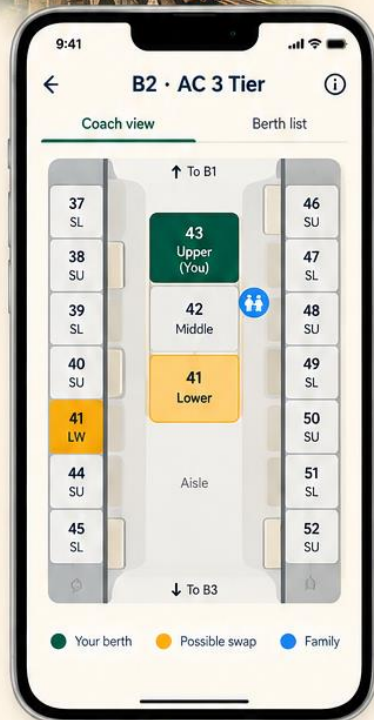
1. Home screen

Your journey. A better fit awaits.



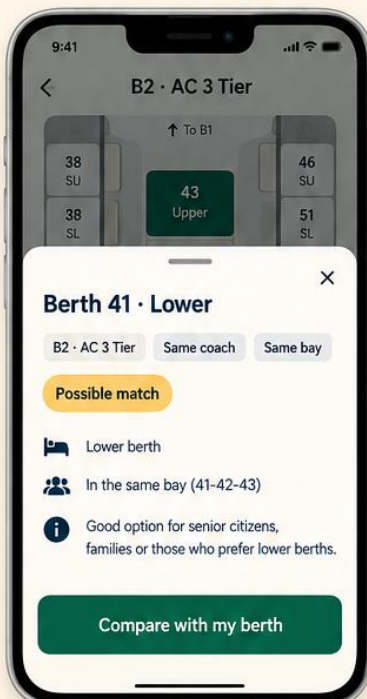
2. Add journey

Tell us about your trip.



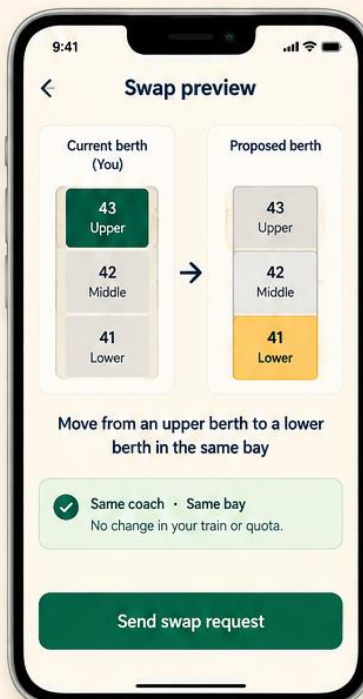
3. Coach seat map

Find a better seat, easily.



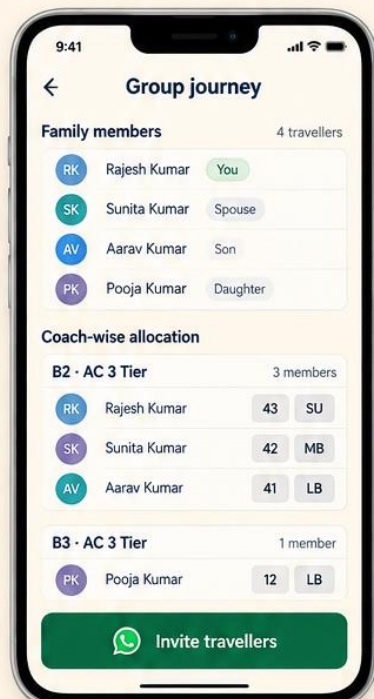
4. Berth detail

Check details. Make a smarter choice.



5. Swap preview

A simple step to a more comfortable journey.



6. Group journey

Keep your loved ones close.



Verified travellers



A safer, friendlier community



Made for Indian rail travellers



Same trains. Happier journeys.

[Open full image](#)[Download file](#)

you

first we should ask whether they want to swap seat in bus or train or flight , then ask their booking details source destination, booking id, gender, then we should show what their seat in compartment, why they are looking for swap what seat they are looking for swap. retrieve visual compartment bus seats , then they can pay 50 for swap request. if someone with same requests is there it will be matched and the other party details will be shown . you can improve it. it should be dead easy

Gpt 6 Astra

Yes — the flow should be much simpler and more visual. The best version is:

Choose journey → enter trip details → see your seat → choose desired seat → pay ₹49 → get matched.

Do not make users fill a long form before they understand the product.

Recommended Mobile Flow

Screen 1: Choose Transport

Large visual cards:

TEXT

What are you travelling by?

[Train]

See your coach and berth

[Bus]

See your bus seat layout

[Flight]

See your cabin and seat

Use icons and illustrations, not dropdowns.

Primary action:

TEXT

[Continue]

After selection, keep the chosen mode visible at the top:

TEXT

Train selected

Screen 2: Enter Journey Details

Use one simple screen with progressive fields.

TEXT

Your journey

From

[Mumbai Central]

To

[New Delhi]

Travel date

[24 October 2026]

[Continue]

For flights and buses, add:

TEXT

Operator / airline

[Select operator]

Bus / flight number

[Enter number]

For trains:

TEXT

Train number or name

[Search train]

Important improvement

Do not ask for the booking ID first.

Ask for it only after the user has seen the value of the app, or make it optional:

TEXT

Booking reference

[Optional]

Used only to help verify your journey.

Never shown to other passengers.

Never display booking IDs, PNRs, QR codes, ticket images, or passenger documents to matched users.

Screen 3: Passenger Details

Keep this short.

TEXT

About your journey

How many people are travelling?

[-] 1 [+]

Your name

[Display name]

Gender

[Prefer not to say v]

Is this journey detail correct?

[Confirm]

Gender handling

Gender should be optional and private by default.

Possible product uses:

- Matching a passenger who is more comfortable swapping with the same gender.
- Improving safety.
- Supporting women traveling alone.

But do not make gender mandatory unless absolutely required.

Use:

TEXT

Gender preference

[No preference]

[Same gender preferred]

Do not expose a person's gender publicly before both parties accept.

Screen 4: Retrieve and Show the Current Seat

After the user submits details:

TEXT

Finding your seat map...

Searching:

Mumbai Central → New Delhi

Train 12951 · 24 October

Then show:

TEXT

We found your seat

Coach B3

Berth 42 · Upper berth

[View my coach]

The seat map should be visual immediately.

Example train view

TEXT

B3 · AC 3 Tier

| | |
|---------------|-----------|
| 41 Lower | 44 Lower |
| 42 Middle | 45 Middle |
| 43 Upper | 46 Upper |
| aisle | |
| 47 Side Lower | |
| 48 Side Upper | |

• Your seat: 42 Middle

Do not make users interpret only:

TEXT

B3 / 42 / M

Show:

- Coach.
- Bay or compartment.
- Seat number.

- Position.
- Door and toilet location where reliable.
- Confidence status.

TEXT

Layout status: Expected

Please confirm your berth position.

For buses, show the actual bus shape:

TEXT

Driver

| | | | |
|----------------|----|----|----|
| | | | |
| 1A | 1B | 1C | 1D |
| 2A | 2B | 2C | 2D |
| 3A | 3B | 3C | 3D |
| | | | |
| Emergency exit | | | |
| | | | |

For flights, show:

- Cabin.
- Row.
- Window/aisle.
- Exit row.
- Galley.
- Lavatory.
- Restricted seats.

If the layout cannot be verified:

TEXT

We found a likely layout.

Please confirm your seat location before continuing.

Never fabricate an exact map.

Screen 5: Why Do You Want to Swap?

Use visual choice cards.

TEXT

Why are you looking for a swap?

Choose all that apply:

- ☐ I want a lower berth
- ☐ I want a window seat
- ☐ I want an aisle seat
- ☐ I want to sit with family
- ☐ I want to sit with friends
- ☐ I need easier access
- ☐ I want to move away from the toilet
- ☐ I prefer a different seat position
- ☐ Other

For trains, show relevant berth options.

For buses:

- Window.
- Aisle.
- Front.
- Back.
- Single seat.
- Away from wheel or toilet.
- Near companion.

For flights:

- Window.
- Aisle.
- Extra legroom.
- Away from lavatory.
- Near companion.
- Not an exit row.

Keep this screen simple and mode-specific.

Screen 6: Choose the Seat You Want

Instead of another text form, show the same visual map.

Header:

TEXT

What seat would you prefer?

Seat states:

- Green: your current seat.
- Amber: seats matching your request.
- Gray: unknown or not participating.
- Blue: your group members.
- Striped: pending swap.
- Locked: unavailable or restricted.

When a user selects a preferred seat:

TEXT

You selected:

B3 · 41 Lower

Same bay · Same coach

One level below your current berth

[Continue]

Also offer:

TEXT

[I am flexible]

This is essential because strict preferences reduce matches.

Screen 7: Review Request

Before payment, show a simple summary.

TEXT

Your swap request

Current seat

B3 · 42 Middle

Looking for
B3 · 41 Lower

Reason
Lower berth preferred

Journey
Mumbai Central → New Delhi
24 October 2026

Gender preference
No preference

Add a clear notice:

TEXT
This is a passenger coordination request.
It does not automatically change your railway, bus, or airline booking.
Operator or crew approval may be required.

Payment Design

Charging ₹50 before a match may reduce adoption, especially when the user does not know whether another compatible passenger exists.

Better model

Use:

TEXT
₹49 swap request fee

But make the outcome clear:

- No match: refund or credit.
- Match found: fee is consumed.
- Other passenger declines: refund or credit.
- User cancels before matching: refund according to policy.
- Request expires: refund or credit.
- Successful match: no additional passenger-to-passenger payment.

Recommended payment screen

TEXT

Activate your swap request

₹49

Includes:

- ✓ Matching with compatible passengers
- ✓ Request notifications
- ✓ Visual seat comparison
- ✓ In-app coordination

You are not paying for another passenger's seat.
A match is not guaranteed.

[Pay ₹49 and find matches]

For the first launch, I would test:

1. Free matching during beta.
2. Then ₹29 or ₹49 per journey.
3. Refund or credit if no suitable match is found.

Avoid requiring payment before showing whether there are likely compatible users. A better sequence is:

TEXT

Show estimated match availability

- User confirms request
- Pay to activate request

Example:

TEXT

We found 3 possible passengers nearby.

Activate your request for ₹49?

This should improve conversion substantially.

Screen 8: Matching

After payment:

TEXT

Looking for compatible passengers

Your request is active until:
24 October · 6:00 PM

We are checking:

✓ Same journey

✓ Same route segment

✓ Compatible seat preferences

✓ Operator restrictions

Show progress without pretending that active searching guarantees a result.

If no match exists:

TEXT

No matching passenger yet

Your request is still active.

You can:

[Invite travel companions]

[Make preferences flexible]

[Cancel request]

Screen 9: Match Found

Do not immediately expose the other person's private details.

First show an anonymous match card:

TEXT

We found a possible match

Passenger R

Verified journey details

Their seat

B3 · 41 Lower

Your requested seat

B3 · 42 Middle

Same coach · Same bay

[View proposed swap]

Only reveal limited details:

- Display name or alias.

- Verification level.
- Seat and journey segment.
- General gender preference if relevant and consented.
- Optional group context.

Do not reveal:

- Phone number.
- Email.
- Booking ID.
- PNR.
- Full ticket.
- Legal name.
- Home address.
- Public passenger history.

Screen 10: Mutual Acceptance

Use a clear two-sided status:

TEXT

Swap request

You: Accepted ✓

Passenger R: Waiting...

When both accept:

TEXT

Both passengers agreed

Current arrangement:

B3 · 42 Middle ↔ B3 · 41 Lower

Next step:

Confirm with railway staff or the operator if required.

[View safety instructions]

[Open coordination chat]

Never label this:

TEXT

Booking changed

unless the operator integration genuinely confirms it.

Make It “Dead Easy”

Use progressive disclosure

Only ask for information when needed.

First ask

1. Bus, train, or flight?
2. From and to?
3. Date?
4. Booking or service details?
5. Current seat?
6. Desired seat?
7. Payment?

Delay or avoid

- Full identity verification.
- Long preference forms.
- Mandatory gender.
- Ticket upload.
- Free-form explanations.
- Complex account setup.
- Manual coach configuration unless needed.

Use smart defaults

- Autofill train route after train number.

- Detect class from booking data if available.
- Show the current seat automatically.
- Preselect “same coach.”
- Use “I’m flexible” by default.
- Remember previous preferences.
- Allow group members to join through an invite link.

One primary button per screen

Examples:

TEXT

Continue

Confirm my seat

Choose preferences

Review request

Pay ₹49

View match

Accept request

Avoid multiple competing actions.

Recommended Final User Journey

TEXT

1. Choose Train, Bus, or Flight

2. Enter origin, destination, and date

3. Enter service number or booking reference

4. Confirm your name and optional gender preference

5. Show their current seat visually

6. Ask why they want to swap

7. Show matching seats on the visual map

8. Let them choose one or stay flexible

9. Show possible match count

10. Charge ₹49 to activate request

11. Match compatible passengers

12. Show anonymous match details

13. Let both parties accept

14. Unlock limited coordination chat

15. Show operator approval warning

16. Confirm completion after travel

Strongest Product Improvement

The most important change is this:

Do not make users pay blindly before knowing whether there are possible matches.

Use this sequence:

TEXT

Enter journey

→ See current seat

→ Select preferences

→ Show likely match count

→ Pay ₹49 to activate

That keeps the app easy, improves trust, and reduces payment abandonment.

Also make the first launch group-focused:

“Invite your family or travel group”

A stranger-to-stranger marketplace may have no matching density at the start. Group invitations create immediate users on the same journey and make the visual seat map useful from day one.

you

show me designs

Gpt 6 Astra



[Open full imageDownload file](#)

you

i want to implement this tell me what to give to ai coding platforms. give me proper architecture with a flywheel virality effect so more people will use the app : Yes. At this point, ****do not give the coding tool only the screenshots****. Give it a ****single implementation brief + the HD screen references + explicit business rules****. I would build the first version as a ****mobile-first PWA****, with one shared SwapSeat shell and three transport engines: ****Train, Bus, Flight****. ## What you should give the developer / Lovable / Replit / Cursor / Claude Upload all the final HD UI images you generated, then give them this master specification: # SwapSeat — Master PWA

Implementation Specification ## 1. Product Build ****SwapSeat****, a mobile-first PWA that helps verified travellers find and complete peer-to-peer seat swaps on: - Indian trains - Intercity buses - Flights Core promise: ****Find your comfort. Share your seat.**** The product is NOT a ticket-booking platform. SwapSeat must never sell, issue, modify, or claim to officially reassign a transport operator's ticket unless an official integration explicitly supports it. The product is a ****peer-to-peer seat-swap coordination and payment platform****. --- # 2. NON-NEGOTIABLE UX RULES #### Mobile only Design for: - 390 × 844 - 393 × 852 - 430 × 932 The primary experience must feel like a native mobile application. Do NOT create: - desktop dashboards as the primary UI - tablet layouts - web-style multi-column pages - oversized desktop navigation - tab-heavy desktop interfaces Everything should be optimized for one-handed mobile use. Use a consistent iOS/Android-quality mobile visual system. --- # 3. SHARED HOME SCREEN The first/home experience must remain consistent across all transport types. Home shows: #### SwapSeat branding ****SwapSeat**** Supporting copy: ****Same journeys. Happier together.**** #### Transport choices Three large mobile cards: - Train - Bus - Flight Each opens the relevant journey flow. Do not create completely different home screens for each mode. The user should immediately understand: > I can swap a seat on my train, bus or flight. --- # 4. CORE USER TYPES There are two primary roles. #### Requester The traveller actively looking for a different seat. #### Acceptor A traveller who receives a compatible request. Important: A person can be both roles on different journeys. --- # 5. SHARED BUSINESS LOGIC ## Journey matching Travellers must belong to: - the same train/bus/flight - same travel date - same relevant journey/service - compatible source → destination overlap Do NOT match somebody who leaves the transport service before the seat needs to remain occupied. Example: Traveller A: Hyderabad → Bengaluru Traveller B: Hyderabad → Bengaluru Compatible. But: Traveller A: Hyderabad → Bengaluru Traveller B: Hyderabad → Kurnool Do not blindly match them if B must vacate the seat before A's destination. The system must calculate ****segment overlap****. Both travellers must

explicitly confirm: ****From**** → ****To**** before a swap can proceed. --- # 6. SEAT PRIVACY / ANTI-LEAKAGE RULE This is critical. Exact seat numbers of the other traveller must remain hidden until payment conditions are satisfied. Before payment, show: - seat type - approximate position - coach/cabin/deck - front/middle/rear - window/aisle - upper/lower where applicable - near/far from entrance where applicable - comfort score - amenities - travel segment - source → destination Do NOT expose the exact seat number of another traveller before the required payment. Example: Instead of: > Seat 21 show: > Middle section > Window > Same coach > Away from entrance or: > Rear section > Aisle > Lower deck > Near middle The requester can see their OWN purchased seat numbers. The other traveller's exact seat numbers remain protected. --- # 7. PAYMENT MODEL Make pricing configurable from the backend. Current product rule: #### Search / Matching ****₹49**** The traveller pays ₹49 to activate the matching/search service. This is NOT a seat-booking fee. If no suitable match is found: ****₹49 → SwapSeat Wallet credit**** The user can use this wallet balance on a future search. #### Second payment / one-sided swap If the accepter already paid the ₹49 search fee because they were also looking for a swap: ****No second search fee.**** If the accepter was NOT looking for a swap and therefore did NOT pay the search fee: After the accepter accepts, the requester is asked to pay: ****₹99**** to complete that swap. Make this pricing configurable: ``text SEARCH\_FEE = 49 ONE\_SIDED\_SWAP\_FEE = 99 `` Never hard-code the pricing into UI logic. Store pricing on the backend. --- # 8. REQUESTER STATE MACHINE The requester journey should use explicit states. ``text DRAFT → JOURNEY\_SELECTED → SEATS\_SELECTED → PREFERENCES\_SELECTED → SEARCH\_PAYMENT\_PENDING → SEARCH\_PAYMENT\_SUCCESS → MATCHING → MATCH\_FOUND → REQUEST\_SENT → ACCEPTED → COMPLETION\_PAYMENT\_REQUIRED → COMPLETION\_PAYMENT\_SUCCESS → SWAP\_CONFIRMED → JOURNEY\_ACTIVE → JOURNEY\_COMPLETED `` Failure states: ``text NO\_MATCH PAYMENT\_FAILED REQUEST\_DECLINED REQUEST\_EXPIRED JOURNEY\_CANCELLED USER\_CANCELLED `` --- # 9. REQUESTER FLOW — TRAIN ## Screen T1 Welcome / Home Transport selector. ## Screen T2 Enter Journey Allow: #### Option A PNR / Booking ID #### Option B Manual entry Fields: - Source - Destination - Train - Date - Seat number(s) Manual entry must work even if an official booking lookup fails. --- ## Screen T3 Coach + Seat Layout Use a proper data-driven Indian railway coach layout. Do NOT use generic rectangular theatre-style seating. Seat maps must be rendered from transport data. The user's purchased seat(s) are highlighted. Allow multiple selected seats. --- ## Screen T4 Your Seat Details Show: - coach -

berth/seat type - window information - washroom relationship where applicable - entrance position - coach position - comfort score - useful tips Example: ****Comfort score 8/10**** --- ##
 Screen T5 What are you looking for? Preferences: - Lower berth - Middle berth - Upper berth - Window - Aisle - Same coach - Same bay - Family seating - More comfort - Other Reason: - Travelling with family - Medical/comfort requirement - Senior citizen - Personal preference - Need seats together - Other --- ## Screen T6 Pay ₹49 Explain: ****₹49 search fee**** Benefits: - verified matching - matching for this journey - secure request system - wallet credit if no suitable match is found --- ## Screen T7 Finding Matches Animated matching UI. Match against: - same train - same journey/date - source → destination compatibility - seat preferences - traveller verification --- ## Screen T8 Matches Found Do NOT show other travellers' exact seat numbers. Show: - name - profile photo - verification - match percentage - source → destination - seat characteristics - coach information - comfort indicators - preference compatibility Allow selecting multiple travellers. --- ## Screen T9 Send Requests Requester can send to multiple compatible travellers. Once one traveller accepts AND the required transaction is completed: ****automatically stop/cancel pending alternative requests.**** Do not continue sending requests once the swap is locked. --- ## Screen T10 Request Sent / Tracking Show: - requests sent - pending - accepted - declined - matching status Important copy: > We'll continue trying with other compatible travellers. --- ## Screen T11 No Match If there is no suitable match: Show: ****Need more matches?**** Actions: - Share to WhatsApp - X - Instagram - Facebook - Copy link - Other share options Example share text: > I'm looking to swap my seat on [train/bus/flight] from [source] to [destination]. If you're travelling on the same service, let me know. Find the swap on SwapSeat: [link] ₹49 automatically becomes: ****SwapSeat Wallet credit**** --- # 10. REQUESTER FLOW —
 BUS Add bus-specific information. Bus search MUST include: - source - destination - date - multiple departure times - operator - bus service - vehicle type - AC/non-AC - sleeper/seater/semi-sleeper - boarding point - dropping point - duration - amenities where available - live/service status when available This prevents users confusing multiple buses running between the same cities. Do NOT display the bus fare. SwapSeat is not selling the bus ticket. --- # 11. BUS SEAT MAP Seat map must be data-driven. Support: - lower deck - upper deck - sleeper - seater - semi-sleeper - left/right orientation - driver position - aisle - multiple seat configurations The user's OWN purchased seat numbers are visible. Other travellers' exact seat numbers remain hidden. --- # 12. REQUESTER FLOW — FLIGHT Flight booking retrieval must support: #### PNR + passenger surname Also allow: #### Booking ID / manual entry After retrieval show: - airline - flight number - date - departure airport - arrival airport - terminal where

available - departure time - arrival time - duration - aircraft where available - cabin class - booking passenger - purchased seat(s) --- # 13. FLIGHT SEAT MAP Use aircraft-specific layouts where available. Examples of data dimensions: - Economy - Premium Economy - Business - First where applicable - A/B/C - D/E/F - exit rows - front cabin - rear cabin - window - middle - aisle Never hard-code one aircraft layout. Create a reusable aircraft-seat-map renderer. --- # 14. FLIGHT PRICING Default: ``text SEARCH\_FEE = ₹49 `` If the accepter already paid the search fee: No additional acceptance fee. If the accepter was not looking for a swap and never paid the search fee: After they accept, the requester can be charged: ``text ONE\_SIDED\_SWAP\_FEE = ₹99 `` Keep this entirely configurable. --- # 15. ACCEPTER FLOW The accepter receives: Push notification: > New SwapSeat request They see: - requester profile - verification - journey - source → destination - transport - timing - number of seats - approximate seat characteristics - preference/reason - compatibility No exact requester seat number before payment. --- # 16. ACCEPTER COMPARISON SCREEN Show side-by-side: #### Your seat - approximate position - seat type - window/aisle - coach/deck/cabin - comfort score #### Their requested seat - approximate position - requested characteristics - coach/deck/cabin - comfort score Exact seat number hidden. --- # 17. ACCEPTANCE PAYMENT LOGIC #### If accepter already paid ₹49 search fee They accept. No second payment. #### If accepter never paid They accept. Then requester gets: > ₹99 required to complete this swap. Only after payment is completed should the final seat numbers be unlocked. --- # 18. SWAP CONFIRMATION Once all required payment conditions are fulfilled: Reveal: #### Your seat Exact seat number #### Their seat Exact seat number Show: - original seat - new seat - journey - source - destination - transport details - confirmation ID --- # 19. CHAT Chat becomes available only after the swap reaches the appropriate confirmed state. Chat is journey-bound. During journey: ``text Chat available until [destination] `` The users can: - message - coordinate boarding - share meeting/coach information - coordinate the actual swap When destination is reached: ``text Journey completed Chat closed `` The conversation remains available as history, but messaging is disabled. --- # 20. JOURNEY TRACKING Transport-specific live journey screen. Show: - source - intermediate stops when available - destination - ETA - current status - delay/on-time - trip progress Train: - station progression Bus: - route/stop progression Flight: - departure - in-air - arrival --- # 21. PROFILE / TRUST Traveller profile: - verified status - swaps completed - rating - successful swaps - cancellation history - badges - account age - reports/block Never expose unnecessary personal information. --- # 22. WALLET Create a SwapSeat wallet. Fields: - balance - transaction history - earned credits - used credits - expiration rules if

introduced later Example: ``text ₹49 No match found. ₹49 has been credited to your SwapSeat Wallet. Use it on your next journey. `` Wallet credits must be recorded server-side. --- # 23.

DATABASE MODEL Create relational backend tables similar to: ``text users profiles traveller_verifications journeys journey_segments transport_services bookings booking_passengers booking_seats seat_maps seat_map_versions seat_positions swap_preferences swap_requests swap_request_targets matches payments payment_intents payment_transactions wallets wallet_transactions chat_threads chat_messages journey_events journey_status notifications ratings reports blocks audit_logs `` --- # 24. IMPORTANT DATA

MODEL Do NOT store seat layout as UI-specific HTML. Store seats as structured data.

Example: ``json { "transportType": "bus", "layoutType": "2x2", "deck": "lower", "seats": [{ "id": "S21", "label": "21", "row": 10, "column": 1, "position": "left", "seatType": "window" }] }

`` Train and aircraft layouts should use the same concept. The frontend renders the layout from data. --- # 25. MATCHING ENGINE Match using weighted criteria. Example: ``text same service same date source/destination overlap seat preference coach compatibility window/aisle berth type approximate location traveller status number of seats `` Create a match score.

Example: ``text 92% Match `` But make the score explainable. Example: ``text Same journey ✓ Same destination ✓ Window preference ✓ Same coach ✓ Family seating ✓ `` --- # 26.

MULTI-SEAT SUPPORT Never assume one traveller = one seat. A user may have: - 1 seat - 2 seats - 3 seats - family/group seats Allow: **Add another seat** and multi-selection throughout the journey. Matching must understand: ``text requestSeats = 3 targetSeats = 3 `` not only one-to-one swaps. --- # 27. MULTI-REQUEST LOGIC Requester may send requests to several

compatible travellers. Example: ``text 8 compatible travellers found Select: ✓ Rahul ✓ Priya ✓ Sneha ✓ Arjun `` Once one successful swap is completed: ``text cancel pending requests lock journey reveal seats notify all other pending users `` --- # 28. API ARCHITECTURE Create a

transport-provider abstraction. ``typescript interface TransportProvider { searchServices(...) fetchBooking(...) getSeatMap(...) getTripStatus(...) } `` Implement: ``text TrainProvider BusProvider FlightProvider `` Do not couple the frontend directly to a specific transport

provider. This makes India → global expansion much easier. --- # 29. FALLBACK DATA

MODE VERY IMPORTANT FOR MVP. If live transport data/API is unavailable: Allow: ``text Manual Entry `` The user can enter: - source - destination - service number - date - seat number(s) - coach/cabin/deck The product must remain usable without a live API. --- # 30.

ADMIN DASHBOARD Build a private admin console. Admin must be able to: - view users - verify users - inspect journeys - inspect matches - inspect payments - issue wallet credits - refund

payments - suspend users - block users - investigate reports - view failed matches - see request lifecycle - inspect audit logs --- # 31. PAYMENT ARCHITECTURE Use a payment abstraction.

```
``typescript interface PaymentProvider { createPayment(...) verifyPayment(...)
refundPayment(...) } ``
```

Do NOT hard-wire payment business logic into the UI. The server decides whether: - ₹49 search fee - ₹99 one-sided completion fee - wallet credit - refund is required. Never trust a frontend payment-status value. --- # 32. NOTIFICATIONS Support: - push notification - email - SMS/OTP where appropriate Important events: - match found - request received - request accepted - request declined - payment required - payment successful - seat numbers unlocked - swap confirmed - journey approaching - traveller messaged - journey started - destination reached - chat closed --- # 33. SECURITY Must include: - authentication - OTP/email verification - authorization - payment verification server-side - rate limiting - abuse prevention - request throttling - report/block - audit trail Never expose: - another traveller's PNR - payment details - unnecessary booking identifiers - exact seat information before the correct state - personal information unnecessarily --- # 34. DESIGN SYSTEM Use the supplied HD mobile screenshots as visual references. DO NOT paste screenshots into the app. Rebuild the UI as actual responsive components. Visual characteristics: - white base - clean blue/green accents - rounded cards - subtle shadows - large readable headings - strong primary CTA - clear status indicators - minimal visual clutter - mobile-first spacing - consistent iconography Use reusable components:

```
``text AppHeader TransportSelector JourneyCard BookingCard SeatMap SeatLegend
SeatPreference ComfortScore MatchCard TravellerCard PaymentCard RequestStatus WalletCard
Chat JourneyTimeline ConfirmationCard ``
```

--- # 35. SCREEN IMPLEMENTATION RULE Every screen in the supplied reference images must become a real route/state in the application. Do not skip screens. Do not merge multiple important states into one screen merely to reduce development time. Important transitions must be functional: ``text screen A → CTA → screen B → CTA → screen C `` --- # 36. UX QUALITY BAR The app should feel like a product that could be shipped. Avoid: - generic AI-generated dashboards - desktop-like cards - unnecessary hamburger menus - random tabs - placeholder buttons - fake interactions - meaningless charts - generic stock UI Use realistic mobile interaction: - bottom sheets - inline selectors - segmented controls - sticky CTAs - confirmation sheets - modal payment states - skeleton loading - empty states - error states - success states --- # 37. MVP DEVELOPMENT ORDER Build in this order:

```
#### Phase 1 Shared mobile shell - authentication - home - profile - wallet - notifications ####
Phase 2 Train engine - journey input - booking/manual entry - seat maps - preference - payment - matching - requests - confirmation ####
Phase 3 Bus engine - bus search - timings - bus layouts -
```

matching ### Phase 4 Flight engine - PNR + surname - booking retrieval - aircraft layouts - flight-specific matching ### Phase 5 Cross-platform infrastructure - chat - notifications - journey tracking - ratings - reporting - wallet ### Phase 6 Admin + analytics Track: ``text journey searches payment conversion match rate request acceptance rate successful swap rate no-match wallet credits average matching time cancellation rate repeat usage `` --- # 38. MOST IMPORTANT PRODUCT PRINCIPLE SwapSeat is NOT: > "Show me somebody's seat number." SwapSeat is: > "Understand what kind of seat I have, understand what kind of seat another traveller wants, find compatible people, facilitate trust and payment, and reveal the exact exchange only when the transaction is protected." This principle should drive the entire architecture. ## What I would upload alongside that prompt Use the HD images you've already created as **design references**, organized like this: ``text /design /shared home-screen.png /train requester-flow.png acceptor-flow.png /bus requester-flow.png acceptor-flow.png /flight requester-flow.png acceptor-flow.png `` Also give the builder the **actual train seat-map reference images** you supplied, because the seat-map renderer is one of the most important parts of the product. ## Recommended technical structure For an MVP, I would use: **Frontend:** Next.js + TypeScript + Tailwind + PWA **Backend:** PostgreSQL/Supabase-style backend **Realtime:** WebSocket/realtime layer for .

Gpt 6 Astra

Yes. Give the coding platform a structured package instead of only screenshots or a long product description.

Create this folder:

```
TEXT
swapseat-build/
├── SWAPSEAT_MASTER_SPEC.md
├── AI_AGENT_INSTRUCTIONS.md
├── PRODUCT_RULES.md
├── VIRAL_GROWTH_PLAN.md
├── DESIGN_REFERENCES/
│   ├── home.png
│   ├── transport-selection.png
│   ├── journey-details.png
│   ├── seat-map.png
│   ├── preferences.png
│   ├── payment.png
│   ├── match-found.png
│   ├── request-status.png
│   └── group-journey.png
├── SAMPLE_DATA/
│   ├── trains.json
│   ├── buses.json
│   └── flights.json
```

seat-maps/
README.md

Use the following as AI_AGENT_INSTRUCTIONS.md.

MD

SwapSeat - AI Coding Agent Instructions

You are building SwapSeat, a mobile-first progressive web app for coordinating possible seat exchanges on trains, buses, and flights.

Read all project files before making changes.

First task

Do not immediately implement the full product.

First inspect the repository and report:

1. Framework and language.
2. Existing routes and components.
3. Existing database.
4. Authentication setup.
5. Payment setup.
6. PWA support.
7. Existing design system.
8. Existing tests.
9. Deployment setup.
10. Which parts can be reused.
11. Which parts are missing.
12. Any conflicts with the product specification.

Then provide a phased implementation plan.

Do not replace the existing stack unless there is a strong technical reason.

Implementation order

Build in this order:

Phase 1: Mobile product shell

- Responsive mobile-first layout.
- PWA manifest.
- Service worker.
- Home screen.
- Transport selector.
- Bottom navigation.
- Authentication placeholder or existing authentication.
- Profile.
- Notifications.
- Wallet placeholder.

Phase 2: Train MVP

- Train journey input.
- Manual journey entry.
- Sample train provider.

- Data-driven coach layouts.
- Seat-map renderer.
- Current seat selection.
- Preference selection.
- Match preview.
- Swap request flow.
- Request status.
- Mock payment flow.

Phase 3: Real backend

- PostgreSQL schema.
- Server-side authorization.
- Journey records.
- Seat claims.
- Preferences.
- Match candidates.
- Swap requests.
- Request expiry.
- Notifications.
- Audit logs.

Phase 4: Group virality

- Create group journey.
- Invite members.
- Safe deep links.
- WhatsApp sharing.
- Family/group map.
- Group dashboard.
- Referral tracking.
- Abuse limits.

Phase 5: Payments

- Payment provider abstraction.
- ₹49 configurable search fee.
- Wallet credit when no match is found.
- Optional ₹99 completion fee for one-sided requests.
- Refund and credit logic.
- Server-side payment verification.

Phase 6: Bus and flight engines

- Bus service search and map layouts.
- Flight booking/manual entry.
- Aircraft-specific map layouts.
- Transport-specific matching rules.

Required rules

- Build actual working interactions, not static screenshot replicas.
- Use the supplied screenshots as visual references only.
- Do not paste screenshots into the app.
- Use reusable components.
- Keep all seat layouts data-driven.
- Keep matching and payment decisions server-side.
- Never trust payment state from the client.
- Never expose private booking data.

- Never show an unknown seat as available.
- Never claim an official ticket change unless an official integration confirms it.
- Add loading, empty, error, offline, and success states.
- Add tests for critical flows.
- Keep the app usable with manual entry when live APIs are unavailable.
- Make every mutation idempotent.
- Add accessible labels and a list alternative for visual maps.
- Test on narrow mobile screens before desktop layouts.

First implementation acceptance criteria

The first milestone is complete when a user can:

1. Open the PWA.
2. Choose Train.
3. Enter a sample journey.
4. Select Sleeper or AC 3 Tier.
5. Select a coach.
6. See a visual coach map.
7. Select their own berth.
8. Choose desired berth features.
9. See a sample compatible match.
10. Compare current and proposed seats.
11. Start a swap request.
12. See the request status.
13. Open the journey offline.
14. Understand which data is illustrative, expected, or verified.

Run:

```
```bash
npm run lint
npm run typecheck
npm test
npm run build
```

If a command does not exist, report that clearly and run the closest available validation.

Use this as `PRODUCT\_RULES.md`.

```
```md
# SwapSeat Product Rules

## Product positioning
```

SwapSeat is a passenger coordination platform.

It does not automatically transfer, sell, modify, or reissue transport tickets.

Use this wording:

> Coordinate a possible seat exchange.

Do not use:

> Transfer your ticket.

Use this wording:

> Both passengers accepted the coordination request.

Do not use:

> Your booking has changed.

Transport selection

The first question is:

> What are you travelling by?

Options:

- Train
- Bus
- Flight

The user then enters:

- Source.
- Destination.
- Travel date.
- Service, train, bus, or flight details.
- Booking reference or manual details.
- Current seat.
- Number of travelers.

Booking reference fields must be optional when possible.

Privacy

Never expose another user's:

- PNR.
- Booking ID.
- Ticket barcode.
- QR code.
- Full ticket image.
- Phone number.
- Email.
- Legal name.
- Home address.
- Payment details.

Exact seat information may be restricted until the required request/payment state is reached, but the product should still provide enough information for users to judge whether a request is relevant.

Before revealing exact private details, show:

- Transport.
- Journey.
- Segment.
- Coach, cabin, or deck.
- Approximate position.

- Seat characteristics.
- Match reasons.
- Verification status.

User journey

```text

Choose transport

→ Enter journey

→ Retrieve or manually enter booking

→ See current seat visually

→ Choose swap reason

→ Choose desired seat features

→ Preview possible matches

→ Pay or activate request

→ Send request

→ Other traveler accepts

→ Complete required payment state

→ Reveal confirmed exchange details

→ Coordinate within journey-bound chat

## Gender

Gender must not be mandatory.

Store it only when necessary and with consent.

Possible preference:

- No preference.
- Same-gender preference.
- Prefer not to say.

Do not publicly expose gender by default.

## Payment

All pricing must be configurable on the backend.

Default test values:

### TEXT

SEARCH\_FEE = 49 INR

ONE\_SIDED\_COMPLETION\_FEE = 99 INR

The ₹49 fee is for activating a search/request service. It is not payment for another passenger's seat.

If no suitable match is found:

#### TEXT

Refund or credit ₹49 to SwapSeat Wallet

Do not promise a successful match.

Do not charge both parties a second search fee when both already paid.

The backend decides:

- Whether a fee is required.
- Whether a wallet credit is issued.
- Whether a refund is available.
- Whether a second completion payment is required.

## Matching

Match only when the following are compatible:

- Same transport service.
- Same travel date.
- Compatible source and destination segment.
- Compatible class/cabin/deck.
- Compatible seat preferences.
- Both users opted into matching.
- No active conflicting request.
- No blocked relationship.
- No restricted seat conflict.

For trains, segment overlap is:

#### TS

```
Math.max(a.boardingStopIndex, b.boardingStopIndex) <
Math.min(a.destinationStopIndex, b.destinationStopIndex)
```

## Seat-map confidence

Every seat map must show one of:

- Verified layout.
- Expected layout.
- User-confirmed layout.
- Illustrative layout.

If the system cannot confidently identify the layout, ask the user to confirm it.

## Request state machine

**TEXT**  
draft

→ journey\_selected  
→ preferences\_selected  
→ payment\_pending  
→ matching  
→ request\_sent  
→ accepted  
→ completion\_payment\_required  
→ confirmed  
→ completed

Failure states:

**TEXT**

declined  
expired  
cancelled  
no\_match  
invalidated  
disputed

## Multi-seat support

Do not assume one user has only one seat.

Support:

- Individual travelers.
- Couples.
- Families.
- Group bookings.

Requests must include:

**TEXT**

```
requestedSeatCount
targetSeatCount
```

Do not complete a three-seat request with only one compatible seat.

## Chat

Chat becomes available only after the request reaches the appropriate confirmed state.

Chat is connected to the journey and closes after the destination or configured journey end.

```
Use this as `VIRAL_GROWTH_PLAN.md`.
```

```
```md
```

```
# SwapSeat Viral Growth Plan
```

```
## Core growth loop
```

```
```text
```

```
One traveler creates a journey
```

```
→ Invites family or travel group
```

```
→ More travelers join the same journey
```

```
→ More seats and preferences become visible
```

```
→ Match quality improves
```

```
→ More successful requests occur
```

```
→ Users share their result
```

```
→ More travelers create future journeys
```

## Loop 1: Group journey invitations

When a user creates a journey, immediately show:

Travelling with family or friends?

Actions:

- Create group journey.
- Invite on WhatsApp.
- Copy safe invite link.
- Share through system share sheet.

The invitation must prefill:

- Transport type.
- Service.
- Travel date.

- Source.
- Destination.

The recipient should not need to re-enter the journey.

Do not include:

- PNR.
- Ticket number.
- Phone number.
- Private seat details.
- Legal name.

## Loop 2: Family map

Let a group owner create a shared visual journey:

### TEXT

Create family trip

→ Invite members

→ Each member adds their seat

→ Everyone appears on the coach/cabin map

→ Group identifies seating problems

→ Group shares the map

This creates immediate value even when there are no strangers available for matching.

## Loop 3: No-match sharing

When no match exists, show:

We need more travelers on this journey.

Actions:

- Share to WhatsApp.
- Copy invite.
- Share to X.
- Share through native system sharing.

Suggested message:

### TEXT

I am traveling on [service] from [source] to [destination] on [date].  
I am looking for a seat swap. Join this private journey group:  
[[link]]

The link must open a safe landing page and require authentication before showing private details.

## Loop 4: Group organizer mode

Support organizers for:

- Weddings.
- College trips.
- Religious groups.
- Tour groups.
- Corporate travel.
- Family reunions.

Organizer features:

- Group invite link.
- Passenger list.
- Seat-map view.
- Missing member reminders.
- Group notifications.
- Passenger status.
- Paid group dashboard.

## Loop 5: Shareable success card

After a completed coordination request, generate an optional card:

### TEXT

Our journey seating was coordinated with SwapSeat.  
[service] · [date]

Never put private ticket data on the card.

## Loop 6: Referral rewards

Use non-cash rewards:

- Free group journey.
- Free search credit.
- Extra active journey.
- Saved preferences.
- Group organizer trial.

Do not reward unlimited invitations.

Apply:

- Invitation limits.
- Duplicate detection.
- Invalid-link detection.
- Account-level abuse controls.

## Loop 7: Search-engine landing pages

Create public informational pages for:

- Supported train layouts.
- Bus seat layouts.
- Aircraft seat-layout explanations.
- “How to find your berth.”
- “Lower vs middle vs upper berth.”
- Route-specific travel guides.

Public pages must not reveal current passengers.

Primary CTA:

Add your journey and see your seat map.

## Growth metrics

Track:

- Journey creation rate.
- Invite creation rate.

- Invite acceptance rate.
- Group members per journey.
- Match coverage.
- Request acceptance rate.
- Completed coordination rate.
- No-match rate.
- Referral conversion.
- Repeat journey creation.
- Paid group conversion.
- Abuse per 1,000 invites.

## North-star metric

Use:

Completed useful journey coordination per active departure.

Do not optimize only for app installs.

Use this as `README.md`.

```md

SwapSeat Build Package

Product

SwapSeat is a mobile-first PWA for coordinating possible seat exchanges on trains, buses, and flights.

Initial launch

Start with Indian trains.

Recommended first implementation:

- English.
- Hindi-ready localization.
- Manual train entry.
- Sample Sleeper and AC 3 Tier layouts.
- Mobile PWA.
- Group invitations.
- Visual seat maps.
- Mock matching.
- Mock payments.
- No live railway integration initially.

Design references

The files in `DESIGN\_REFERENCES/` are visual references.

Rebuild them as responsive components. Do not embed them as screenshots.

Required first route flow

```
```text
/
→ /journey/new
→ /journey/[id]/seat-map
→ /journey/[id]/preferences
→ /journey/[id]/matches
→ /requests/[id]
```

## Required reusable components

**TEXT**

- AppShell
- BottomNavigation
- TransportSelector
- JourneyForm
- ServiceSearch
- SeatMap
- CoachSelector
- SeatDetailSheet
- SeatLegend
- PreferenceSelector
- MatchCard
- SwapPreview
- PaymentCard
- RequestTimeline
- GroupInviteCard
- WalletCard
- OfflineBanner
- VerificationBadge

## Required tests

- User can create a journey.
- User can select a berth.
- User can view a visual map.
- Unknown seats are not marked available.
- Preferences produce deterministic mock matches.
- A request can be accepted.
- Expired requests cannot be accepted.
- Two concurrent acceptances cannot double-commit a seat.

- Offline mode cannot confirm a live request.
- Private booking data is not returned to unauthorized users.
- Group invite does not expose private journey data.

## Give the coding platform this first message

```text

Build SwapSeat from the attached specification files and design references.

First inspect the repository. Do not make major changes yet.

Return:

1. Repository assessment.
2. Existing architecture.
3. Reusable components.
4. Missing infrastructure.
5. Database recommendation.
6. Route plan.
7. First milestone plan.
8. Risks and unresolved assumptions.

Then implement only Milestone 1:

- Mobile-first PWA shell.
- Home screen.
- Train-first journey setup.
- Sample Indian train data.
- Sleeper and AC 3 Tier visual maps.
- Current berth selection.
- Preferences.
- Mock match preview.
- Swap request state.
- Offline cached demo journey.
- Accessible list alternative.

Do not implement flights, live APIs, real payments, or ticket OCR yet.

Use the supplied images as design references, not as page backgrounds.

Important architectural correction

Do not build three separate products.

Build one shared platform:

TEXT

SwapSeat Core

- Journey Engine
- Seat Map Engine
- Matching Engine
- Payment Engine
- Wallet Engine

```
└─ Request State Machine
└─ Chat Engine
└─ Notification Engine
└─ Trust and Moderation
└─ Admin Console
```

Then add transport adapters:

TEXT

```
Transport Adapters
└─ IndianTrainAdapter
└─ BusAdapter
└─ FlightAdapter
```

Each adapter provides:

TS

```
interface TransportAdapter {
  searchServices(input: SearchServicesInput): Promise<Service[]>;
  resolveBooking(input: BookingLookupInput): Promise<Booking>;
  getSeatMap(input: SeatMapInput): Promise<SeatMapResolution>;
  getTripStatus(input: TripStatusInput): Promise<TripStatus>;
  validateSwap(input: SwapValidationInput): Promise<SwapValidation>;
}
```

This lets you start with Indian trains while keeping the architecture ready for buses, flights, and international providers.

Recommended implementation milestones

1. Design system and mobile PWA shell.
2. Train journey flow with mock data.
3. Visual seat-map engine.
4. Authentication and database.
5. Matching and swap requests.
6. Group invitations and viral sharing.
7. Payment and wallet.
8. Admin moderation.
9. Live train-provider integration.
10. Bus support.
11. Flight support.

The most important first success criterion is:

A user can understand their current seat, desired seat, and proposed swap visually within one minute.

That should be proven before investing in live APIs, payment complexity, or expansion to buses and flights.

you

do you have anything to add for making this go viral. for global users let them select country in pwa and see the options we have needs tweaked if yes tweak for country specific . keep the pricing according to the countries purchasing power. let users give feedback or comments to improvement have this as a section in settings .

Gpt 6 Astra

Yes. Add a **country-aware product layer**, a structured feedback system, and stronger viral loops based on group density and successful coordination.

The key principle should be:

One global platform, locally adapted journeys, rules, payments, languages, and pricing.

Do not create completely separate country versions. Build one core product with country-specific configuration.

Add These Files

Add the following files to the implementation package:

TEXT

COUNTRY_LOCALIZATION_ARCHITECTURE.md

GLOBAL_PRICING_STRATEGY.md

VIRAL_GROWTH_SYSTEM.md

FEEDBACK_AND_IMPROVEMENT.md

Also add these folders:

TEXT

SAMPLE_DATA/

countries.json

transport-options.json

country-configurations/

india.json

united-states.json

united-kingdom.json

united-arab-emirates.json

singapore.json

australia.json

canada.json

1. Country Selection

Do not ask users to select a country only during signup. Let them change it later from Settings.

First launch

Show:

TEXT

Where are you travelling?

[Select country]

Then show:

TEXT

What would you like to do?

[Train]

[Bus]

[Flight]

The selected country should influence:

- Available transport types.
- Operators.
- Search fields.
- Booking identifiers.
- Currency.
- Language options.
- Date and time format.
- Payment methods.
- Seat-map conventions.
- Privacy wording.
- Verification methods.
- Supported providers.
- Pricing.

- Customer support options.
- Local safety and policy information.

Country selector design

Use searchable country cards:

TEXT

Choose your travel country

[Search countries]

Popular:

IN India

US United States

GB United Kingdom

AE United Arab Emirates

SG Singapore

AU Australia

All countries

Do not force users to select their residence country. Ask for:

TEXT

Travel country

Optionally collect:

TEXT

Home country

only when needed for payments, language, or support.

A user may travel internationally, so their selected country should be attached to the journey, not permanently define their account.

2. Country-Aware Product Model

The country belongs primarily to the journey.

TS

```
interface TravelContext {  
  countryCode: string;  
  regionCode?: string;  
  currency: string;
```

```
    locale: string;
    timezone: string;
    transportTypes: TransportType[];
    supportedProviders: string[];
}
```

A user profile can have a default country, but every journey must have its own travel country.

Example:

TEXT

User profile:
Home country: India

Journey 1:
India · Train · INR

Journey 2:
United Kingdom · Train · GBP

Journey 3:
United States · Flight · USD

3. Country Configuration

Each country should have a configuration file or database record.

```
TS
interface CountryConfiguration {
    countryCode: string;
    name: string;
    flag: string;
    defaultLocale: string;
    supportedLocales: string[];
    defaultCurrency: string;
    supportedCurrencies: string[];
    timezoneStrategy: "journey" | "user" | "service";
    transportTypes: TransportType[];
    providers: ProviderConfiguration[];
    bookingFields: BookingFieldConfig[];
    seatMapRules: SeatMapRules;
    matchingRules: MatchingRules;
    paymentMethods: PaymentMethod[];
    pricingPlanId: string;
    legalNoticeKey: string;
    supportChannels: SupportChannel[];
    launchStatus: "active" | "beta" | "coming_soon";
}
```

Example:

JSON

```
{
  "countryCode": "IN",
  "name": "India",
  "defaultLocale": "en-IN",
  "supportedLocales": ["en-IN", "hi-IN"],
  "defaultCurrency": "INR",
  "transportTypes": ["train", "bus", "flight"],
  "launchStatus": "active",
  "paymentMethods": ["upi", "card", "netbanking"],
  "pricingPlanId": "india-default"
}
```

4. Country-Specific Onboarding

After selecting a country, show only the relevant options.

India

TEXT

Available now:

- ✓ Indian trains
- ✓ Intercity buses
- ✓ Selected flights

Journey fields may include:

- Train number.
- PNR or manual details.
- Source.
- Destination.
- Coach.
- Berth.
- Boarding station.
- Class.
- Bus operator.
- Flight number.
- Airline.

- Booking reference.

United States

Potential fields:

- Rail operator.
- Bus operator.
- Flight number.
- Confirmation code.
- Seat number.
- Origin and destination.
- Departure time.
- Aircraft or vehicle type.

United Kingdom

Potential fields:

- Train operator.
- Rail service.
- Departure station.
- Arrival station.
- Coach and seat.
- Reservation reference.

General fallback

Every country must support:

TEXT

Manual entry

If live provider support is unavailable, say:

TEXT

Live lookup is not available for this journey yet.
You can enter your journey and seat manually.

Do not show unsupported providers as if they are integrated.

5. Global Pricing Strategy

Do not convert ₹49 directly into every currency.

Use country-specific pricing plans based on:

- Local purchasing power.
- Local payment fees.
- Taxes.
- Payment conversion.
- Expected support cost.
- Travel category.
- Match density.
- User willingness to pay.
- Whether the market is beta or established.

Pricing principles

1. Basic browsing should remain free.
2. Users should see likely match availability before paying.
3. Charge for request activation or coordination convenience.
4. Do not charge for simply receiving a request.
5. Do not charge for accepting a request.
6. Credit or refund users when no suitable match is found.
7. Do not promise a successful swap.
8. Use local currency.
9. Show taxes and fees clearly.
10. Allow promotional pricing during launch.

Pricing model

TS

```
interface PricingPlan {  
  id: string;  
  countryCode: string;  
  currency: string;  
  searchFeeMinor: number;  
  completionFeeMinor?: number;  
  groupPassFeeMinor?: number;  
  noMatchOutcome: "refund" | "wallet_credit" | "partial_credit";  
  taxIncluded: boolean;  
  active: boolean;  
}
```

Use minor currency units:

TEXT

INR 4900 = ₹49.00
USD 499 = \$4.99
GBP 399 = £3.99

Do not store money as floating-point values.

Pricing tiers

Use three localized tiers:

TEXT

Entry
Standard
Group

Example structure:

TEXT

India:

Entry: ₹29-₹49

Group: ₹99-₹199

United States:

Entry: \$2.99-\$4.99

Group: \$9.99-\$14.99

United Kingdom:

Entry: £2.49-£3.99

Group: £7.99-£11.99

UAE:

Entry: AED 9-15

Group: AED 29-49

These should be configured and tested, not permanently hardcoded.

Currency display

Show:

| |
|--------|
| TEXT |
| ₹49 |
| \$4.99 |
| £3.99 |
| AED 12 |
| €3.99 |

Avoid showing a converted foreign price beside the local price unless useful.

Regional experiments

Allow the admin dashboard to configure:

- Country.
- Currency.
- Price.
- Promotional price.
- New-user discount.
- Group discount.
- Refund policy.
- Wallet-credit policy.
- Payment methods.
- Tax behavior.
- Experiment cohort.

6. Better Viral Growth System

The strongest viral loop is not a generic referral code.

It is:

One journey creates a reason to invite people who are already travelling together.

Viral loop

TEXT

Create a journey

- Invite group members
- Group members add their seats
- Shared map exposes seating problems
- Group requests become possible
- More members join
- Group receives useful coordination
- Members use SwapSeat on future journeys
- New groups are created

Feature: One-tap group invite

After creating a journey, immediately display:

TEXT

Travelling with family or friends?

Invite them to your journey map.

Actions:

TEXT

- [WhatsApp]
- [Copy invite]
- [Share]

The invite should prefill:

- Country.
- Transport mode.
- Service.
- Date.
- Source.
- Destination.

The receiver should open a mobile page with:

TEXT

Join this journey

Mumbai → Delhi

Train 12951

24 October

[Join journey]

After authentication, the journey details should be prefilled.

Feature: Group seat map

Allow the owner to see:

- Who has joined.
- Who has not added a seat.
- Which members are split across coaches.
- Which members are far apart.
- Who wants a different seat.
- Whether the group has possible internal swaps.

Use statuses:

TEXT

Invited

Joined

Seat added

Preferences added

Needs help

Completed

Feature: Invite progress

Create a progress card:

TEXT

3 of 6 travelers joined

Invite the rest to improve your group's seat options.

[Invite remaining travelers]

This is more motivating than a generic referral message.

Feature: Group unlocks

Reward group activity:

TEXT

2 members joined:

Shared map unlocked

4 members joined:

Group coordination unlocked

6 members joined:

Free group trip pass unlocked

The reward should encourage real journey participants, not fake referrals.

Feature: No-match rescue

When there are no compatible users:

TEXT

No suitable match yet.

More passengers on this journey means better options.

[Invite someone travelling with you]

[Share anonymously]

[Make preferences flexible]

Feature: Public informational pages

Create SEO-friendly public pages for:

- Train coach layouts.
- Bus seat layouts.
- Airline seat explanations.
- Window versus aisle guides.
- Lower versus middle versus upper berth guides.
- Country-specific journey instructions.

Each page should lead to:

TEXT

Add your journey

Never show current passenger lists publicly.

Feature: Successful coordination card

After a completed request:

TEXT

Seat coordination completed

Train 12951 · Mumbai to Delhi
Coordinated safely with SwapSeat

Allow sharing only if the user chooses.

Do not include:

- PNR.
- Exact ticket details.
- Phone numbers.
- Legal names.
- Payment information.

Feature: Travel community referrals

Build referral partnerships with:

- Tour operators.
- Student travel groups.
- Family travel communities.
- Religious travel organizers.
- Travel creators.
- Bus operators.
- Hostels.
- Travel agencies.

Give organizers:

- Group dashboard trial.
- Free group pass.
- Invite analytics.
- Journey coordination tools.

7. Avoid Viral Abuse

Every viral feature needs limits.

Implement:

- Invite rate limits.
- Link expiration.
- Revoke invite.
- Duplicate invite detection.
- Report invite.
- Block user.
- No unsolicited SMS.
- No automatic WhatsApp sending.
- Explicit consent for notifications.
- Private journey membership.
- No public passenger directory.
- Suspicious referral detection.
- Device and account abuse checks.

Never reward raw invite volume. Reward verified journey participation.

8. Feedback and Improvement Section

Add a dedicated section in Settings:

TEXT

Settings

- Account
- Language and country
- Notifications
- Privacy and safety
- Wallet and payments
- My journeys
- Help centre
- Give feedback
- Request a feature
- Report a problem
- About SwapSeat

Feedback landing screen

TEXT

Help us make SwapSeat better

What would you like to tell us?

[Suggest an improvement]

[Report a problem]

[Report an incorrect seat map]

[Tell us about your journey]

[Rate your experience]

Suggestion form

Fields:

TEXT

What should we improve?

[Text area]

Which part of SwapSeat?

[Journey setup]

[Seat map]

[Matching]

[Payment]

[Notifications]

[Group journey]

[Other]

How important is this?

[Nice to have]

[Important]

[Could not use the app without it]

[Submit feedback]

Bug report form

TEXT

What went wrong?

[Select issue]

Journey:

[Optional journey selector]

Description:

[Text area]

Attach screenshot:

[Optional]

[Send report]

Automatically attach:

- App version.
- Device type.
- Operating system.
- Country configuration.
- Locale.
- Current screen.
- Anonymous error identifier.

Do not automatically attach:

- Ticket images.
- PNR.
- Payment credentials.
- Personal messages.

Seat-map correction form

Place this directly on the seat map:

TEXT

Is this layout incorrect?

[Report map issue]

Options:

TEXT

My seat is shown in the wrong place

Coach layout is different

Missing seat

Missing toilet or door

Wrong seat label

Other

Allow the user to select the incorrect seat visually.

Post-journey feedback

After the travel date:

TEXT

How did your SwapSeat experience go?

[It helped me]
[I found a match but did not complete it]
[No match was available]
[The map was incorrect]
[I had a safety concern]
[Other]

Then ask:

TEXT
How likely are you to recommend SwapSeat?
0 – 10

Keep this short.

9. Feedback Admin System

Create an admin feedback dashboard.

Admin can filter by:

- Country.
- Transport type.
- Provider.
- App version.
- Language.
- Feedback category.
- Severity.
- Status.
- Date.
- Journey type.

Feedback states:

TEXT
new
→ acknowledged
→ under review
→ planned
→ in progress

→ released
→ declined

Admin fields:

TEXT
title
description
category
country
transportType
priority
votes
status
internalNotes
publicResponse
createdAt
resolvedAt

Public roadmap

Later, create an optional public page:

TEXT
You asked
We are building
Released

Users can vote on suggestions, but do not expose private feedback.

Example:

TEXT
Add Marathi language support
312 votes
[Vote]

Require authentication to limit duplicate votes.

10. Feedback Flywheel

Build this loop:

TEXT
User completes journey
→ Gives feedback

```
→ Product team identifies country-specific problem
→ Fix is released
→ User is notified
→ User trusts the product more
→ User invites more travelers
→ More journey data and feedback arrives
```

Use visible release notes:

TEXT

New in SwapSeat

```
✓ Better AC 3 Tier maps in India
✓ Added Hindi support
✓ Added group invites for family journeys
✓ Added AED payments in the UAE
```

This helps users see that their feedback produces real improvements.

11. Country-Specific Feature Flags

Use configuration and feature flags rather than code branches.

TS

```
interface CountryFeatureFlags {
  trainSearch: boolean;
  busSearch: boolean;
  flightSearch: boolean;
  manualEntry: boolean;
  bookingLookup: boolean;
  groupJourneys: boolean;
  walletCredits: boolean;
  pushNotifications: boolean;
  liveTripStatus: boolean;
  chat: boolean;
  localizedPricing: boolean;
}
```

Example:

JSON

```
{
  "countryCode": "IN",
  "features": {
    "trainSearch": true,
    "busSearch": true,
    "flightSearch": true,
    "manualEntry": true,
    "bookingLookup": false,
    "groupJourneys": true,
    "walletCredits": true,
```

```
"localizedPricing": true
}
}
```

User-facing availability states

Show:

TEXT
Available now
Beta
Manual entry available
Coming soon

Never show a disabled option without explaining why.

12. Global Architecture Update

Use this structure:

TEXT
SwapSeat Core
— Authentication
— User Profiles
— Travel Context
— Journey Engine
— Transport Adapter Registry
— Seat Map Engine
— Booking Resolver
— Matching Engine
— Swap Request State Machine
— Payment Engine
— Wallet Engine
— Chat Engine
— Notification Engine
— Feedback Engine
— Moderation Engine
— Country Configuration
— Localization
— Feature Flags
— Analytics
— Admin Console

Transport Adapters
— IndiaTrainAdapter
— IndiaBusAdapter
— IndiaFlightAdapter
— UKTrainAdapter
— USRailAdapter
— GlobalBusAdapter

GlobalFlightAdapter

Country-aware request context

Every API request should be able to resolve:

TEXT

user
country
journey
transport
currency
locale
timezone
feature flags
pricing plan
provider availability

Do not determine pricing or availability only in the frontend.

13. Analytics for Virality

Track the full invite funnel:

TEXT

journey_created
group_prompt_seen
invite_created
invite_shared
invite_opened
invite_authenticated
invite_joined
seat_added
preference_added
group_match_found
group_request_sent
group_request_completed
referral_journey_created

Measure:

- Invite share rate.
- Invite open rate.
- Invite join rate.
- Seats added per invited journey.

- Group members per journey.
- Match rate before and after invitations.
- Percentage of journeys receiving a match from an invite.
- Repeat usage by invited users.
- Country-specific viral coefficient.
- Cost per activated journey.

North-star metric

Use:

Completed useful coordination events per active departure.

Secondary growth metric:

Verified journey participants invited per activated journey.

Do not use raw downloads as the main success metric.

14. Strong Product Loop by Country

Each country should launch with a specific wedge.

Examples:

TEXT

India:

Family train journeys and berth coordination

United Kingdom:

Train seat and group coordination

United States:

Intercity bus and flight group seating

United Arab Emirates:

Flight and long-distance bus seating

Singapore:

Flight and coach group coordination

Australia:

Flight and coach group journeys

These are launch hypotheses, not permanent assumptions. Validate each market before investing heavily.

15. Important Pricing Improvement

Use this flow:

TEXT

Add journey
→ See current seat
→ Choose desired seat
→ Show estimated match availability
→ Show local price
→ Pay to activate request

Do not ask for payment before the user knows whether the journey has potential matches.

Example:

TEXT

We found 4 possible matches on this journey.

Activate your request for ₹49?

For no-match outcomes:

TEXT

No suitable match was found.

Your ₹49 has been added to your SwapSeat Wallet.

The wallet should have a clear expiration policy, or no expiration during the initial launch. Avoid confusing users with complicated credit rules.

16. Updated Coding-Agent Instruction

Add this to the master prompt:

TEXT

Implement SwapSeat as a global platform with country-aware configuration.

The selected country must be attached to each journey and must control:

- Available transport modes.
- Providers.
- Search fields.
- Booking fields.
- Seat-map conventions.
- Language.
- Currency.
- Pricing plan.
- Payment methods.
- Feature flags.
- Legal notices.
- Support options.

Do not create country-specific hardcoded UI branches.
Use country configuration objects and provider adapters.

Add Settings sections for:

- Country and language.
- Currency display.
- Privacy.
- Notifications.
- Wallet.
- Give feedback.
- Report a problem.
- Request a feature.

Implement a feedback system with:

- Improvement suggestions.
- Bug reports.
- Incorrect seat-map reports.
- Post-journey feedback.
- Optional ratings.
- Admin moderation and status tracking.

Implement viral growth features:

- Group journey creation.
- Safe invite links.
- WhatsApp/system sharing.
- Group member progress.
- Family map.
- No-match sharing.
- Referral rewards based on verified journey participation.
- Invite abuse prevention.

Do not reward raw invite count.

Do not expose private booking information through invites.

Do not make pricing global or hardcoded.

Use backend country-specific pricing plans stored in minor currency units.

Final recommendation

Your strongest viral product is not a generic seat marketplace. It is a **shared journey room**:

One traveler creates a journey, invites everyone traveling with them, the group sees the visual seat map, and SwapSeat helps coordinate the best arrangement.

Build that first. Then strangers can be added as a second layer once each journey has enough participating travelers.

you

with the recent discussion (not first discussion) we had give a zip of files to give to ai coders to implement i will give the zip to ai coding platforms

Gpt 6 Astra

I can prepare the package contents, but **I can't attach a downloadable ZIP from this chat**. The script below creates `SwapSeat-AI-Coder-Pack.zip` containing the implementation files.

This package follows **our recent discussion**: shared Train/Bus/Flight home, country-specific availability and pricing, ₹49/₹99 India pricing experiments, protected seat details, group invitations, feedback in Settings, and a global-ready architecture.

It also resolves important implementation gaps:

- Country selection does not let users manipulate checkout prices.
- Exact seat locations cannot leak through pre-payment map coordinates.
- Competing acceptances cannot reserve the same seat.
- No-match credits are issued once, at a defined deadline.
- “Both agreed” does not mean the operator changed the booking.
- Partial journey overlap is not enough for a full-trip swap.

Run this Python script in a local terminal or an AI coding platform with Python. It needs no additional packages. It creates the ZIP and an unpacked folder. No screenshots are included because actual image files are not available in this conversation; the package includes instructions for adding them.

PYTHON

```
from pathlib import Path
from zipfile import ZipFile, ZIP_DEFLATED
import json

ROOT = Path("SwapSeat-AI-Coder-Pack")
ZIP_PATH = Path("SwapSeat-AI-Coder-Pack.zip")

if ROOT.exists() or ZIP_PATH.exists():
    raise SystemExit(
        "Output already exists. Move or rename it before running again."
    )

files = {}
```

```
files["README.md"] = r"""
# SwapSeat Implementation Package
```

Scope

This package represents the latest product direction:

- One mobile-first PWA.
- Shared Train / Bus / Flight entry point.
- Country-aware availability, language, pricing, and providers.
- Indian train pilot first; bus and flight adapters follow.
- Visual seats and berths.
- Paid search activation and optional one-sided completion fee.
- Protected counterpart seat details.
- Journey-bound coordination.
- Group invitations and measurable referral loops.
- Feedback and improvement tools in Settings.

This package is a specification, not implemented software.
It contains no live integrations, actual provider credentials,
certified layouts, or production legal approvals.

Read in order

1. START_HERE.md
2. PRODUCT_DECISIONS.md
3. docs/01_UX_AND_DESIGN.md
4. docs/02_ARCHITECTURE.md
5. docs/03_DATA_AND_API.md
6. docs/04_MATCHING_AND_STATES.md
7. docs/05_PAYMENTS_AND_CREDITS.md
8. docs/06_COUNTRIES_AND_LOCALIZATION.md
9. docs/07_GROWTH_FLYWHEELS.md
10. docs/08_FEEDBACK_AND_SETTINGS.md
11. docs/09_SECURITY_AND_OPERATIONS.md
12. docs/10_TESTS_AND_RELEASE.md

Important distinctions

- Demo data is not a verified transport layout.
- Phone verification is not booking verification.
- Matching score is not probability of success.
- User agreement is not an official seat reassignment.
- Unknown passenger participation is not an empty seat.
- Regional prices are experiments, not validated market prices.
- Store credit is not a cash wallet.

Builder handoff

Upload this ZIP, the existing repository if any, and actual design images.
Ask the coding agent to follow START_HERE.md.

Do not upload production secrets, passenger records, or private tickets.
"""

```
files["START_HERE.md"] = r"""
# Instructions for the AI Coding Agent
```

Inspect the repository before editing. Preserve suitable existing
conventions.

Do not replace the stack merely to match a preferred framework.

First report:

- Current stack and routes.
- Database and authentication.
- Existing reusable components.
- Tests, deployment, and PWA support.
- Missing infrastructure.
- Specification conflicts.
- Proposed milestone plan.

Then implement in small, independently testable milestones.

Delivery order

M0: repository assessment and decision log.

M1: mobile UI, sample data, transport selector, country selector, train map, preferences, protected match preview, offline shell.

M2: authentication, database, deterministic matching, request workflow, authorization, audit trail, reporting, private admin console.

M3: journey groups, safe invitations, group map permissions, feedback forms, settings, notification preferences.

M4: sandbox payments, quote engine, credit ledger, webhook processing, no-match adjudication, refunds, concurrency tests.

M5: reviewed train layouts and licensed provider integration.

M6: bus and flight pilots after mode-specific readiness checks.

Admin security and reporting are not deferred until after launch.

Agent rules

- Do not fake integrations or verification.
- Label mock flows and demo data.
- Never trust client price, entitlement, seat ownership, or payment status.
- Use server-side authorization and schema validation.
- Do not leak protected seats through API payloads or SVG coordinates.
- Preserve unrelated user changes.
- Run available lint, typecheck, tests, and production build.
- Report commands actually run and any failures.
- Maintain docs/IMPLEMENTATION_STATUS.md.
- Record unresolved decisions in docs/DECISIONS.md.
- Do not claim the whole product is finished after implementing a demo.

First milestone

Build a functional mobile demo with:

country selection -> transport selection -> manual train details -> select own berth -> visual preferences -> protected match preview -> mock checkout -> request tracking -> simulated mutual agreement.

Include Settings with country/language and feedback entry points.

Use clearly illustrative fixtures. No real payments.

Disabled modes must explain whether they are coming soon or unsupported.

Completion report

List:

- Files changed.

- Functionality implemented.
- Tests run.
- Known limitations.
- Security-sensitive unfinished work.
- Next milestone.

"""

```
files["PRODUCT_DECISIONS.md"] = r"""  
# Product Decisions and Precedence
```

This file overrides conflicting earlier briefs or screenshots.

Positioning

SwapSeat coordinates possible passenger seat exchanges.
It does not sell seats or independently change tickets.

Platform

Mobile-first, not mobile-exclusive.
Optimize 360, 390, 393, and 430 CSS-pixel widths.
Keep desktop access usable through a centered mobile-oriented workspace.
Private admin tools may use desktop layouts.

Launch coverage

The home screen contains Train, Bus, Flight.
Only enabled services are actionable.
India train pilot is the initial production scope.
Other countries/modes are preview or waitlist until actually supported.

Eligibility

MVP requires the same canonical service instance, compatible class,
and identical boarding/destination stop indexes.
Partial overlap is not sufficient for a full-trip exchange.
Future partial swaps require explicit boundaries and ownership validation.

Fees

India experiment:
search activation: INR 49.
one-sided completion: INR 99 additional, if applicable.

The maximum potential total, INR 148, must be disclosed before first
payment.
The acceptor never pays merely to respond or accept.

If both participants have a qualifying activation entitlement for the
same service, waive the one-sided completion fee.
The entitlement condition is snapshotted into the proposal quote.

No-match outcome

Credit or refund is decided by the market policy and disclosed at checkout.
No-match adjudication occurs at a published deadline, not whenever a page
temporarily shows zero candidates.

Default pilot recommendation: free matching until useful density exists.

Paid activation is a feature flag, not a launch prerequisite.

Credits

Use non-transferable promotional/service credits, not a cash wallet.

No deposits, withdrawals, peer transfers, or cross-currency conversion.

Enable paid credits only after payment/legal review for the target market.

A credit is not described as a refund.

Privacy

Before required entitlement and mutual consent:

counterpart exact seat labels, coordinates, unique location identifiers,
and booking references are excluded from responses.

Own seat remains visible.

Group members can separately consent to sharing seats within their group.

Optional personal information

Gender and reason are optional.

No medical records or free-text health disclosure required.

Do not use inferred gender or profile photos for matching.

Do not default to public photos, full names, cancellation histories,
or public passenger manifests.

Scores

Use explainable preference fit, not an invented comfort score.

If a percentage is shown, disclose the factors and denominator.

Never present it as the chance a swap will happen.

Groups

Support multiple passenger seats in the data model and group UI.

Production MVP matches atomic one-seat-for-one-seat swaps.

Do not claim multi-seat bundle matching until bundle constraints and
transactional all-or-nothing acceptance are implemented.

"""

files["docs/01 UX AND DESIGN.md"] = r"""

UX and Design

Visual direction

Warm ivory surfaces, railway green primary actions, amber attention states,
blue group indicators, clear typography, restrained shadows.

Use supplied image references if present; never embed screenshot pages.

Suggested tokens:

background #F7F4ED

surface #FFFDF8

ink #17302B

primary #176B59

attention #E58A32

group #3F6FA8

danger #B94B45

Verify contrast rather than assuming these combinations pass.

Use readable interface typography and script-compatible locale fonts.

Primary journey

1. Choose travel country; offer a suggested value without forcing it.
2. Choose Train / Bus / Flight.
3. Enter service, boarding date, origin and destination.
4. Optional booking lookup, or manual entry.
5. Enter coach/cabin/deck and own seat.
6. Show own seat visually and confirm layout confidence.
7. Choose desired features and optional reason.
8. Show privacy-safe match information.
9. Review search price and possible additional fee.
10. Activate search.
11. Send request.
12. Counterpart reviews and accepts.
13. Complete additional payment if applicable.
14. Revalidate; mutually agree; reveal authorized details.
15. Coordinate, obtain operator/crew approval when required.
16. Both report the outcome.

Combine simple inputs into short steps. Avoid 16 separate mandatory forms. Allow map exploration before login. Require auth for private or paid actions.

Screens

| | |
|---------------------------|---------------------------------------|
| / | shared transport home |
| /onboarding/country | country selector |
| /journeys/new | adaptive service form |
| /journeys/:id/map | own map and coach/deck/cabin selector |
| /journeys/:id/preferences | visual desired features |
| /journeys/:id/matches | protected candidates |
| /checkout/:quoteId | itemized quote and outcomes |
| /requests/:id | request state timeline |
| /groups/:id | consented group map and progress |
| /invite/:token | safe invitation preview |
| /inbox | notifications and conversations |
| /settings | account and app preferences |
| /settings/feedback | feedback hub |
| /settings/credits | credits and payment history |
| /help | safety and support |

Maps

Data-driven SVG or accessible HTML, never screenshot-only maps.
Train: top-down bay plus berth elevation; no guessed numbering.
Bus: deck switch, driver, aisle, actual sleeper/seater geometry.
Flight: actual layout variant when available, cabin and restrictions.

Show source, version, confidence, freshness, and correction action.
Generic templates are illustrative even after a user confirms resemblance.

Privacy-preserving preview

Own seat: exact.
Unpaid counterpart: broad features and non-unique region only.
Do not highlight a single hidden berth on an exact numbered map.
Disable combinations of descriptors that identify a unique location.

Approximate maps must be explicitly labeled.

Accessibility and PWA

44 CSS-pixel minimum targets, preferably 48.

Text/icons as well as color.

Keyboard operation and accessible list alternative.

Visible focus, reduced motion, logical sheet focus handling.

Safe-area padding, 200% zoom support, RTL-capable layout.

Offline: cached public layout and optionally saved own journey summary.

No offline payment, acceptance, or availability confirmation.

Never silently queue acceptance.

Show last-updated time and connection status.

Install and notification prompts appear after demonstrated value.

Browser features use capability detection and fallbacks.

"""

files["docs/02 ARCHITECTURE.md"] = r"""

Architecture

Use a modular monolith initially, not microservices.

Suggested greenfield baseline:

- TypeScript web application.

- Next.js or suitable existing React stack.

- PostgreSQL.

- Managed authentication.

- Background worker.

- Private object storage if evidence is introduced.

- Transactional outbox.

- Payment and notification adapters.

- Redis only when justified for queueing or rate limits.

Pin supported dependency versions during implementation.

Do not assume packages or platform APIs from this brief are current.

Modules

identity

market-config

service-resolution

seat-layouts

journey-claims

groups

matching

requests

pricing

payments

credits

messaging

notifications

feedback

moderation

analytics

Dependency direction

UI -> server API -> domain services -> database/adapters.
UI never calls booking or payment providers using secret credentials.
Transport adapters supply data; shared domain services apply policies.

Provider contracts

TransportProvider:
- capabilities()
- searchServices()
- resolveService()
- lookupBooking() [optional authorized capability]
- resolveLayout()
- getTripStatus() [optional]
- validateRestrictions()

Responses include source, confidence, observedAt, expiresAt,
provider identifiers, operating service identifiers, and errors.

PaymentProvider:
- createIntent()
- verifyWebhook()
- fetchPayment()
- refund()

NotificationProvider:
- deliver()
- getDeliveryStatus()

Reliability

Use transactional outbox events for:
request_created, reservation_held, payment_received,
agreement_recorded, hold_expired, credit_issued.

Workers are at-least-once; consumers must be idempotent.
Use event IDs, retry backoff, dead-letter handling, and reconciliation.
Push/realtime are hints; clients refetch authoritative server state.

Deployment

Separate development, staging, and production.
Payment sandbox and mock providers in preview environments.
Versioned migrations and backups.
Structured logs with sensitive-field redaction.
Feature flags for market, mode, provider, and paid activation.
"""

```
files["docs/03 DATA AND API.md"] = r"""  
# Data Model and API
```

Produce actual reviewed migrations during implementation.

Core tables

users, profiles, sessions
markets, market_capabilities, pricing_versions
operators, stations, service_instances, service_stops
layout_templates, layout_versions, equipment_assignments
traveler_journeys, passengers, seat_claims, preferences

groups, group_members, group_seat_permissions
invite_tokens, invite_events
match_candidates, swap_requests, request_participants
request_seat_items, seat_reservations, request_events
quotes, payment_intents, payment_events, refunds
credit_accounts, credit_ledger_entries
conversations, messages
notifications, outbox_events
feedback_items, feedback_attachments, feedback_events
reports, blocks, admin_audit_events

Canonical service identity

operator + operating service identifier + origin departure instant.
Store service-origin date separately from passenger boarding date.
Overnight trains must not split into separate services merely because
passengers board on different calendar dates.

Flight codeshares resolve to the operating service.
Bus services include departure time and operator identity, not route alone.

Use IANA timezones on stops. Store instants in UTC.
A country does not imply a single timezone.
International journeys have multiple country codes.

Seat claims

Reference service, equipment, layout version, physical seat,
passenger, segment, verification level, and claim status.

Do not trust user-entered coach/seat as verified ownership.
Conflicting manual claims become disputed, not automatically overwritten.
A booking's multiple seats reference distinct passengers.
For dependents, record guardian permission without public child profiles.

Constraints

- boarding index < destination index
- immutable pricing and layout versions
- unique provider webhook event
- unique idempotency key per actor and operation
- unique no-match award per activation entitlement
- unique active seat reservation for an MVP service-seat segment
- balanced payment/credit reconciliation
- group membership checked server-side
- request from/to participant cannot be identical

API groups

GET /markets
GET /markets/:code/capabilities
GET /services/search
POST /services/resolve
POST /booking-lookups
POST /journeys
GET/PATCH /journeys/:id
GET /journeys/:id/map
PUT /journeys/:id/preferences
GET /journeys/:id/matches

POST /quotes
POST /payments/intents
POST /payments/webhooks/:provider
POST /requests
POST /requests/:id/accept
POST /requests/:id/cancel
POST /requests/:id/outcome
POST /groups
POST /groups/:id/invites
POST /invites/:token/join
POST /feedback
GET /feedback/mine
POST /reports
GET /settings
PATCH /settings

Mutations validate schemas and authorization.
Sensitive mutations require idempotency keys and expected entity version.
Payment webhook endpoints authenticate provider signatures, not user cookies.

Seat privacy DTOs

OwnSeatDTO: exact authorized information.
CandidateDTO: opaque candidate ID and coarse non-identifying features.
AgreedSwapDTO: authorized seats after entitlement and mutual consent.

Do not send a complete private seat map and hide labels with CSS.
Filter sensitive values before serialization, logs, events, and caching.
"""

```
files["docs/04_MATCHING_AND_STATES.md"] = r"""  
# Matching and State Machines
```

Eligibility first, ranking second

Require:

- same canonical operating service
- exact segment for MVP
- compatible ticket class and operator rules
- confirmed assigned seat; RAC/unassigned unsupported initially
- fresh equipment/layout version
- reciprocal acceptable preferences
- no blocked relationship
- no active seat reservation conflict

Restrictions are hard exclusions, not negative scoring.
Do not let payment bypass accessibility or exit-row restrictions.

Rank eligible candidates using explainable factors:
desired features, family proximity with consent, same coach,
walking distance when known, and claim verification confidence.

For an acceptor who is not searching, require explicit approval of
the proposed exchange; never infer their willingness.

Separate states

Journey: draft, active, paused, completed, cancelled, invalidated.

Activation: unpaid, active, exhausted, expired, credited, refunded.
Request: pending, held, payment due, mutually agreed,
awaiting operator action, completed, declined,
cancelled, expired, invalidated, disputed.
Payment: created, pending, succeeded, failed, refunded.
Hold: active, released, converted, expired.

Payment success and passenger consent are different facts.

Multiple requests

Allow bounded fan-out with spam limits.
A recipient acceptance attempts a transactional temporary hold
on all participating seats.
Lock resources in stable order to avoid deadlocks.
Only one conflicting request can win.

During hold:

- pause alternatives
- disclose payment deadline
- ask requester to complete required checkout

After successful revalidation and payment:

- convert hold to commitment
- cancel incompatible alternatives
- notify their participants
- reveal permitted details

If hold expires:

- release resources
- revalidate alternatives before resuming
- do not revive stale requests automatically

Late successful payment does not override an expired hold.
Recheck availability; otherwise refund/credit according to policy.

Completion

Record each participant's outcome independently.
Only both-party confirmation produces "both reported completed".
Missing response is not success.
Operator confirmation has a separate source and status.

Equipment changes invalidate affected proposals and trigger revalidation.

Chat

Open after mutual agreement and required payment.
Safety reporting remains available before and after chat closure.
Use scheduled end plus a configurable grace period unless trustworthy
service status is available.
Do not claim GPS-based arrival without consent and actual data.

"""

```
files["docs/05 PAYMENTS AND CREDITS.md"] = r"""  
# Payments, Pricing, Credits
```

Server quote

A quote contains:

- market and pricing version
- currency and minor-unit exponent
- search fee
- conditional completion fee
- maximum possible total
- applicable tax presentation
- credits used and cash due
- no-match policy
- expiry
- eligible journey and request identifiers

Amounts use integer minor units.

Currency exponent is metadata; not every currency has two decimals.

India experiment

Search: 4900 paise.

Conditional completion: 9900 paise additional.

Maximum fees before applicable tax treatment: 14800 paise.

Show the full conditional fee before first payment.

If final tax treatment is not configured, block paid production checkout.

No recipient fee just for accepting.

Search entitlement must belong to the same relevant service/activation, not any historical payment.

Payment authority

Only signed server-verified provider events grant entitlement.

Client checkout success only triggers a status refresh.

Handle duplicate, delayed, and out-of-order webhooks.

Reconcile provider state with internal ledger periodically.

No-match adjudication

Activation has:

matchingDeadline, eligibilityVersion, policyVersion, resolutionStatus.

Define "suitable match" explicitly in the checkout policy.

Recommended pilot: no mutually agreed valid swap by deadline qualifies, excluding clearly disclosed user cancellations/abuse.

Do not credit at the first empty result.

Credit issuance and activation closure occur in one transaction.

Retries must never duplicate awards.

Credits UI

Settings -> Payments and credits.

Show separate cash payments, refunds, promotional credits, service credits.

Do not combine currencies into one balance.

Ledger entries reference source payment/activation and reversal events.

Balances derive from ledger entries; do not edit balances directly.

Admin adjustments require reason, authorization, and audit.

Store credit is not cash, cannot be transferred, and is not a refund.

Policy and expiration require clear disclosure and market review.
If credits are not approved for a market, use an approved refund model.

Refund scenarios

Explicit policies for:

no match, recipient cancellation, operator refusal, platform failure,
equipment change, user cancellation, duplicate payment, late payment.

Do not launch paid flows until each scenario has an approved outcome.
No platform-controlled passenger-to-passenger payments.

"""

files["docs/06_COUNTRIES AND LOCALIZATION.md"] = r"""
Country and Localization Architecture

Separate these concepts:

- UI locale
- preferred travel country
- actual journey country/countries
- display currency
- transaction currency
- checkout/billing market
- operator policy jurisdiction

Changing the country selector must not change an existing paid entitlement
or allow arbitrary cheaper checkout.

First launch

Suggest a country from browser locale or coarse server context.

Always allow manual correction.

Do not require GPS.

Explain that country selection controls supported travel options.

Coverage matrix

Resolve capability by:

market + mode + operator + service + provider + layout support.

Statuses:

available, pilot, manual_supported, waitlist, unsupported.

Manual entry does not override an unsupported or prohibited service.

Country-wide flags alone cannot imply universal support.

Country selection UI

Show available modes and clearly labeled coming-soon modes.

Allow users to request country/operator support.

Keep journeys visible when their current selected country changes.

Pricing localization

Do not merely exchange-rate convert India fees.

Evaluate purchasing power as one input alongside local willingness to pay,
payment cost, support cost, taxes, match density, and conversion.

All non-India prices in sample data are disabled test hypotheses.

Do not claim that these prices are validated purchasing-power equivalents.
Run limited, transparent experiments and review country-level outcomes.

Resolve checkout market server-side using an approved policy based on
journey market and permitted billing criteria.

Never use inferred ethnicity, gender, health, or desperation for pricing.

Internationalization

Externalize strings and pluralization.

Support RTL, names in local scripts, flexible address/phone formats.

Use locale-aware number/date/currency formatting.

Store service timezones per stop.

International journeys show local times and date changes explicitly.

Global auth must not depend exclusively on Indian phone numbers.

Enable available email/phone identity methods by market.

"""

files["docs/07_GROWTH_FLYWHEELS.md"] = r"""

Growth Flywheels

Virality is a hypothesis to measure, not an outcome to promise.

Main loop: useful before matching

Create journey -> invite travel companions -> add seats ->

see shared map -> discover seating problem -> coordinate ->

invite on next journey.

Provide free basic group-map value even without strangers joining.

Do not paywall safety, request responses, or the basic invite process.

Invitations

Use random high-entropy revocable tokens.

Store token hashes, expiry, group scope, maximum uses, and creator.

Public previews expose minimal metadata chosen by the inviter.

Joining requires authentication and permission checks.

Private groups may require organizer approval.

Never include booking references or seat details in:

URLs, social preview tags, analytics, public share images.

Use native share capability where available and copy-link fallback.

Do not claim Instagram or other apps accept every share format.

Do not auto-send messages or import address books.

Growth experiments

A. Prefilled family invitation immediately after own seat confirmation.

B. Group completion reminder: "3 of 5 travelers added seats."

C. Free organizer trial after genuine group activation.

D. No-match invitation with honest wording and optional flexible preferences.

E. Post-trip generic success card without location history by default.

F. Organizer partnerships for real departures.

G. Accurate public layout explainers with no passenger data.

Avoid minimum-member gates for essential group functionality.
A couple should obtain value without recruiting six people.

Rewards

Reward a qualified activation:
authenticated distinct traveler + authorized group membership +
valid journey setup + fraud checks + a meaningful action.

No rewards for raw clicks or contact uploads.
Phone verification alone does not prove a journey.
Cap rewards per group, departure, account, and period.
Rewards are not cash and are not transferable.
Provide appeals for false-positive fraud decisions.

Metrics

Events:
invite_created, share_sheet_opened, invite_opened, invite_joined,
seat_added, group_activated, match_eligible, request_agreed,
outcome_both_confirmed, referred_user_new_journey.

Share-sheet opened does not mean invitation delivered.

Measure by country and departure cohort:
- invite-to-activation conversion
- active participants per departure
- match coverage before/after invitations
- coordination completion
- invited users creating later journeys
- complaints and invite abuse
- cost per incremental activated departure

Estimate reproduction:
distinct eligible invitees per activated inviter
x invite-to-activation conversion.

Deduplicate recipients and compare against a holdout before claiming lift.

Guardrails

No public passenger directories.
No spam countdowns, fake match counts, or fake social proof.
No incentives for sharing medical reasons or live personal locations.
Suppress growth prompts during reports, disputes, refunds, and emergencies.
"""

```
files["docs/08_FEEDBACK_AND_SETTINGS.md"] = r"""  
# Settings and Feedback
```

Settings

Account and sessions
Travel country and language
Currency display
Notifications
Privacy and group visibility
Payments and credits
My journeys

Help and safety
Give feedback
Request a feature
Report a problem
Delete account

Feedback hub

- Suggest improvement
- Report bug
- Report incorrect seat map
- Request country/operator support
- Share journey experience
- Safety issue: route directly to safety support

Fields:

category, short title, description, optional journey,
optional contact permission, optional attachment.

Attach only disclosed diagnostics:

app version, locale, device category, route name, error reference.
Do not attach tickets, chat, PNR, or payment data automatically.

Attachments require file limits, malware checks, EXIF removal where
appropriate, private storage, signed access, and retention controls.
Offer a screenshot-redaction reminder.

Feedback lifecycle

new -> acknowledged -> triaged -> planned -> in progress ->
released / declined / duplicate.

Allow user to view their submissions and staff responses.

Notify about resolution only with appropriate communication preferences.

Admin filters:

market, mode, operator, layout version, app version, category, severity.
Private internal notes remain private.

Public feature board is optional and separately moderated.

Never automatically publish private feedback.

Use explicit consent and sanitized public summaries.

One vote per authenticated user; abuse limits.

In-context feedback

Seat map: report exact layout version and selected geometry element.

Checkout: report pricing confusion without payment credentials.

Journey end: one-tap outcome followed by optional comment.

Improvement loop

collect -> triage -> prioritize -> fix -> validate ->
notify affected users -> measure reduced failures.

Prioritize severity, affected journeys, map accuracy, and safety,
not votes alone.

"""

```
files["docs/09_SECURITY_AND_OPERATIONS.md"] = r"""  
# Security and Operations
```

Required controls

```
Server-side object authorization on every private resource.  
OTP/email rate limits, secure sessions, CSRF protection where applicable.  
Provider secrets never in client bundles.  
Private booking evidence with short retention.  
Encrypted sensitive fields when operationally needed.  
HMAC rather than plain hashes for searchable phone identifiers.  
Scoped admin roles, MFA, audited access.  
Report/block controls and dispute handling.
```

Payment privacy boundary

```
Do not leak counterpart identity through:  
seat IDs, geometry, exact bay highlights, deterministic public IDs,  
HTML source, client stores, realtime payloads, logs, analytics,  
service-worker caches, or social cards.
```

```
Payment gating is not a guarantee people cannot recognize each other.  
Do not degrade safety or misrepresent anonymity to protect fees.
```

PWA caching

```
Allowlist cacheable public assets/layout versions.  
Never cache payment APIs, booking lookup, protected candidate responses,  
or private chat in a shared service-worker cache.  
Opt-in minimal own-journey storage.  
Clear user-scoped local data on logout/account switch.  
Display stale data timestamps.
```

Operational readiness

```
Dashboards:  
provider failures, match-worker delay, expired holds,  
payment reconciliation, duplicate credits, authorization errors,  
layout corrections, reports, feedback backlog.
```

```
Have playbooks for:  
provider outage, wrong layout, payment webhook outage,  
compromised invite, harassment, data incident, refund failure.
```

External launch gates

```
Before paid launch, obtain appropriate professional review of:  
operator rules and data licenses, privacy notices and retention,  
consumer disclosures/refunds, taxes, credits, messaging consent,  
age/guardian handling, and country-specific obligations.
```

```
The specification does not certify regulatory compliance.
```

```
Use published support channels; never imply emergency response capability.  
"""
```

```
files["docs/10_TESTS_AND_RELEASE.md"] = r"""  
# Tests and Release Gates
```

Unit

Canonical service identity, overnight boarding dates.
Exact-segment eligibility.
Reciprocal preferences and restrictions.
Pricing quote versioning and currency exponents.
Request transition guards.
Credit calculation and no-match adjudication.
Localization and timezone formatting.
Coarse-preview privacy rules.

Integration

Cross-user private access denied.
Invite revoked/expired cannot join.
Duplicate webhook grants one entitlement.
Duplicate adjudication grants one credit.
Concurrent acceptances yield one seat hold.
Payment arriving after hold expiry does not steal a seat.
Equipment changes invalidate requests.
Blocks stop matching and messaging.
Multi-seat bundles fail atomically when enabled.
Account switching clears local private data.

End to end

Country -> available transport -> manual journey -> own seat ->
preferences -> quote -> sandbox payment -> request ->
acceptance -> optional additional fee -> mutual agreement ->
safe reveal -> chat -> independent outcomes.

Also test:

offline, no candidates, declined, cancelled, expired,
wrong layout, unsupported provider, payment failed, refund pending,
feedback submission, invite activation.

Release gates

M1 demo:

all sample claims clearly labeled; no actual payment.

Private beta:

authorization tests, moderation, audit, feedback, reliable layouts.

Paid pilot:

verified webhooks, concurrent hold tests, financial reconciliation,
approved refund/credit policy, honest maximum-price disclosure.

Country expansion:

reviewed providers/layouts, local pricing/tax setup,
translated critical flows, support readiness, operator-policy review.

Bus/flight expansion:

mode-specific restrictions and equipment changes tested.

Performance targets

Measure on representative lower-end phones and throttled networks.
Target responsive map interaction and small route bundles.

Set Core Web Vitals budgets during implementation and record measurements.
Do not claim performance targets were achieved without testing.

"""

```
files["DESIGN REFERENCES/README.md"] = r"""
```

```
# Add Actual Design References Here
```

No rendered screen files are included in this package.

Add actual PNG/JPG/PDF exports for:

- shared home and country selector
- journey form
- own seat map
- preferences
- protected match preview
- checkout with conditional fee disclosure
- request timeline
- group map
- settings and feedback

Reference images guide appearance, not business rules.

PRODUCT DECISIONS.md overrides contradictory screenshot text.

Never copy incorrect train berth numbering from an illustration.

Layouts need source and version review.

"""

```
files["SAMPLE DATA/README.md"] = r"""
```

```
# Sample Data
```

Configuration here is illustrative and disabled for live paid use.

Do not claim non-India countries are launched.

Do not claim any example prices are purchasing-power validated.

Build synthetic demo services rather than inventing live schedules.

Seat map fixtures must be labeled illustrative until reviewed.

Do not substitute arbitrary sequential berth numbering for actual geometry.

Production requires separate approved layout and provider records.

"""

```
markets = [
```

```
{
```

```
    "countryCode": "IN",
```

```
    "name": "India",
```

```
    "locale": "en-IN",
```

```
    "supportedLocales": ["en-IN", "hi-IN"],
```

```
    "currency": "INR",
```

```
    "status": "pilot",
```

```
    "modes": {
```

```
        "train": "pilot",
```

```
        "bus": "waitlist",
```

```
        "flight": "waitlist"
```

```
    },
```

```
    "paidActivationEnabled": False,
```

```
    "creditsEnabled": False
```

```
}]
```

```
]
```

```

for code, name, locale, currency in [
    ("US", "United States", "en-US", "USD"),
    ("GB", "United Kingdom", "en-GB", "GBP"),
    ("AE", "United Arab Emirates", "en-AE", "AED"),
    ("SG", "Singapore", "en-SG", "SGD"),
    ("AU", "Australia", "en-AU", "AUD"),
    ("CA", "Canada", "en-CA", "CAD"),
]:
    markets.append({
        "countryCode": code,
        "name": name,
        "locale": locale,
        "currency": currency,
        "status": "waitlist",
        "modes": {
            "train": "waitlist",
            "bus": "waitlist",
            "flight": "waitlist"
        },
        "paidActivationEnabled": False,
        "creditsEnabled": False
    })

files["SAMPLE_DATA/markets.json"] = json.dumps(markets, indent=2)

files["SAMPLE_DATA/pricing-experiments.json"] = json.dumps({
    "warning": "Disabled hypotheses. Not purchasing-power validated.",
    "plans": [
        {
            "id": "IN-pilot-v1",
            "market": "IN",
            "currency": "INR",
            "minorUnitExponent": 2,
            "searchFeeMinor": 4900,
            "conditionalCompletionFeeMinor": 9900,
            "maximumFeeMinorBeforeTaxReview": 14800,
            "taxStatus": "requires_review",
            "noMatchPolicy": "requires_approved_policy",
            "enabled": False
        },
        {
            "id": "US-research-v1",
            "market": "US",
            "currency": "USD",
            "candidateSearchFeesMinor": [299, 399, 499],
            "conditionalCompletionFeeMinor": None,
            "enabled": False
        },
        {
            "id": "GB-research-v1",
            "market": "GB",
            "currency": "GBP",
            "candidateSearchFeesMinor": [249, 299, 399],
            "conditionalCompletionFeeMinor": None,
            "enabled": False
        },
        {
            "id": "AE-research-v1",
            "market": "AE",
            "currency": "AED",
            "candidateSearchFeesMinor": [900, 1200, 1500],

```

```
        "conditionalCompletionFeeMinor": None,  
        "enabled": False  
    }  
]  
}, indent=2)
```

```
files["PROMPT TO PASTE.txt"] = r"""  
Read the attached SwapSeat implementation package.
```

```
Treat PRODUCT DECISIONS.md as authoritative if older screenshots or  
conversation text conflict.
```

```
Inspect the repository and follow START_HERE.md.  
First report the architecture assessment and milestone plan.  
Then implement M1 only, unless I authorize more.
```

```
Build real mobile interactions, not screenshot backgrounds.  
Include country selection, transport availability, a clearly illustrative  
train journey/map, visual preferences, protected candidate preview,  
mock checkout, request tracking, Settings, and feedback entry points.
```

```
Use no real payments and make no claims of live booking verification.  
Maintain a decision log and implementation status.  
Run available tests and report actual results.  
"""
```

```
files["docs/IMPLEMENTATION_STATUS.md"] = r"""  
# Implementation Status
```

```
Initial status: specification only.
```

```
Milestone	Status	Evidence
M0 Assessment	Not started	
M1 Mobile demo	Not started	
M2 Secure backend	Not started	
M3 Groups and feedback	Not started	
M4 Sandbox payments	Not started	
M5 Train pilot	Not started	
M6 Bus and flight	Not started	
```

```
The coding agent must update this file with actual work and test results.  
"""
```

```
files["docs/DECISIONS.md"] = r"""  
# Decision Log
```

```
Record date, decision, alternatives, rationale, owner, and consequences.
```

```
## Open decisions before production
```

```
- Approved train data/layout sources.  
- Operator-specific coordination policies.  
- Identity and booking verification method.  
- Exact no-match deadline and qualification policy.  
- Refund versus credits by market.  
- Tax-inclusive presentation by market.  
- Conditional completion fee experiment.  
- Retention schedule.
```

```
- Support operating hours and escalation owners.
- Initial routes/operators and launch cohort.
- Approved production seat-map fixtures.
"""
```

```
for relative_path, content in files.items():
    path = ROOT / relative_path
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(content.strip() + "\n", encoding="utf-8")

with ZipFile(ZIP_PATH, "w", ZIP_DEFLATED) as archive:
    for path in sorted(ROOT.rglob("*")):
        if path.is_file():
            archive.write(path, path.relative_to(ROOT.parent))

print(f"Created: {ZIP_PATH.resolve()}")
print(f"Files included: {len(files)}")
print("Upload the ZIP and actual design reference images to your coding
platform.")
print("Paste PROMPT_TO_PASTE.txt as your first instruction.")
```

After generating it: upload `SwapSeat-AI-Coder-Pack.zip` to your coding platform, add your actual mobile design images, and paste the contents of `PROMPT_TO_PASTE.txt`.

The pack deliberately keeps production payments and unsupported countries disabled until their pricing, provider coverage, policies, and payment handling are ready. This lets the coding agent build the global architecture without falsely presenting global availability.

you

1)i dont want demo i want real working app 2)If both participants have a qualifying activation entitlement for the same service, waive the one-sided completion fee. search activation: INR 49. one-sided completion: INR 99 additional, if applicable. Splitted 49 for requestor /acceptor side platform fee 49 credits for the acceptor as he was not looking for exchange but exchanged he can use those credits next time journey if he wants to swap the 50% credis will be credited to his wallet. give another zip now with changes

Gpt 6 Astra

Understood: **the deliverable must be a real working application, not a demo**, with real accounts, persistent journeys, actual matching between users, payments, wallet credits, notifications, and administration.

I cannot attach a ZIP directly from this chat. Below is a replacement package generator that creates `SwapSeat-Production-v2.zip`, ready to upload to your coding platform. It works independently of the previous script.

Your revised fee model

Your message contains one numerical ambiguity: **50% of ₹99 is ₹49.50, not ₹49.** I have configured the package to follow your stated **₹49 accepter credit**:

| Situation | Requester pays | Acceptor pays | Acceptor receives |
|---|-------------------|-------------------|--|
| Activate a search | ₹49 | — | — |
| Both participants already have qualifying search activation | No additional fee | No additional fee | No completion reward |
| Acceptor was not searching | ₹99 additional | ₹0 | ₹49 in future-use credits after the completed exchange |

For the one-sided case:

- Requester's total: **₹148**.
- Additional completion charge: **₹99**.
- Platform allocation from that charge: **₹50**, before applicable costs and tax treatment.
- Acceptor's future-use credit: **₹49**.
- Credits are usable on a future SwapSeat search, not withdrawable cash.

This is explicitly marked as an assumption requiring your confirmation before live payments. The configuration also supports an exact 50% reward if you instead mean ₹49.50.

The wallet is specified as credits usable only for SwapSeat's own services—not a transferable cash wallet. That distinction matters in the RBI framework; the package requires review before launch rather than claiming compliance automatically. (rbi.org.in)

Generate the new ZIP

Run this Python script. No additional packages are needed.

```
PYTHON
from pathlib import Path
from zipfile import ZipFile, ZIP_DEFLATED
import json

ROOT = Path("SwapSeat-Production-v2")
ZIP_PATH = Path("SwapSeat-Production-v2.zip")

if ROOT.exists() or ZIP_PATH.exists():
    raise SystemExit(
        "Output already exists. Rename or move it before running again."
    )

files = {}
```

```
files["README.md"] = r"""  
# SwapSeat Production Implementation Package v2
```

Deliverable

Build a real, deployable mobile-first PWA.

Not a demo.

Not a static prototype.

Not simulated matching presented as real.

Not fake booking verification.

Required production capabilities:

- Real authentication.
- Persistent user journeys and seat claims.
- Real matching between actual opted-in users.
- Data-driven visual seat maps.
- Payment gateway integration.
- Credits ledger.
- Request acceptance and concurrency protection.
- Group invitations.
- Notifications and journey-bound chat.
- Feedback in Settings.
- Moderation and administration.
- Country-aware coverage, language, pricing, and policies.

Start with Indian trains, while implementing shared transport interfaces.
Bus and flight modes become actionable only when operationally ready.

Manual entry is a valid real workflow.

Invented schedules, fake participants, and fabricated exact layouts are not.

Read order

1. START_HERE.md
2. PRODUCT_AND_PRICING.md
3. ARCHITECTURE.md
4. DATA_AND_API.md
5. REQUEST_AND_PAYMENT_WORKFLOWS.md
6. WALLET_AND_REWARDS.md
7. MOBILE_UX.md
8. COUNTRY_AND_TRANSPORT.md
9. GROWTH_AND_FEEDBACK.md
10. SECURITY_AND_OPERATIONS.md
11. TESTS_AND_ACCEPTANCE.md
12. OWNER_INPUTS.md

Specification precedence

This v2 package supersedes previous demo-first and payment instructions.

Important unresolved owner decision:

INR 49 reward versus exactly 50% of INR 99.

Configuration currently proposes INR 49.

Do not activate production charges until this is confirmed.

Actual design files

Add actual screenshots to DESIGN REFERENCES/.

No image files are included in this generated package.

Financial scope

Rewards are service credits, not payment for ownership of a seat.

Do not implement passenger-to-passenger cash transfers.

Do not claim regulatory, tax, or operator approval from this specification.

"""

files["START_HERE.md"] = r"""

AI Coding Agent Instructions

Build the complete functioning application.

Do not stop after a visual prototype.

Inspect the existing repository and preserve suitable architecture.

Identify blockers, then continue implementing all work not blocked by them.

Initial report

- Repository architecture.

- Existing functionality.

- Database/authentication/deployment status.

- Reusable components.

- Missing production dependencies.

- Implementation plan.

- Owner decisions and credentials required.

Working rules

- Implement actual backend-backed interactions.

- Tests may use mocks; production must not silently use them.

- Missing provider configuration produces an honest unavailable state.

- Do not invent passenger records, matches, verification, or payments.

- Never stop at a list of recommendations if implementation is possible.

- Do not bypass security to make a feature appear functional.

- Keep sensitive decisions server-side.

- Preserve unrelated user changes.

- Record commands actually run and validation results.

- Maintain IMPLEMENTATION STATUS.md.

Delivery stages

1. Foundation:

database, migrations, authentication, authorization,

country configuration, app shell, private administration.

2. Real train workflow:

service identity, manual entry, seat claims, reviewed layouts,

preferences, actual matching, request lifecycle.

3. Payments and credits:

quotes, gateway checkout, signed webhooks, ledger,

accepter rewards, redemption, refunds, reconciliation.

4. Coordination and growth:

group invitations, consented group maps,

chat, notifications, referral attribution.

5. Operations:

feedback, moderation, account deletion, monitoring,
backups, deployment, incident procedures.

6. Expansion:

bus and flight adapters with operator-specific restrictions.

Payment sandbox testing is a mandatory validation environment,
not the final product.

Definition of completion

A real deployment must support two independently authenticated users
completing a full eligible request flow with persistent server state.

Production payments require actual configured provider credentials,
approved pricing, tax/refund configuration, and owner launch approval.

Do not mark live payments complete based only on a sandbox screenshot.
Do not mark an integration complete based only on an interface definition.

"""

files["PRODUCT AND PRICING.md"] = r"""

Product and Revised Pricing

Positioning

SwapSeat coordinates possible seat exchanges.
It does not independently change transport reservations.

Passenger agreement, payment, operator approval, and physical completion
are distinct events.

India proposed pricing

Search activation: INR 49.

If both participants have qualifying active search entitlements
for the relevant service:

- No additional completion charge.
- No one-sided acceptor reward.

If the acceptor was not searching and has no qualifying entitlement:

- Requester pays INR 99 additional.
- Acceptor pays nothing.
- Acceptor receives INR 49 future-use credits after completion.
- INR 50 of the completion charge is the proposed platform allocation.

Requester total for this one-sided flow: INR 148.

Platform allocation is not net profit.
Gateway costs, taxes, support, refunds, and credit obligations are
separate.

Ambiguity requiring owner confirmation

The owner mentioned both:

- INR 49 reward.

- 50% credits.

Exactly 50% of INR 99 is INR 49.50.

Proposed configuration:

rewardMode = fixed_minor

rewardMinor = 4900

Alternative:

rewardMode = percentage_bps

rewardBps = 5000

rewardMinor = 4950 for a 9900 charge

Do not silently choose a different amount.

Do not advertise "50%" when granting INR 49 from INR 99.

Block live checkout until the owner confirms the pricing version.

Entitlements

Qualifying activation:

- Same relevant canonical service.

- Active and not refunded, reversed, or exhausted.

- Associated with the appropriate traveler/claim.

- Funded by a valid payment or valid redeemed service credits.

Historic payment on another journey does not qualify.

Snapshot eligibility when issuing the accepted-proposal quote.

Do not alter a valid quote merely because a user later changes preferences.

Upfront disclosure

Before collecting INR 49, disclose:

- Search fee.

- Conditional INR 99 additional charge.

- Maximum potential INR 148.

- Exact tax presentation.

- No guaranteed match.

- No-match resolution policy.

- Acceptor reward conditions.

- Operator approval limitations.

No surprise second charge.

No-match activation outcome

Retain the earlier no-match credit policy, subject to approved market rules.

Issue the search credit once at the defined matching deadline.

Never issue it merely because a temporary search result is empty.

No-match credits and acceptor rewards are separate ledger entry types.

Initial scope

Multi-passenger groups are supported.

Atomic one-seat-for-one-seat matching launches first.

Multi-seat bundle matching must not be falsely represented as supported.

""

```
files["ARCHITECTURE.md"] = r"""  
# Architecture
```

Use a modular monolith initially.

Suggested greenfield stack:

- TypeScript.
- Next.js or equivalent suitable existing web stack.
- PostgreSQL.
- Managed authentication or securely implemented identity service.
- Background worker.
- Transactional outbox.
- Private object storage.
- Optional Redis for queues and rate limiting.
- Payment, transport, and notification adapters.

Use current supported versions verified during implementation.

Modules

```
identity  
market-configuration  
service-resolution  
layout-management  
journey-claims  
groups  
matching  
requests  
pricing  
payments  
credits  
messaging  
notifications  
feedback  
moderation  
administration  
analytics
```

Boundaries

Client -> authenticated server API -> domain services ->
database/providers.

The browser never determines:

- Payment success.
- Required charge.
- Reward eligibility.
- Wallet balance.
- Seat ownership.
- Exact counterpart disclosure.
- Final request acceptance.

Transport adapter capabilities

```
searchServices  
resolveCanonicalService  
lookupBooking, only when authorized and supported  
resolveLayout
```

getTripStatus, when supported
validateRestrictions

Each response:

source, confidence, observedAt, version, expiry, capability status.

Payments

createIntent
verifyWebhook
fetchPayment
refund
reconcile

Use an India-capable gateway selected and configured by the owner.
Keep integration abstract enough to support additional markets.

Events

payment captured
activation granted
request held
agreement recorded
completion reported
reward pending
reward available
credit redeemed
refund processed

Emit via transactional outbox.
Workers tolerate duplicates and retry safely.
"""

files["DATA AND API.md"] = r"""
Data and API

Tables

users, profiles, sessions
markets, market_capabilities, pricing_versions
operators, stations, service_instances, service_stops
layout_templates, layout_versions, equipment_assignments
traveler_journeys, passengers, seat_claims, preferences
groups, group_members, group_seat_permissions, invite_tokens
match_candidates, swap_requests, request_participants
seat_reservations, request_events
activation_entitlements
quotes, payment_intents, payment_events, refunds
credit_accounts, credit_entries, credit_reservations
reward_awards, reward_events
conversations, messages, notifications
feedback_items, feedback_attachments
reports, blocks, admin_audit_events, outbox_events

Canonical service identity

Use actual operating service, origin departure instance, and operator.
Passenger boarding date is not necessarily service origin date.
Include bus departure time and operator.

Resolve flight codeshares to operating service where possible.

Use UTC instants plus IANA timezones per stop.

Claims

Seat claim references:

passenger, service, equipment, layout version, seat, boarding/destination, verification, status.

Conflicting manually entered claims become disputed.

Do not overwrite another claim or mark manual entry operator-verified.

Ledger entries

Fields:

id, accountId, currency, signedAmountMinor, type, sourceType, sourceId, pricingVersion, idempotencyKey, effectiveAt, createdAt.

Do not directly edit balances.

Reversals are new entries linked to original entries.

Separate pending reward awards from spendable credit entries.

Constraints

Unique webhook provider/event.

Unique reward per qualifying request/accepter.

Unique no-match credit per activation.

Unique command idempotency key per actor/action.

Unique active seat reservation for the supported segment model.

boardingIndex < destinationIndex.

Immutable paid quote and pricing version.

No self-swap.

No negative available balance from redemption.

API groups

/markets

/services/search

/services/resolve

/booking-lookups

/journeys

/journeys/:id/map

/journeys/:id/preferences

/journeys/:id/matches

/quotes

/payments/intents

/payments/webhooks/:provider

/requests

/requests/:id/accept

/requests/:id/cancel

/requests/:id/outcomes

/wallet

/wallet/entries

/wallet/rewards

/groups

/groups/:id/invites

/invites/:token/join

/feedback
/reports
/settings

All private resources need object-level authorization.

Privacy DTOs

Own seat: exact authorized data.

Candidate: opaque ID and coarse non-identifying features.

Agreed exchange: exact seats only after mutual agreement and payment rules.

Do not leak exact counterpart seats through coordinates, unique bay descriptors, API IDs, analytics, or client-side hidden fields.

"""

files["REQUEST_AND_PAYMENT_WORKFLOWS.md"] = r"""

Workflows

Matching eligibility

Same canonical service.

Identical boarding/destination segments for MVP.

Compatible class and operator restrictions.

Assigned eligible seats.

Reciprocal preferences or explicit non-searching acceptor consent.

No blocked relationship.

No conflicting commitment.

Valid current equipment/layout.

Partial overlap alone does not permit a full-journey swap.

Separate state machines

Activation:

unpaid -> active -> resolved / expired / credited / refunded

Request:

pending -> held -> payment_due -> mutually_agreed ->

awaiting_operator_action -> completed

Alternative outcomes:

declined, cancelled, expired, invalidated, disputed

Payment:

created -> pending -> captured -> refunded / disputed

Reward:

ineligible / pending -> review_hold -> available / cancelled / reversed

Acceptance and concurrency

Recipient accepts.

Server locks participating resources in deterministic order.

Acquire a temporary exclusive hold.

Pause conflicting alternative requests.

Create immutable completion quote if required.

Successful payment does not override an expired hold.

Revalidate seats and quote before final commitment.
Late payment without available seats triggers approved refund handling.

After commitment:
cancel incompatible alternatives and notify affected users.

Completion fee

When accepter lacks qualifying activation:
requester pays INR 99 additional.
Create pending accepter reward only after captured payment and
valid mutual agreement.

Pending reward is not spendable.

Both users must separately report actual completion.
Require scheduled journey start to have occurred, or a reviewed verified
early completion scenario, before automatic reward eligibility.

Check disputes, payment reversals, and fraud signals.
Then issue one spendable reward credit.

Missing response does not automatically imply completion.
Provide admin resolution with evidence and audit.

Operator constraints

If required approval is refused:
do not mark the exchange completed.
Do not release the accepter reward.
Apply the approved cancellation/refund policy.

Chat

Enable at mutually agreed state after required payment.
Close using trusted journey end or scheduled end plus grace period.
Do not pretend to know live arrival without a real data source.
Safety reporting remains available after closure.
""

files["WALLET AND REWARDS.md"] = r""
Wallet and Acceptor Rewards

UI name: SwapSeat Wallet.
Explanatory label: Credits for future SwapSeat services.

Credits:
- Not withdrawable.
- Not transferable.
- Not redeemable for transport tickets.
- Not payment to another passenger.
- Currency-specific.
- Subject to disclosed market terms.

Production enablement requires appropriate market review.

Balances

Available service credits.

Pending completion rewards.
Credits reserved in a checkout.
Payment/refund history.

Never present pending rewards as spendable.

Example

Requester:
search payment INR 49
additional completion payment INR 99
total INR 148 under configured inclusive pricing

Acceptor:
pays INR 0
pending reward INR 49
after completed exchange and checks:
available credits INR 49

On a future qualifying journey:
search price INR 49
credits applied INR 49
cash due INR 0
create valid search activation without a fake gateway payment.

Redemption

Allow up to 100% of eligible future search activation fees
under the proposed policy.

Do not automatically allow reward credits to fund completion fees.
This limits reward-on-reward loops.

Reserve credits atomically during checkout.
Commit debit when entitlement is created.
Release reservations on expiry/failure.
Prevent concurrent overspending.

Reward availability is restricted to a later eligible journey
than the source exchange. Evaluate on the server.

Refunds

Never refund service credits as cash.
Cash refund cannot exceed actual captured cash for that payment.
Restore redeemed credits separately when policy requires.

If a source transaction is reversed after reward issuance:
freeze unspent related rewards.
If already spent, flag for review and restrict further rewards.
Do not auto-charge a passenger's card.

Abuse controls

No self-referrals or self-swaps.
Flag repeated pair farming and implausible journey frequency.
Reward once per qualifying exchange.
Use booking and journey confidence, not OTP alone.

Shared devices/family relationships are review signals, not automatic guilt.

Provide appeals and audited admin decisions.

Economics dashboard

Track separately:

cash captured, gateway costs, tax treatment,
refunds, credits granted, credits redeemed,
pending credits, available credit obligations,
fraud loss and support cost.

Do not count a credit award as cash paid to the acceptor.

"""

files["MOBILE UX.md"] = r"""

Mobile UX

Mobile-first, usable at 360/390/393/430 CSS-pixel widths.

Desktop gets a usable responsive view, not a broken mobile-only page.

Navigation

Home

Journeys

Requests

Inbox

Settings

Main screens

Country selector.

Train / Bus / Flight home.

Service and segment form.

Booking lookup or manual details.

Own seat map and layout confirmation.

Preferences and optional reason.

Protected candidates.

Search checkout with maximum fee disclosure.

Request tracking.

Additional checkout when needed.

Mutual agreement and operator-action instructions.

Completion confirmation.

Wallet with pending/available rewards.

Group map.

Settings and feedback.

Required acceptor copy

"You are not being charged to accept."

If reward eligible:

"Complete this exchange and receive INR 49 in SwapSeat credits
for a future eligible search."

Do not say "Earn cash for your seat."

Do not call INR 49 exactly 50% of INR 99.

Visual maps

Train: proper bay/berth geometry and elevation.
Bus: decks, driver, aisle, sleeper/seater.
Flight: operating aircraft/configuration and cabin restrictions.

Own seat may be exact.
Protected counterparts use broad non-unique regions.
Never hide the number while exposing the exact location.

All layouts show source, confidence, version and correction action.
User confirmation does not make an illustrative layout authoritative.

Accessibility

Large touch targets.
Screen-reader labels.
Keyboard navigation.
List alternative.
Reduced motion.
Focus management.
Color plus icon/text status.
RTL-ready components.

PWA

Installable shell.
Offline saved own journey and public layout where allowed.
No offline acceptance, payment, reward claim or live availability promise.
Clear stale timestamp.
Private local data cleared on logout/account switch.
"""

```
files["COUNTRY AND TRANSPORT.md"] = r"""  
# Country and Transport
```

Separate:
UI locale, travel country, journey countries, billing market,
display currency, transaction currency, operator jurisdiction.

Changing country must not manipulate checkout price.

Coverage

Capability matrix:
country + mode + operator + service + provider + layout.

States:
available, pilot, manual_supported, waitlist, unsupported.

Only actual supported capabilities are enabled.
Manual entry is not permission to bypass an operator prohibition.

Global pricing

Country plans are server-side, versioned, and explicit.
Do not simply FX-convert INR 49.
Consider local willingness to pay, purchasing power, fees,
taxes, support cost, and match density.

Configure separately:

search fee

completion fee

accepter reward

reward type

credit redemption policy

no-match policy

Do not auto-launch unreviewed market prices.

Do not describe hypothetical prices as validated purchasing-power pricing.

Currency amounts use minor units plus currency exponent metadata.

Providers

Train:

origin departure instance, boarding stop/date, coach type, class,
rake/configuration, reviewed berth layout.

Bus:

operator, departure time, boarding/dropping points, vehicle type,
deck, seat layout, service identity.

Flight:

operating carrier, flight segment, departure date, cabin,
aircraft layout variant, assignment changes, seat restrictions.

Live lookup requires authorized provider access.

Manual data remains explicitly user-reported.

"""

files["GROWTH AND FEEDBACK.md"] = r"""

Growth and Feedback

Main flywheel

Create real journey -> invite companions -> add seats ->

use shared map -> identify compatible exchange ->

complete coordination -> accepter receives future-use credits ->

accepter returns on a later journey -> invites a new group.

Measure this loop; do not promise virality.

Group utility

Basic group map remains useful without strangers.

Prefill service and date from safe invitation.

Require permission before displaying group members' exact seats.

Allow couples and small groups to get value without recruiting a threshold.

Sharing

Native share sheet and copy-link fallback.

Optional WhatsApp action.

No automated unsolicited messages or contact uploads.

Revocable random invitation tokens with expiry and use limits.

No PNR, private seat data or full names in URLs/social previews.

Rewards

Acceptor reward is distinct from referral rewards.
Avoid stacking incentives without budget and fraud controls.
Reward actual qualified actions, not link clicks.
Do not label OTP-only travelers booking-verified.

Metrics

invite-to-activation conversion
participants per departure
match coverage
mutual agreement rate
both-party completion rate
reward issuance and redemption
accepter's next-journey activation
fraud/complaints
incremental retention versus a comparison cohort

Settings feedback

Give feedback
Report a bug
Report incorrect map
Request feature
Request country/operator support
Report safety issue

Optional attachments and consented diagnostics.
Never automatically attach tickets, chats or booking identifiers.
Admin triage, status tracking, private response, release notification.
Public suggestions require explicit separate consent and moderation.
"""

```
files["SECURITY AND OPERATIONS.md"] = r"""  
# Security and Operations
```

Object authorization on every private API.
Secure sessions and rate-limited authentication.
Schema validation and CSRF defenses where applicable.
Payment webhook signatures validated against raw payload.
Deduplicate provider event IDs.
Do not rely on webhook ordering.
Reconcile payments and credits.

Never store secrets in frontend code.
Private evidence storage with retention limits.
Do not retain booking credentials unnecessarily.
Minimize logs and redact identifiers.

Admin:
MFA, scoped permissions, audit trails,
dual review for large financial adjustments where appropriate.

PWA caching:
allowlist public assets and layouts.
Do not service-worker-cache private booking/payment/chat endpoints.

Moderation:
reports, blocks, disputed seats, fake claims, reward abuse,

appeals, suspensions, support escalation.

Production operations:

CI, migrations, backups, restore testing,
structured monitoring, worker alerts, webhook failure alerts,
financial reconciliation, incident runbooks.

Legal/operator review:

privacy, consumer disclosures, taxes, credits/rewards,
refunds, messaging consent, operator restrictions and data licensing.

Do not describe this package as legal approval.

Do not launch unsupported services because the UI looks complete.

"""

files["TESTS AND ACCEPTANCE.md"] = r"""

Required Tests

Pricing

- Search fee 4900 paise.
- One-sided completion 9900 paise.
- Fixed reward 4900 paise.
- Completion platform allocation 5000 paise.
- Requester total 14800 paise under approved inclusive pricing.
- Owner-unconfirmed pricing blocks live checkout.
- Both qualifying entitlements waive completion and reward.
- Expired/refunded/unrelated activation does not qualify.
- Exact percentage mode returns 4950 when configured.

Payments and credits

- Invalid webhook rejected.
- Duplicate webhook has one effect.
- Out-of-order events converge safely.
- Authorized but uncaptured payment grants no paid reward.
- Pending reward not spendable.
- Duplicate completion awards once.
- Both-party completion required unless audited admin resolution.
- INR 49 credit funds a later INR 49 search with zero cash.
- Concurrent redemption cannot overspend.
- Cash refunds exclude credit-funded portions.
- Late payment does not override expired seat hold.
- No-match credit awarded once at defined deadline.
- Refund/reversal prevents improper reward availability.

Domain

- Overnight boarding resolves correct service.
- Partial segment overlap excluded for MVP.
- Conflicting claims disputed.
- Concurrent acceptances cannot double-commit.
- Equipment changes trigger invalidation.
- Blocked users cannot match/message.
- Protected candidate payloads do not reveal seat geometry.

Real end-to-end acceptance

Two independent authenticated accounts:

```
create actual eligible journeys ->
add seats -> preferences -> paid activation ->
actual candidate -> request -> acceptance ->
conditional checkout -> agreement ->
completion reports -> reward ->
later journey -> redeem reward.
```

```
Use payment sandbox to automate tests.
Then perform a controlled owner-approved live verification
with appropriate genuine transactions and refund checks.
```

```
Production contains no seeded fake passengers or synthetic match counts.
```

```
## Delivery evidence
```

```
Repository, migrations, environment template, deployment instructions,
working deployed URL, test results, webhook setup instructions,
admin access procedure, monitoring and backup procedures,
known integration limitations.
"""
```

```
files["OWNER_INPUTS.md"] = r"""
# Owner Inputs Required
```

```
The agent must implement unblocked work while requesting these inputs.
```

1. Confirm INR 49 reward or exact 50% = INR 49.50.
2. Confirm requester search INR 49 plus conditional INR 99.
3. Confirm whether prices are tax-inclusive.
4. Approve refund and no-match policies.
5. Approve credit redemption and expiration policy.
6. Payment gateway merchant account and secure credentials.
7. Hosting, database and application domain.
8. Authentication and notification providers.
9. Authorized transport data access and reviewed layout sources.
10. Operator-policy scope for launch.
11. Support contact and moderation operators.
12. Actual design images.
13. Launch approval after acceptance testing.

```
Do not ask the owner to paste production secrets into public files.
Use the platform's secret manager.
"""
```

```
files["DESIGN REFERENCES/README.md"] = r"""
Place actual design image exports here.
No generated screenshots are embedded in this package.
```

```
Required references:
country selector, transport home, journey entry, own map,
preferences, protected match, fee disclosure, request tracking,
accepter reward, wallet, group map, Settings and feedback.
```

```
Business and privacy rules override contradictory screenshots.
"""
```

```
files["IMPLEMENTATION STATUS.md"] = r"""
# Implementation Status
```

Specification supplied; application not implemented by this package.

The coding agent must record:

feature, implementation status, tests, deployed evidence, blockers.

Do not equate:

UI built with backend complete,
sandbox payment with live payment complete,
provider interface with provider integration,
manual claim with booking verification.

"""

files["PROMPT_TO_PASTE.txt"] = r"""

Build the real working SwapSeat application from this v2 package.

This is not a request for a demo or a static prototype.

Follow START_HERE.md and implement production-capable frontend,
backend, database, matching, payments, wallet credits,
group invitations, feedback, administration and deployment.

Inspect the existing repository first and preserve suitable working code.

Continue through the implementation stages; do not stop after mock screens.

The revised India proposal is:

INR 49 search activation;

INR 99 additional completion charge for a one-sided exchange;

INR 49 future-use credits for the non-searching acceptor after completion;

no additional completion fee or reward when both already qualify.

The owner must confirm INR 49 versus exact 50% = INR 49.50

before live payment enablement.

Use mocks only in tests/development.

Production must show honest unavailable/manual states when an
external provider is not configured.

Request missing credentials and owner decisions without fabricating them.

Keep pricing, eligibility, credit ledger, rewards, payment verification
and seat disclosure server-side.

Run tests and supply deployment/setup documentation.

Update IMPLEMENTATION_STATUS.md with actual evidence and blockers.

"""

files["config/pricing-IN.json"] = json.dumps({

 "version": "IN-v2-proposed",

 "currency": "INR",

 "minorUnitExponent": 2,

 "searchFeeMinor": 4900,

 "oneSidedCompletionFeeMinor": 9900,

 "reward": {

 "mode": "fixed minor",

 "fixedAmountMinor": 4900,

 "alternativePercentageBps": 5000,

 "releaseCondition": "completed exchange and risk_checks",

 "redeemOn": ["future search activation"],

 "maxRedemptionPercent": 100,

 "cashWithdrawal": False,

 "transferable": False

 },

```

        "waiveCompletionWhenBothQualify": True,
        "rewardWhenBothQualify": False,
        "taxTreatment": "owner review required",
        "ownerConfirmed": False,
        "liveEnabled": False,
        "ambiguity": "INR 49 fixed reward versus exact 50% of INR 99."
    }, indent=2)

files["config/coverage.json"] = json.dumps({
    "defaultTravelCountry": "IN",
    "countries": [{
        "code": "IN",
        "train": "requires production readiness",
        "bus": "requires provider and layout readiness",
        "flight": "requires provider and layout readiness"
    }],
    "unknownCountryBehavior": "waitlist or explicit supported manual flow",
    "allowCountrySelectorToOverrideCheckoutMarket": False
}, indent=2)

for relative_path, content in files.items():
    target = ROOT / relative_path
    target.parent.mkdir(parents=True, exist_ok=True)
    target.write_text(content.strip() + "\n", encoding="utf-8")

with ZipFile(ZIP_PATH, "w", ZIP_DEFLATED) as archive:
    for path in sorted(ROOT.rglob("*")):
        if path.is_file():
            archive.write(path, path.relative_to(ROOT.parent))

print(f"Created: {ZIP_PATH.resolve()}")
print(f"Included files: {len(files)}")
print("Upload this ZIP and actual design images to your coding platform.")
print("Paste PROMPT_TO_PASTE.txt as your implementation instruction.")

```

The package also requires signed payment verification and duplicate-event handling rather than trusting the checkout success screen; Razorpay's integration materials explicitly cover signature verification and duplicate webhook events. (github.com)

SwapSeat Mobile-first PWA Prototype (Next.js + Postgres + Mock APIs) with Trust/Safety and Reduced Flow Steps^{6 STEPS}

1.

Scaffold Next.js PWA + design system foundations

- Create a Next.js (App Router) project with a phone-first shell and PWA installability.

- Create/modify:

- `next.config.ts` (PWA headers/caching strategy if needed)
- `src/app/layout.tsx` (global providers, viewport meta, fonts)
- `src/app/globals.css` (tokens + base styles)
- `public/manifest.webmanifest`
- `public/icons/*` (maskable + standard icons: 192/512 + apple-touch)
- `src/app/offline/page.tsx` (offline fallback page)

| | |
|--|---|
| • PWA Requirements: | |
| • Manifest: name “SwapSeat”, theme_color deep emerald, display standalone, start_url /, icons include maskable. | |
| • Service worker: | <ul style="list-style-type: none"> • Use next-pwa or a Workbox-based custom service worker. • Cache strategy: <ul style="list-style-type: none"> • App shell (HTML/CSS/JS): precache. • API responses (mock APIs): stale-while-revalidate. • Images: cache-first with max entries. • Offline behavior: <ul style="list-style-type: none"> • Allow viewing already-visited screens and cached mock data. • Show offline banner + retry CTAs when API calls fail. |
| • Mobile-first layout constraint: | |
| • Implement a centered “phone frame” container on desktop to cap width (e.g. 420–480px) while still responsive on real phones. | |
| • Create src/components/layout/PhoneShell.tsx with: | <ul style="list-style-type: none"> • Sticky top step header area • Scrollable content • Bottom tab bar (Home / My Trips / Messages / Profile) |
| • Design system (tokens + reusable primitives): | |
| • Add tokens in globals.css (or Tailwind config if using Tailwind): | <ul style="list-style-type: none"> • Colors: --brand-emerald: #0B7A4B, neutrals, success/warn/error, surface white cards • Radii: 12/16/24 • Shadows: soft elevation levels • Spacing scale: 4/8/12/16/24/32 • Typography: mobile legible sizes + line heights |
| • Create reusable UI components (keep minimal but consistent): | <ul style="list-style-type: none"> • src/components/ui/Button.tsx, Card.tsx, Chip.tsx, Tabs.tsx, BottomSheet.tsx, ProgressSteps.tsx • src/components/ui/Skeleton.tsx, EmptyState.tsx, InlineError.tsx, Toast.tsx |
| • Accessibility defaults: | <ul style="list-style-type: none"> • 44px tap targets, focus rings, semantic headings, aria-labels for icon buttons, sufficient contrast on emerald. |

2.

Model the domain in Postgres + seed realistic mock data

| | |
|--|--|
| • Add Postgres + migrations (Prisma recommended for speed in prototyping). | |
| • Create/modify: | <ul style="list-style-type: none"> • prisma/schema.prisma • prisma/migrations/* • src/lib/db.ts |

| | |
|---|--|
| | <ul style="list-style-type: none"> prisma/seed.ts |
| • Core entities (support both Requester and Acceptor flows; payments mocked): | |
| • User | <ul style="list-style-type: none"> id, name, avatarUrl, ageRange (don't store exact DOB for prototype), gender (Optional), verificationStatus (unverified/pending/verified), createdAt Safety: blockedUserIds (or join table), reportedCount |
| • Journey | <ul style="list-style-type: none"> id, mode (train/bus/flight), from, to, departureAt, arrivalAt, carrierName, serviceNumber (train no/flight no/bus id), pnrLast4 (store only partial), createdByUserId |
| • Coach (for trains primarily; allow null for other modes) | <ul style="list-style-type: none"> id, journeyId, code (B9/B10), layoutType, metaJson |
| • Seat | <ul style="list-style-type: none"> id, coachId, seatNumber, berthType (lower/middle/upper/side-lower/side-upper/aisle/window), x, y |
| • Trip | <ul style="list-style-type: none"> id, journeyId, userId, coachCode, seatIds[] (or join table TripSeat), travellingTill (station code/string), status (upcoming/past) |
| • SwapRequest | <ul style="list-style-type: none"> id, journeyId, requesterTripId, desiredSeatTypes[], reasonCode, reasonNote, onlyTillDestination boolean status (draft/active/matched/confirmed/expired/cancelled/no_match) Privacy fields: seatVisibility state machine (hidden → revealed_on_payment → revealed_on_confirm) |
| • Match | <ul style="list-style-type: none"> id, swapRequestId, acceptorTripId, score (0–100), highlightsJson (reasons), status (suggested/sent/declined/accepted/expired) |
| • Payment | <ul style="list-style-type: none"> id, userId, contextType (request_fee/confirm_fee), contextId, amountPaise (4900), status (created/succeeded/failed/refunded), provider (mock), providerRef Model refunds/failures explicitly for the requested handling. |
| • ChatThread + ChatMessage | |

| |
|--|
| <ul style="list-style-type: none"> <div>threadId</div>, <div>journeyId</div>, <div>requesterUserId</div>, <div>accepterUserId</div>, <div>status</div>
 <div>(open/closed)</div>, <div>closesAt</div> (arrival time) |
| <ul style="list-style-type: none"> Seed data: <ul style="list-style-type: none"> 1–2 sample journeys per mode, 3 coaches with seat grids for trains, 15–25 travellers with varied profiles, and precomputed match scores. Include edge cases in seed: <ul style="list-style-type: none"> No matches, payment failed, refund issued, blocked user, reported user, offline cached journey. |

3.

Build mock API routes + match/scoring + privacy/trust rules

| |
|--|
| <ul style="list-style-type: none"> Implement Next.js Route Handlers that back the clickable prototype with real CRUD (but no real PNR/payment gateway). |
| <ul style="list-style-type: none"> Create/modify: <ul style="list-style-type: none"> <div>src/app/api/journeys/route.ts</div> (list/create) <div>src/app/api/journeys/[id]/route.ts</div> (detail) <div>src/app/api/journeys/[id]/coaches/route.ts</div> <div>src/app/api/coaches/[id]/seats/route.ts</div> <div>src/app/api/trips/route.ts</div> (create from PNR-last4/manual form; mocked validation) <div>src/app/api/swap-requests/route.ts</div> (create/list) <div>src/app/api/swap-requests/[id]/route.ts</div> (get/update status) <div>src/app/api/swap-requests/[id]/matches/route.ts</div> (list + filters) <div>src/app/api/matches/[id]/send/route.ts</div> (mark “sent” to traveller(s)) <div>src/app/api/matches/[id]/respond/route.ts</div> (accept/decline) <div>src/app/api/payments/create/route.ts</div> (mock create) <div>src/app/api/payments/confirm/route.ts</div> (mock success/failure) <div>src/app/api/payments/refund/route.ts</div> (mock refund flow) <div>src/app/api/chat/threads/route.ts</div>,
<div>src/app/api/chat/threads/[id]/messages/route.ts</div> <div>src/app/api/notifications/subscribe/route.ts</div> (store push subscription, mocked) |
| <ul style="list-style-type: none"> Match scoring (simple, explainable, UI-friendly): <ul style="list-style-type: none"> Compute score by seat-type alignment, same coach preference, comfort score proximity, travellerTill compatibility, and reason affinity (e.g., family/senior/medical prioritized). Return “highlights” for UI (e.g., “Same coach”, “Lower berth”, “Both travelling till Secunderabad”). |
| <ul style="list-style-type: none"> Privacy + trust/safety enforcement in APIs: <ul style="list-style-type: none"> Seat numbers hidden pre-payment: |

| |
|--|
| <ul style="list-style-type: none"> In matches list: show coach code optionally, but seat number masked (“B10 • Seat **”). Reveal seat numbers only if requester has paid ₹49 for the journey (one-time) AND match is accepted / in compare screen rules you define. |
| <ul style="list-style-type: none"> Personal detail minimization: <ul style="list-style-type: none"> No phone numbers; chat only. Age as range (e.g., 25–34) not exact. |
| <ul style="list-style-type: none"> Report/block endpoints (even if basic for prototype): <ul style="list-style-type: none"> src/app/api/safety/report/route.ts src/app/api/safety/block/route.ts |
| <ul style="list-style-type: none"> Verification badge rules: <ul style="list-style-type: none"> Seed some verified users; enforce badge display based on verificationStatus. |
| <ul style="list-style-type: none"> Payment handling logic (mocked but realistic states): <ul style="list-style-type: none"> Requester pays once per journey to send requests (₹49): <ul style="list-style-type: none"> If payment fails: allow retry; keep draft request. If “no match found”: allow refund request path or credit (model as refunded payment in prototype). Acceptor pays to confirm swap (₹49): <ul style="list-style-type: none"> If payment succeeds: mark swap confirmed and open chat thread. If payment fails/abandoned: keep match in “accepted_pending_payment” for limited time then expire.** |

4.

Implement streamlined Requester flow (reduced steps + progress + great states)

| |
|--|
| <ul style="list-style-type: none"> Build screens as routes with a shared stepper header and progressive disclosure to reduce step count. |
| <ul style="list-style-type: none"> Create/modify: <ul style="list-style-type: none"> src/app/(app)/home/page.tsx (Welcome) src/app/(app)/request/new/page.tsx (single “Plan journey” entry; replaces separate welcome→plan if desired) src/app/(app)/request/new/seats/page.tsx src/app/(app)/request/new/preferences/page.tsx src/app/(app)/request/new/review/page.tsx src/app/(app)/request/[id]/matches/page.tsx src/app/(app)/request/[id]/send/page.tsx (or bottom sheet within matches) src/app/(app)/pay/request-fee/page.tsx src/app/(app)/pay/success/page.tsx src/app/(app)/my-requests/page.tsx (Active/Completed) |
| <ul style="list-style-type: none"> Flow improvements (explicitly implement): |

| |
|---|
| <ul style="list-style-type: none"> Combine steps where possible: <ul style="list-style-type: none"> “Seat Map” + “Your Seat(s)” can be one screen: select seats → shows dynamic chips + comfort score panel below. “Potential Matches” + “Send to Travellers” can be one screen with multi-select and a sticky CTA (“Send Requests (N)”). |
| <ul style="list-style-type: none"> Progress indicators: <ul style="list-style-type: none"> ProgressSteps component shows 1) Journey 2) Seats 3) Preferences 4) Review 5) Matches. Persist draft in local storage to survive refresh/offline: <code>src/lib/draftStore.ts</code>. |
| <ul style="list-style-type: none"> UI components to implement for the flow: <ul style="list-style-type: none"> <code>src/components/seatmap/SeatMap.tsx</code> <ul style="list-style-type: none"> Coach selector tabs (B9/B10/B11), grid layout, legend, multi-select. “View full coach layout” opens full-screen bottom sheet/modal. <code>src/components/comfort/ComfortScore.tsx</code> <ul style="list-style-type: none"> Meter (0–10) + explainable bullets (washroom distance, window/aisle, berth level). <code>src/components/travellers/TravellerCard.tsx</code> <ul style="list-style-type: none"> Avatar, verification badge, match %, note, and masked seat info pre-payment. <code>src/components/share/ShareCard.tsx</code> for “Need More Matches” (WhatsApp etc; links mocked). Empty/loading/error states (must be designed, not default): <ul style="list-style-type: none"> Skeletons for seat map and traveller cards. Empty matches state: explain what happens next (“We’ll keep searching”), CTA to broaden filters and share. Network/offline: <ul style="list-style-type: none"> Banner “You’re offline—showing saved info” and disable payment/send actions. “Only travellers going till my destination” toggle: <ul style="list-style-type: none"> Implement filter in match query; explain with helper text. |

5.

Implement Acceptor flow + compare UI + chat lifecycle + trip timeline

| |
|---|
| <ul style="list-style-type: none"> Build acceptor experience including notification entry, request details, compare, payment confirm, chat, and journey tracking. |
| <ul style="list-style-type: none"> Create/modify: <ul style="list-style-type: none"> <code>src/app/(app)/inbox/page.tsx</code> (Messages tab; list incoming requests) <code>src/app/(app)/incoming/[matchId]/page.tsx</code> (Request Details) <code>src/app/(app)/incoming/[matchId]/coach/page.tsx</code> (Coach Layout masked) <code>src/app/(app)/incoming/[matchId]/compare/page.tsx</code> (Compare & Decide) <code>src/app/(app)/pay/confirm-fee/page.tsx</code> <code>src/app/(app)/swap/confirmed/[threadId]/page.tsx</code> |

| |
|---|
| <ul style="list-style-type: none"> src/app/(app)/chat/[threadId]/page.tsx src/app/(app)/trip/[tripId]/tracking/page.tsx src/app/(app)/trip/[tripId]/arrived/page.tsx src/app/(app)/chat/[threadId]/closed/page.tsx src/app/(app)/my-trips/page.tsx (upcoming/past) |
| • Compare & decide screen (key UX deliverable): |
| <ul style="list-style-type: none"> Side-by-side cards: “Your seat” vs “Their seat” Comfort scores + pros/cons chips: <ul style="list-style-type: none"> window/aisle, berth level, near washroom, legroom, luggage, “closer to family seats” Prominent trust actions: View profile (limited), Report/Block, Decline. |
| • Chat lifecycle requirements: |
| <ul style="list-style-type: none"> “chat available till destination, auto-closes after arrival” Enforce via ChatThread.closesAt and API guarding: <ul style="list-style-type: none"> After close: messages become read-only; show locked state with “View Conversation”. |
| • JOURNEY TRACKING: |
| <ul style="list-style-type: none"> Use mock station timeline data (Journey.metaJson) rendered in TrackingTimeline.tsx. Show “running status” changes via mocked polling or SWR revalidation. |
| • “View E-Ticket (PDF)”: |
| <ul style="list-style-type: none"> For prototype: generate a simple PDF-like page or downloadable HTML-to-PDF stub: <ul style="list-style-type: none"> src/app/(app)/ticket/[swapId]/page.tsx (render printable ticket) |
| • Add to calendar: |
| <ul style="list-style-type: none"> Generate .ics file route: <ul style="list-style-type: none"> src/app/api/calendar/[tripId].ics/route.ts |

6.

Trust & safety, notifications strategy, and production-grade UX polish

| |
|---|
| • Trust & safety layer across the app: |
| <ul style="list-style-type: none"> Verification badges: <ul style="list-style-type: none"> UI Component VerifiedBadge.tsx; display in traveller cards, profiles, request details. Report/Block UX: <ul style="list-style-type: none"> ReportUserSheet.tsx and BlockUserConfirm.tsx (bottom sheets) Update APIs + hide blocked users from match lists/inbox. Privacy: <ul style="list-style-type: none"> Mask seat numbers until payment rules satisfied. Mask personal details (no phone, no exact DOB, no last names if desired). |
| • Payment failure/refund/no-match handling (must be visible in UI): |
| <ul style="list-style-type: none"> Payment states: |

| | |
|--|---|
| | <ul style="list-style-type: none"> “Failed” screen with retry + alternative method “Pending” state (simulate UPI pending) Refund request: allow “Request refund if no match within X hours” (prototype rule) → sets <code>Payment.status=refunded</code> |
| • No match: | <ul style="list-style-type: none"> After a threshold (e.g., 2 hours simulated) mark request <code>no_match</code>; <p>Show:</p> <ul style="list-style-type: none"> Expand preferences CTA Share CTA Refund/credit CTA |
| • Push/local notification strategy for PWA: | |
| • For prototype: | <ul style="list-style-type: none"> Implement Notifications permission prompt only after a user action (“Enable alerts”). Local notifications (in-app) via toasts + inbox badges. |
| • For web push (optional but planned in code): | <ul style="list-style-type: none"> Store push subscription in <code>notifications/subscribe</code>. Service worker listens for <code>push</code> event and displays notification (if implementing real push later). Provide a “lock-screen style” notification mock screen route <code>src/app/(app)/debug/notification/page.tsx</code> to match design. |
| • Accessibility + one-handed reach: | <ul style="list-style-type: none"> Keep primary CTAs sticky near bottom. Ensure all icon-only buttons have labels; focus management for bottom sheets. Increase line height and spacing; avoid tiny seat tap targets (seat cells min 36–44px with zoom option). |
| • Final “clickable prototype” polish: | <ul style="list-style-type: none"> Add realistic transitions (page enter, bottom sheet), but keep performant. Ensure all flows are navigable end-to-end using seeded data: <ul style="list-style-type: none"> Requester creates request → sees matches → pays → sends → tracks status Acceptor receives → compare → pays → confirmed → chat → tracking → arrived → chat closed Make sure desktop shows phone-width container and still allows scrolling with visible bottom tabs. |

This is a request to build an ultimate mobile-first PWA for a seat swapping app with flowchart visuals for flight, bus, and train journeys. Since I'm in plan mode, I should call `create_plan` with a clear, descriptive instruction capturing this scope.

Ultimate Mobile-First SeatSwap PWA (Flights, Buses, Trains) — Phased Build Plan 6 STEPS

1.

Foundation: Next.js PWA shell, navigation, design system

Implement an installable, mobile-first PWA scaffold with the required UI chrome and design system so the app is “visible and clickable” immediately.

| |
|---|
| • Create/modify PWA + app shell |
| <ul style="list-style-type: none"> • <code>next.config.ts</code> (OR <code>next.config.mjs</code>): enable PWA build (recommend <code>@ducanh2912/next-pwa</code>), set caching strategy. • <code>public/manifest.webmanifest</code>: name, short_name, start_url, scope, display=standalone, background/theme colors, shortcuts (Trips, Requests, Inbox). • <code>public/icons/*</code>: generate full icon set (maskable + standard: 48→512; iOS apple-touch-icon variants). • <code>src/app/layout.tsx</code>: install meta tags, theme-color, viewport, apple mobile web app tags. • <code>src/app/offline/page.tsx</code> (optional) + offline fallback route for SW navigation errors. |
| • Service worker/offline shell decisions |
| <ul style="list-style-type: none"> • Cache the “app shell” routes (tabs) + static assets; network-first for trip data. • Ensure safe behavior when offline: show cached trips/mock + “limited mode” banner. |
| • Mobile-first layout + bottom tab bar + safe-area |
| <ul style="list-style-type: none"> • <code>src/components/layout/AppShell.tsx</code>: fixed bottom tab bar, content area padding, safe-area insets (<code>env(safe-area-inset-bottom)</code>). • <code>src/components/nav/BottomTabs.tsx</code>: Tabs: Trips, Swap, Inbox, Notifications, Profile. • Ensure “overlapping border” tab design per spec. |
| • Design system implementation (“High-Fidelity SaaS” adapted to touch UI) |
| <ul style="list-style-type: none"> • Fonts: Inter (<code>src/app/layout.tsx</code> with <code>next/font/google</code>). • Styling approach: Tailwind (recommended) or CSS modules—choose one and codify tokens: <ul style="list-style-type: none"> • 1px ghost gray borders, pill taxonomy chips, flat elevation (no resting shadows), action blue, hyphen lists, circular data rings. |
| • Add reusable primitives: |
| <ul style="list-style-type: none"> • <code>src/components/ui/Button.tsx</code>, <code>Input.tsx</code>, <code>Card.tsx</code>, <code>Pill.tsx</code>, <code>Ring.tsx</code>, <code>List.tsx</code> (hyphen list), <code>Modal.tsx</code>, <code>Sheet.tsx</code>. |
| • Auth wiring placeholder (UI-only in this step) |
| <ul style="list-style-type: none"> • Add guarded routes and session placeholders: <ul style="list-style-type: none"> • <code>src/lib/auth/requireAuth.ts</code> (stub) • <code>src/app/(auth)/sign-in/page.tsx</code>, <code>src/app/(auth)/onboarding/page.tsx</code> • Decision: plan for NextAuth (Auth Providers) + Postgres adapter later; keep UI callable now with mocked session. |

2.

Mocked end-to-end flows: onboarding → trips → seat maps → swaps → chat → receipt

Build the complete, clickable journeys for flights, buses, trains using mock data that mirrors the uploaded flow diagrams (implement screens + local state, no backend yet).

• Route structure (App Router)

| | | |
|---|---|---|
| • | src/app/(app)/trips/page.tsx | (Trip Dashboard) |
| • | src/app/(app)/trips/add/page.tsx | (Add/Import Trip) |
| • | src/app/(app)/trips/[tripId]/page.tsx | (Trip Details) |
| • | src/app/(app)/trips/[tripId]/seatmap/page.tsx | (Interactive Seat Map) |
| • | src/app/(app)/trips/[tripId]/preferences/page.tsx | (Seat Preferences) |
| • | src/app/(app)/trips/[tripId]/swap/page.tsx | (Browse Requests + Suggestions) |
| • | src/app/(app)/swap/offers/[offerId]/page.tsx | (Offer thread:
offer/counter/accept/decline) |
| • | src/app/(app)/inbox/page.tsx, src/app/(app)/inbox/[threadId]/page.tsx | (Trip-scoped chat) |
| • | src/app/(app)/notifications/page.tsx | |
| • | src/app/(app)/profile/page.tsx | (Trust score, settings, blocks) |

| | | |
|--|---|---|
| ● Mock data layer | | |
| ● | src/mocks/data.ts | users, trips (3 modes), seat layouts, assignments, swap requests, offers, messages. |
| ● | src/mocks/api.ts | async functions that mimic latency and state transitions. |
| ● Add/Import Trip UX (mode-aware) | | |
| ● | Form fields (per request): PNR/booking ref/ticket number + carrier + date; for trains add boarding station; for buses add pickup point. | |
| ● | Trip verification state UI: “Pending verification / Verified / Failed” with retry + help sheet. | |
| ● Interactive seat maps (mocked but realistic structure) | | |
| ● | Shared seat map base: | |
| | src/components/seatmap/SeatMapCanvas.tsx | (pan/zoom, tap targets, legends) |
| | src/components/seatmap/SeatLegend.tsx, | SeatTile.tsx, SeatPopover.tsx |
| ● | Flights: | |
| | Cabin layout with rows/aisles/classes; mark exit rows, extra legroom, paid seats. | |
| | src/components/seatmap/flight/FlightCabin.tsx | |
| ● | buses: | |
| | Deck selector (lower/upper), sleeper vs seater, double-decker support, gender-restriction indicators. | |
| | src/components/seatmap/bus/BusDeck.tsx | |
| ● | TRAINS: | |
| | Coach selector, berth types (lower/middle/upper/side), multi-leg segment indicator. | |
| | src/components/seatmap/train/TrainCoach.tsx | |
| ● Preferences setup (mode-aware taxonomy) | | |
| ● | src/components/preferences/PreferenceForm.tsx | renders common + mode-specific preferences: |

| | |
|---|--|
| | <ul style="list-style-type: none"> • Common: window/aisle (where applicable), front/back, together-with-group, near exit, quiet zone. • Flight: class, exit row, extra legroom, paid-seat parity constraint. • Bus: deck, sleeper/seater, gender-preference rules (where allowed), near door. • Train: coach, lower/middle/upper/side, near restroom, direction of travel if relevant. |
| • Swap marketplace + offer lifecycle (UI state machine) | <ul style="list-style-type: none"> • <code>src/components/swap/RequestList.tsx</code>, <code>SuggestionList.tsx</code>, <code>OfferComposer.tsx</code>, <code>OfferStatusBadge.tsx</code> • Statuses: Draft → Sent → Countered → Accepted → Declined → Expired → Completed. • Include “counter-offer” UX and clear “what each party gives/gets” summary card. |
| • Chat (mock) | <ul style="list-style-type: none"> • Trip-scoped threads, no contact detail prompts; add “Safety tips” banner + report entry points. |
| • Swap confirmation receipt (mock) | <ul style="list-style-type: none"> • <code>src/app/(app)/swap/receipt/[swapId]/page.tsx</code> • Shareable/scannable record: render QR code (e.g. <code>qrcode.react</code>) containing a signed-looking payload (mock now): <code>swapId</code>, <code>tripId</code>, <code>seatA</code>, <code>seatB</code>, <code>timestamp</code>. |
| • Post-swap rating/report/block UI (mock) | <ul style="list-style-type: none"> • <code>src/components/trust/RateSwap.tsx</code>, <code>ReportUserSheet.tsx</code>, <code>BlockUserDialog.tsx</code> • Trust score ring component using circular data rings. |

3.

Backend: Postgres schema, migrations, and core API routes

Replace mock data with a real Postgres model and Next.js API routes (or route handlers) covering all entities and workflows.

| | |
|---|--|
| • Tech choices (recommended for “Anything” fullstack) | <ul style="list-style-type: none"> • ORM: Prisma (fast iteration) or Drizzle (type-safe SQL). Pick one and stick to it. • Use Postgres enums for mode/status where possible. |
| • Create schema + migrations | <ul style="list-style-type: none"> • <code>prisma/schema.prisma</code> (OR <code>src/db/schema.ts</code> if Drizzle) • Tables (minimum requested, with key fields): <ul style="list-style-type: none"> • <code>users</code>: <code>id</code>, <code>email/phone</code> (hashed/normalized), <code>created_at</code>, <code>last_active_at</code> • <code>profiles</code>: <code>user_id</code>, <code>display_name</code>, <code>avatar_url</code>, <code>trust_score</code>, <code>locale</code>, <code>marketing_opt_in</code> • <code>blocks</code>: <code>blocker_id</code>, <code>blocked_id</code>, <code>reason</code>, <code>created_at</code> • <code>reports</code>: <code>reporter_id</code>, <code>reported_id</code>, <code>trip_id</code>, <code>category</code>, <code>description</code>, <code>status</code>, <code>created_at</code> • <code>carriers</code>: <code>id</code>, <code>mode</code> (flight/bus/train), <code>name</code>, <code>code</code> (IATA/ICAO/operator code), <code>region</code> |

- `vehicles`: id, carrier_id, type (aircraft/bus/train), model, metadata
- `seat_layouts`: id, vehicle_id, version, layout_json (structured), created_at
- `trips`: id, mode, carrier_id, vehicle_id (nullable until verified), travel_date, origin, destination, departure_time, arrival_time, status (imported/pending_verified/verified/cancelled/completed)
- `trip_legs` (for trains multi-leg): trip_id, seq, origin, destination, dep/arr, boarding_station
- `trip_members`: trip_id, user_id, role (traveler/admin), verified (bool), verification_method, created_at
- `seat_assignments`: trip_id, user_id, seat_id (string), coach/deck/cabin, class/berth_type, paid_flag, constraints_json
- `swap_preferences`: trip_id, user_id, preference_json, updated_at
- `swap_requests`: id, trip_id, user_id, from_seat_id, desired_constraints_json, note, visibility, status (open/paused/closed), created_at
- `swap_offers`: id, trip_id, request_id, from_user_id, to_user_id, offer_json (what seats), status, expires_at, created_at, updated_at
- `swap_matches` (accepted offers): id, trip_id, offer_id, status (accepted/confirmed/completed/void), confirmed_at, completed_at
- `messages`: id, trip_id, thread_id, sender_id, body, redacted_body, created_at
- `notification_events`: id, user_id, trip_id, type, payload_json, read_at, created_at
- `audit_log`: id, actor_id, trip_id, action, entity_type, entity_id, before_json, after_json, created_at (required by request)

• API routes (Next.js Route Handlers)

- `src/app/api/auth/*` (once auth is enabled; in this step can stub)
- `src/app/api/trips` (GET list, POST create/import)
- `src/app/api/trips/[tripId]` (GET details)
- `src/app/api/trips/[tripId]/verify` (POST trigger verification state change)
- `src/app/api/trips/[tripId]/seatmap` (GET layout + assignments relevant to user)
- `src/app/api/trips/[tripId]/preferences` (GET/PUT)
- `src/app/api/trips/[tripId]/requests` (GET/POST)
- `src/app/api/trips/[tripId]/offers` (POST create offer)
- `src/app/api/offers/[offerId]` (GET, POST counter/accept/decline)
- `src/app/api/matches/[matchId]/receipt` (GET receipt payload for QR)
- `src/app/api/inbox/threads` + `threads/[threadId]/messages` (GET/POST)
- `src/app/api/notifications` (GET, POST mark read)

| |
|--|
| <ul style="list-style-type: none"> <code>src/app/api/reports</code> (Post), <code>src/app/api/blocks</code> (POST/DELETE) |
| • Replace mocks with real data fetching |
| <ul style="list-style-type: none"> Add data access layer: <ul style="list-style-type: none"> <code>src/lib/db.ts</code>, <code>src/lib/repos/*.ts</code> (TripsRepo, SwapRepo, MessagesRepo) Update pages/components to call server actions or route handlers (pick one pattern and apply consistently). |

4.

Authentication, trip ownership verification, and trust & safety controls

Enable authentication, enforce trip-scoped access, add verification mechanics, and implement safety constraints (no PII leakage, rate limits, report/block).

| |
|---|
| • Authentication enablement (callout: required for most features) |
| <ul style="list-style-type: none"> Implement NextAuth (or platform auth) with session cookies: <ul style="list-style-type: none"> <code>src/auth.ts</code>, <code>src/app/api/auth/[...nextauth]/route.ts</code> Gate all (app) routes with middleware: <ul style="list-style-type: none"> <code>src/middleware.ts</code> (redirect to <code>/sign-in</code>) |
| • Trip ownership verification (mode-specific, with practical “ultimate version” approach) |
| <ul style="list-style-type: none"> Implement verification states and methods: <ul style="list-style-type: none"> Manual proof upload (ticket screenshot) + admin/auto review (MVP-friendly) Email/PNR verification integrations as later add-ons Data Model: <code>trip_members.verified</code>, <code>verification_method</code>, store artifacts securely (no raw PII stored if possible). Add “verified badge” required to: <ul style="list-style-type: none"> Create swap requests send offers participate in chat generate receipt QR |
| • Trust & safety enforcement |
| <ul style="list-style-type: none"> Prevent PII leakage in chat: <ul style="list-style-type: none"> Server-side message filter/redaction pipeline for emails/phones/URLs. Store <code>body</code> + <code>redacted_body</code>; Render <code>redacted_body</code> By default. Rate limits (critical) <ul style="list-style-type: none"> Apply to: login attempts, trip import/verify attempts, offer sends, messages. Implement with Upstash Redis or Postgres-based limiter (Upstash recommended). <code>src/lib/ratelimit.ts</code> + wrap API routes. Block/report <ul style="list-style-type: none"> Blocked users cannot message/offer; hide their requests. Reporting workflow creates <code>reports</code> row and flags trust scoring. Audit trail |

- Log transitions for offers/matches, verification changes, block/report actions in `audit_log`.

5.

Matching engine: server-side scoring + mode rules + 3-way chain (stretch)

Implement a smart ranking engine that scores candidate swaps by mutual preference satisfaction, includes mode-specific constraints, and optionally supports 3-way chains.

| | |
|--|---|
| • Core matching architecture | |
| • | <code>src/lib/matching/types.ts</code> : normalized seat attributes + preference model across modes. |
| • | <code>src/lib/matching/score.ts</code> : weighted scoring (0–100) for (user A seat → user B seat) and mutual satisfaction. |
| • | <code>src/app/api/trips/[tripId]/suggestions/route.ts</code> : returns ranked suggestions for a user/request. |
| • Scoring inputs | |
| • | Seat attributes: position (window/aisle/middle), row/zone (front/mid/back), class/cabin, exit row, legroom, quiet zone, near door, deck, sleeper/seater, coach, berth type, proximity to group seats. |
| • | Preferences: “hard constraints” vs “soft preferences” (must-have vs nice-to-have). |
| • Mode-specific constraints & rules (explicitly implement) | |
| • | Flights: |
| • | <ul style="list-style-type: none"> • Paid seat parity: prevent offers that downgrade a paid seat unless explicitly allowed by both. • Cabin class lock unless both accept cross-class. • Group swaps: allow multi-seat offers (2↔2) as a supported “ultimate” rule set. |
| • | buses: |
| • | <ul style="list-style-type: none"> • Deck selection constraints. • Sleeper vs seater cannot cross unless operator allows. • Gender-preference rules: enforce when provided by operator/region flag; store in <code>seat_assignments.constraints_json</code> or layout metadata. • Pickup point association: ensure same service/boarding. |
| • | TRAINS: |
| • | <ul style="list-style-type: none"> • Coach selector and berth-type matching. • Multi-leg: ensure swap is valid for the overlapping segment; incorporate boarding station constraints. |
| • Output format for UI | |
| • | Return explanation payload per suggestion: |
| • | <ul style="list-style-type: none"> • score breakdown, satisfied prefs, violated prefs, rule blocks (if any), “why this match”. |
| • | UI uses this to render pill taxonomy chips and “match quality” rings. |
| • Stretch: 3-way swap chains | |
| • | Model as directed graph: user wants seat X, offers seat Y. |
| • | Implement cycle finding limited to length 3 (A→B→C→A) for performance. |

- `src/lib/matching/threeWay.ts`: propose chain + require all accept; create `swap_offers` grouped by `chain_id` and a coordinator `swap_match`.
- Add UI later behind feature flag.

Real-time chat, notifications (push + in-app), and swap receipts for crew/conductor

Upgrade communications to real-time where feasible, add web push, in-app notification center, and production-grade confirmation receipts.

- Chat implementation (trip-scoped, safe)
 - Choose transport:
 - Option A: WebSockets (e.g., Pusher/Ably) for “ultimate” experience.
 - Option B: SSE + polling fallback.
 - server:
 - `src/app/api/inbox/threads/[threadId]/messages` persists messages and emits events.
 - client:
 - `src/lib/realtime.ts` subscribe per `tripId/threadId`.
- Notifications
 - In-app notification center
 - Store events in `notification_events` and render in `src/app/(app)/notifications/page.tsx`.
 - Web Push where supported
 - Add VAPID keys and subscription storage:
 - `push_subscriptions`: `user_id`, `endpoint`, `keys`, `created_at`
 - `src/app/api/push/subscribe/route.ts`,
`src/app/api/push/send/route.ts` (internal)
 - Trigger events: new offer, counter-offer, accepted swap, boarding reminder.
 - Fallback behavior
 - If push unsupported/denied: rely on in-app + badge counts on tabs.
- Swap confirmation receipts (shareable/scannable)
 - Make receipts tamper-resistant:
 - Server generates signed payload (HMAC) with swap/match id, trip id, both user ids (opaque), seats, timestamp, verification status.

`src/lib/receipt/sign.ts`, `src/app/api/matches/[matchId]/receipt` returns QR data + human-readable summary.

"Show to crew/conductor" mode

High-contrast view, offline-capable (cache receipt page via SW).

Boarding reminders

Scheduler integration required:

Use a cron provider (Vercel Cron / platform scheduler) to enqueue reminders based on trip departure time.

`src/app/api/jobs/boarding-reminders/route.ts` (secured) to fan out notifications.

Integrations callouts (explicit)

Real-time provider (Pusher/Ably) if selecting websockets.

Web push VAPID keys + subscription storage.

Scheduler/cron for reminders.

Optional trip verification integrations (airline/rail/bus APIs) can remain manual initially.

Mobile-first Next.js PWA for multi-modal seat swapping (“ultimate version”) with shared seat-map engine + Neon/Postgres backend^{6 STEPS}

1.

Enable auth + scaffold app shell with mock data first (UI-forward build order)

Detailed setup work so the rest of the plan can be implemented without churn, and so the app is immediately usable with mocked content while backend is built.

• Preamble clarification (re: “downloadable zip”)

- This platform builds the app in-place in the repo; the plan below is a portable implementation spec that can be executed here or handed to another coding agent to produce an equivalent codebase. If a zip is required later, it can be produced by exporting the repository directory after implementation.

• Enable authentication (requirement #7)

- Use the project settings tool to enable **Email auth** (magic link or password—choose one; recommend magic link for PWA).
- Define redirect URLs for local/dev/prod (e.g. `/auth/callback`).
- Store provider keys in env (`.env.local`) and add them to deployment environment.

• Next.js App Router structure + design system foundations

- Create/confirm structure under `apps/web/src/app`.
- Install + configure:
 - `@tanstack/react-query` (with devtools in dev)
 - `sonner` for toasts
 - `lucide-react` icons
 - `shadcn/ui` components (Sheet, Tabs, Badge, Button, Card, Dialog, Drawer/Sheet, Toast hook if needed, Input, Select, DropdownMenu, Avatar, Skeleton)
- Ensure global styles match design language:
 - Inter font (via `next/font/google`) in `apps/web/src/app/layout.tsx`
 - Surfaces: white / gray-50; Borders: 1px #E5E7EB; **no resting drop shadows**
 - Pills (badges/chips) for metadata/status/soft actions
 - Tabs: active tab uses overlapping border with `-mb-[1px]` and border color #E5E7EB
 - Action color: restrained blue-600
 - Instant hover transitions (`transition-colors duration-0` or minimal)
 - Mandatory focus rings (Tailwind focus-visible ring)
- Add app-wide providers:

| |
|--|
| <ul style="list-style-type: none"> apps/web/src/app/providers.tsx with QueryClientProvider, Toaster from sonner, and an AuthProvider wrapper if needed. Wire in apps/web/src/app/layout.tsx. |
| • Mock-first route scaffolding (requirement #10) |
| <ul style="list-style-type: none"> Implement initial screens with mock data so UX is visible immediately: <ul style="list-style-type: none"> apps/web/src/app/(auth)/sign-in/page.tsx, sign-up/page.tsx (or magic link page) apps/web/src/app/(app)/trips/page.tsx (list upcoming/past) apps/web/src/app/(app)/trips/add/page.tsx apps/web/src/app/(app)/trips/[tripId]/page.tsx (tabbed detail) |
| <ul style="list-style-type: none"> Add a mock data module: <ul style="list-style-type: none"> apps/web/src/lib/mock/mockData.ts exporting mockTrips, mockOffers, mockSeatMaps (see seed shape in final step) |
| <ul style="list-style-type: none"> Navigation shell: <ul style="list-style-type: none"> Mobile-first top bar + bottom nav (Trips, Inbox, Profile) Use pills for status (Upcoming, Past, Verified) |
| <ul style="list-style-type: none"> Keep all UI reading from mock data through “repository” functions that can later swap to API: <ul style="list-style-type: none"> apps/web/src/lib/data/tripsRepo.ts (mock implementation now; later real fetch) |

2.

Implement shared seat-map engine + per-mode layout configs (flight/bus/train)

Build the core differentiator early: interactive seat maps across three modes, reusing one rendering engine with mode-specific configuration.

| |
|---|
| • Core seat-map data model (shared) |
| <ul style="list-style-type: none"> Create types in apps/web/src/lib/seatmap/types.ts: <ul style="list-style-type: none"> Mode = 'flight' 'bus' 'train' Seat = { id, label, row, col, coachId?, deck?, class?, features: string[], status: 'available' 'occupied' 'blocked', isExitRow?, isAisle?, isWindow?, facing?: 'forward' 'backward' } SeatAssignment = { userId?, passengerLabel, seatId } VehicleLayout = { id, mode, name, meta, decks?: Deck[], coaches?: Coach[], grid?: GridSpec, pointsOfInterest: POI[] } |
| <ul style="list-style-type: none"> Define a “layout config” approach: <ul style="list-style-type: none"> apps/web/src/lib/seatmap/configs/flight.ts, bus.ts, train.ts Each exports buildLayout(input): VehicleLayout and legend, rules (seat labels, aisle gaps, blocked zones). |
| • Rendering components (shared) |

| | | |
|--|--|--|
| • | apps/web/src/components/seatmap/SeatMap.tsx | |
| | | <ul style="list-style-type: none"> • Renders a scrollable/pinchable (mobile) grid (start simple: horizontal/vertical pan with CSS overflow; pinch can be deferred). • Seat cells are buttons with focus ring. • uses no resting shadows; relies on border + subtle bg. |
| • | apps/web/src/components/seatmap/SeatCell.tsx | |
| | | <ul style="list-style-type: none"> • Visual states: available (white), occupied (gray-50), selected (blue-600 border), blocked/POI (gray border). • Pill tags for features (Exit row, Extra legroom). |
| • | apps/web/src/components/seatmap/SeatLegend.tsx | |
| | | <ul style="list-style-type: none"> • Pills taxonomy for seat types. |
| • | Bottom sheet for seat details (requirement #4) | |
| | apps/web/src/components/seatmap/SeatDetailSheet.tsx | using shadcn |
| | Sheet/Drawer: | |
| | | <ul style="list-style-type: none"> • Seat label + metadata pills • Actions: <ul style="list-style-type: none"> • “Mark as my current seat” • “Request swap from this seat” • “Propose swap with occupant” (if offer exists/occupied with public offer) |
| • Per-mode UX requirements mapping | | |
| • | Flight: | |
| | | <ul style="list-style-type: none"> • Cabin classes (economy/premium/business) • Row numbering, 3-3 / 2-4-2 configs, aisles • Exit rows, lavatories, galleys as POIs |
| • | BUS: | |
| | | <ul style="list-style-type: none"> • Driver area POI, 2-2 layout, rear bench variant • Deck selector for double-decker/sleeper (implement selector even if mock only) |
| • | Train: | |
| | | <ul style="list-style-type: none"> • Coach selector strip (horizontal pills) • Seat direction, table-of-four bays, sleepers/berths variant • Multiple coaches and segments rendered via selector |
| • “Trip detail” seat-map tab integration | | |
| • | apps/web/src/app/(app)/trips/[tripId]/_components/TripTabs.tsx | using shadcn Tabs |
| • | Seat Map tab shows SeatMap + legend + bottom sheet. | |
| • | Persist selections in URL or local state: | |
| | e.g. | ?coach=A&deck=upper&seat=12A |

3.

Build swap flows UI (offers, proposals, chat, confirmations, trust) using react-query + optimistic updates (mock-backed initially)

Implement the complete “ultimate version” product flow in the UI layer, still backed by mock repos first, but structured exactly like real API usage.

- Screens/routes to implement (requirement #4 + #5)

- Onboarding/auth

| | |
|---|--|
| | <ul style="list-style-type: none"> apps/web/src/app/(auth)/sign-in/page.tsx apps/web/src/app/(auth)/onboarding/page.tsx (seat prefs + notification prefs) |
| • Trips list (upcoming/past) | |
| | <ul style="list-style-type: none"> apps/web/src/app/(app)/trips/page.tsx |
| • Add trip (booking ref or manual) | |
| | <ul style="list-style-type: none"> apps/web/src/app/(app)/trips/add/page.tsx Form modes: <ul style="list-style-type: none"> Booking reference lookup (mock now; later API) Manual entry (mode + operator + date/time + origin/dest + seat) |
| • Trip detail tabs | |
| | <ul style="list-style-type: none"> Seat Map / Offers / My Swaps |
| • Offers browsing & filtering | |
| | <ul style="list-style-type: none"> apps/web/src/app/(app)/trips/[tripId]/offers/page.tsx (or inside tab panel) Filters as pill toggles: <ul style="list-style-type: none"> “Window”, “Aisle”, “Extra legroom”, “Near front”, “Together”, “Sleeper/Lower berth” “Verified only” “Expiring soon” |
| • Proposal inbox | |
| | <ul style="list-style-type: none"> apps/web/src/app/(app)/inbox/page.tsx Sections: Received / Sent |
| • In-app chat per proposal (requirement #5) | |
| | <ul style="list-style-type: none"> apps/web/src/app/(app)/inbox/[proposalId]/page.tsx Message list + composer; include system messages (status changes) |
| • Swap confirmation + share pass | |
| | <ul style="list-style-type: none"> apps/web/src/app/(app)/swaps/[swapId]/confirmed/page.tsx Show both seats pre/post, QR, share button |
| • Profile/preferences | |
| | <ul style="list-style-type: none"> apps/web/src/app/(app)/profile/page.tsx Seat prefs profile (used for auto suggestions) Notification Settings Swap history |
| • Trust/reputation view | |
| | <ul style="list-style-type: none"> apps/web/src/app/(app)/profile/trust/page.tsx Circular SVG data rings for stats (completion rate, response time) Badges: Verified traveler, High reliability |
| • “Ultimate version” logic surfaced in UI (even before backend) | |
| • Smart match scoring (requirement #5) | |
| | <ul style="list-style-type: none"> Implement client-side scoring function now in apps/web/src/lib/match/score.ts: <ul style="list-style-type: none"> Inputs: requester preferences, offered seat features, distance to desired area, trust score, expiry |

| | |
|--|---|
| | <ul style="list-style-type: none"> Output: numeric 0–100 + reasons list (hyphen-prefixed technical list in UI) |
| | <ul style="list-style-type: none"> Offers list sorted by score; show “Why this match?” drawer. |
| • Group swaps / 3-way chains | |
| | <ul style="list-style-type: none"> UI entry in “Create request”: <ul style="list-style-type: none"> “I’m swapping as a group” with passenger count and desired adjacency |
| | <ul style="list-style-type: none"> In Offers tab: <ul style="list-style-type: none"> “Chain opportunities” section (mock suggestions) |
| • Sweetener/credit system (non-payment points) | |
| | <ul style="list-style-type: none"> Add optional “Offer credits” input (0–N) shown as a pill (blue-600 outline) Ensure copy avoids money language. |
| • Real-time-ish polling | |
| | <ul style="list-style-type: none"> In react-query hooks, enable <code>refetchInterval</code> (e.g. 10–20s) on offers/proposals when in foreground. |
| • Countdown to departure + auto-expiry | |
| | <ul style="list-style-type: none"> Display countdown pill in trip header Mark stale offers as expired in UI (based on <code>expiresAt</code>) |
| • Toasts + optimistic updates | |
| | <ul style="list-style-type: none"> Hooks under <code>apps/web/src/lib/api/hooks/*</code> (even if mock): <ul style="list-style-type: none"> <code>useCreateSwapRequest</code>, <code>useSendProposal</code>,
<code>useAcceptProposal</code>, <code>useDeclineProposal</code>, <code>useSendMessage</code> Implement optimistic: <ul style="list-style-type: none"> Sending proposal: optimistic insert into proposals list Accept: optimistic status to “accepted/pending confirmation”; rollback on error Use sonner to toast success/error. |
| • Component library (design system adherence) | |
| | <ul style="list-style-type: none"> Create reusable primitives: <ul style="list-style-type: none"> <code>apps/web/src/components/ui/Pill.tsx</code> (wrap shadcn Badge with variants) <code>apps/web/src/components/ui/MetaRow.tsx</code> (hyphen-prefixed lists) <code>apps/web/src/components/stats/DataRing.tsx</code> (circular SVG ring) Ensure focus-visible rings and no shadow defaults. |

4.

Design PostgreSQL schema in Neon (users, trips, layouts, seats, swaps, proposals, messages, reputation) + seed data

Move to real persistence with a schema that matches the UI flows and supports atomic swaps + group chains.

| | |
|-------------------------|--|
| • Database + migrations | |
| | <ul style="list-style-type: none"> Use the platform’s standard Postgres migration approach (e.g. Drizzle/Prisma/Knex—pick what the repo already uses; if none exists, choose drizzle for App Router compatibility). Create schema files (example if Drizzle): <ul style="list-style-type: none"> <code>apps/web/src/db/schema.ts</code> |

| | |
|--|---|
| <ul style="list-style-type: none"> apps/web/src/db/migrations/* apps/web/src/db/index.ts (Neon connection) | |
| Entities/tables (requirement #6) | |
| users | <ul style="list-style-type: none"> id (uuid pk), email (unique), created_at |
| profiles | <ul style="list-style-type: none"> user_id (pk/fk), display_name, avatar_url, seat_prefs_json, notification_prefs_json, verified_level, trust_score_cached |
| trips | <ul style="list-style-type: none"> id, mode, operator, number, origin, destination, departure_at, arrival_at, booking_ref_hash?, status |
| trip_members | <ul style="list-style-type: none"> id, trip_id, user_id, passenger_label, role ('self' 'dependent') |
| vehicle_layouts | <ul style="list-style-type: none"> id, mode, name, layout_json (stores normalized layout config), version |
| seats | <ul style="list-style-type: none"> id, layout_id, seat_code (e.g. 12A), coach, deck, row, col, features_json, class, facing |
| seat_assignments | <ul style="list-style-type: none"> id, trip_id, seat_id, trip_member_id nullable, is_current boolean, assigned_at Unique constraint: (trip_id, seat_id) to prevent double-occupancy |
| swap_requests | <ul style="list-style-type: none"> id, trip_id, trip_member_id, from_seat_assignment_id, desired_json (constraints like window/aisle, adjacency, coach, etc.) group_size, credits_offered, expires_at, status ('open' 'paused' 'expired' 'fulfilled' 'cancelled') |
| swap_proposals | <ul style="list-style-type: none"> id, trip_id, from_request_id, to_request_id, proposed_map_json (supports 2-way or chain mapping) score, status ('sent' 'accepted' 'declined' 'cancelled' 'expired' 'confirmed') |

| | |
|---|--|
| | <ul style="list-style-type: none"> created_at, updated_at |
| <ul style="list-style-type: none"> proposal_messages | <ul style="list-style-type: none"> id, proposal_id, sender_user_id nullable (null for system), body, kind ('user' 'system'), created_at |
| <ul style="list-style-type: none"> swap_confirmations | <ul style="list-style-type: none"> id, proposal_id, trip_id, confirmation_code, qr_payload, confirmed_at |
| <ul style="list-style-type: none"> reputation_events | <ul style="list-style-type: none"> id, user_id, trip_id, type, delta, meta_json, created_at |
| <ul style="list-style-type: none"> Transactional needs | |
| | <ul style="list-style-type: none"> Seat swap acceptance must atomically: <ul style="list-style-type: none"> Validate proposal still valid (not expired, seats still assigned) Update seat_assignments for all involved (2-way or chain) Create swap_confirmations Insert reputation events |
| <ul style="list-style-type: none"> Seed/fixtures | |
| | <ul style="list-style-type: none"> Add a seed script (or API dev-only endpoint) to populate: <ul style="list-style-type: none"> 1 flight trip + layout (e.g. A320 3-3) 1 bus trip + layout (2-2 + rear bench) 1 train trip + multi-coach layout (coach A/B, table bays) Seats + assignments for ~20–40 seats each 5–10 swap requests and proposals, including: <ul style="list-style-type: none"> At least one 3-way chain example Some expiring soon / expired Some verified users |
| <ul style="list-style-type: none"> Files: | |
| | <ul style="list-style-type: none"> apps/web/src/db/seed.ts (OR scripts/seed.ts) |

5.

Implement REST-ish API routes under App Router (/api) including transactional swap acceptance + chain swaps

Create backend endpoints to support full CRUD plus special transactional actions. Keep route organization consistent and predictable.

| | |
|---|---|
| <ul style="list-style-type: none"> API route conventions (requirement #6) | <ul style="list-style-type: none"> Place routes under: apps/web/src/app/api/**/route.ts Implement GET (list), POST (create) at collection routes; GET, PATCH, DELETE at item routes. Return consistent JSON envelopes: <ul style="list-style-type: none"> { data, error } and proper HTTP codes. |
| <ul style="list-style-type: none"> CRUD endpoints (minimum set to power app) | |
| | <ul style="list-style-type: none"> Users/profiles |

| | |
|--------------------------------------|---|
| | <ul style="list-style-type: none"> GET /api/me PATCH /api/me/profile |
| • Trips | <ul style="list-style-type: none"> GET /api/trips?status=upcoming past POST /api/trips (manual create) GET /api/trips/:tripId |
| • Seat maps / assignments | <ul style="list-style-type: none"> GET /api/trips/:tripId/layout (layout + seats) GET /api/trips/:tripId/assignments POST /api/trips/:tripId/assignments (mark current seat) |
| • Swap requests | <ul style="list-style-type: none"> GET /api/trips/:tripId/requests POST /api/trips/:tripId/requests PATCH /api/requests/:requestId (pause/cancel) |
| • Proposals | <ul style="list-style-type: none"> GET /api/trips/:tripId/proposals?box=inbox sent POST /api/proposals (send proposal) PATCH /api/proposals/:proposalId (decline/cancel) |
| • Proposal messages (chat) | <ul style="list-style-type: none"> GET /api/proposals/:proposalId/messages POST /api/proposals/:proposalId/messages |
| • Transactional endpoints (critical) | <ul style="list-style-type: none"> Accept proposal (2-way or chain) atomically <ul style="list-style-type: none"> POST /api/proposals/:proposalId/accept Implementation notes: <ul style="list-style-type: none"> Use a DB transaction. Lock relevant rows (seat assignments / proposal row) to prevent double-accept. Validate <code>expires_at</code> for involved requests and trip departure cutoff. Update proposal status → accepted/confirmed (depending on whether immediate confirm is allowed). Apply seat assignment swaps based on <code>proposed_map_json</code>: <ul style="list-style-type: none"> map: { tripMemberId -> newSeatId } Create <code>swap_confirmations</code> record and generate: <ul style="list-style-type: none"> <code>confirmation_code</code> (short, human readable) <code>qr_payload</code> (JSON string with tripId, confirmationId, seats, timestamp, signature if desired) |

| | |
|-------------------------------|--|
| | <ul style="list-style-type: none"> Add <code>proposal_messages</code> system message: "Swap confirmed." Insert <code>reputation_events</code> for all participants. |
| • Group/chain swap generation | |
| | <ul style="list-style-type: none"> POST <code>/api/trips/:tripId/chains/suggest</code> |
| | <ul style="list-style-type: none"> Input: group request id; output: suggested chain proposals with computed scores. In v1, can be heuristic (graph search limited depth=3). |
| • Auth enforcement in API | |
| | <ul style="list-style-type: none"> Ensure all routes read the authenticated user (from enabled email auth provider). Return 401 when missing; ensure row-level filtering (only see proposals/messages you participate in). |

6.

Wire real data with react-query, replace mock repos, add PWA shell (manifest + service worker + offline shell)

Finalize by connecting UI to the API + adding installable PWA behavior.

| | |
|--|---|
| • Replace mock repos with real API clients | |
| • Create fetcher utilities: | |
| | <ul style="list-style-type: none"> <code>apps/web/src/lib/api/client.ts</code> (fetch wrapper, JSON parsing, error normalization) |
| • react-query hooks: | |
| | <ul style="list-style-type: none"> <code>apps/web/src/lib/api/hooks/useTrips.ts</code> <code>useTrip.ts</code>, <code>useTripLayout.ts</code>, <code>useAssignments.ts</code> <code>useSwapRequests.ts</code>, <code>useCreateSwapRequest.ts</code> <code>useProposals.ts</code>, <code>useSendProposal.ts</code>, <code>useAcceptProposal.ts</code>, <code>useMessages.ts</code>, <code>useSendMessage.ts</code> |
| • Keep optimistic updates for: | |
| | <ul style="list-style-type: none"> sending proposal accepting/declining (with rollback + sonner toast) |
| • Enable "real-time-ish" polling: | |
| | <ul style="list-style-type: none"> Offers/requests: <code>refetchInterval: 15000</code> when trip page active Inbox: <code>refetchInterval: 10000</code> |
| • Smart match scoring: server + client alignment | |
| | <ul style="list-style-type: none"> Keep client scoring for UI ranking responsiveness. Store <code>score</code> on <code>swap_proposals</code> (and/or computed suggestions) so ordering is consistent across clients. |
| • PWA implementation (requirement #1) | |
| • Add manifest: | |
| | <ul style="list-style-type: none"> <code>apps/web/public/manifest.json</code> with: <ul style="list-style-type: none"> <code>name</code>, <code>short_name</code>, <code>start_url</code>, <code>display: "standalone"</code>, |

| | |
|---|--|
| | <ul style="list-style-type: none"> background_color: "#ffffff", theme_color (use restrained blue or neutral) icons (192/512, maskable) |
| • App icons: | <ul style="list-style-type: none"> apps/web/public/icons/icon-192.png, icon-512.png, maskable-512.png (placeholders acceptable initially) |
| • Service worker + offline shell | <ul style="list-style-type: none"> Use next-pwa or a lightweight Workbox setup: <ul style="list-style-type: none"> Cache app shell routes and static assets. Network-first for API; fallback to cached last-known data for trips/offers list if available. Ensure offline behavior: <ul style="list-style-type: none"> If offline, show a banner pill “Offline mode” and allow browsing cached seat maps and last fetched offers. |
| • Add-to-home-screen prompt handling | <ul style="list-style-type: none"> Client-only hook apps/web/src/lib/pwa/useA2HS.ts Show a bottom sheet CTA after 2–3 meaningful interactions (trip viewed) if beforeinstallprompt fired. |
| • Verify: | <ul style="list-style-type: none"> Lighthouse PWA installability (manifest + SW + icons) Standalone display works and theme color applies |
| • Seed/mock data shape (so app looks populated immediately) | <ul style="list-style-type: none"> Keep mockData.ts for dev fallback and visual regression; shape should match API DTOs: <ul style="list-style-type: none"> TripDTO: { id, mode, operator, number, origin, destination, departureAt, arrivalAt, status, myMemberId, countdownSeconds } LayoutDTO: { layoutId, mode, name, decks?: string[], coaches?: {id,name}[], seats: SeatDTO[], pois: POIDTO[] } SeatDTO: { id, seatCode, row, col, coach?, deck?, class?, features: string[], status } SwapRequestDTO: { id, tripId, memberId, fromSeatCode, desired: { window?, aisle?, together?: {size, adjacent} , coach?, deck?, notes? }, creditsOffered, expiresAt, status } SwapProposalDTO: { id, tripId, fromRequestId, toRequestId, score, status, createdAt, proposedMap: Record<memberId, seatCode> } MessageDTO: { id, proposalId, senderUserId null, kind, body, createdAt } ConfirmationDTO: { id, tripId, proposalId, confirmationCode, qrPayload, confirmedAt } |